

Занятие 9 – Карточная игра. Добавление «фишек»

По итогу получилась карточная игра, в которую уже сейчас можно играть. Однако каждый разработчик знает, что игры нужно со временем совершенствовать, добавлять новые «фишки».

Расширим игру, а именно для начала увеличим колоду в несколько раз для более продолжительной игры. Добавим вариативности, создадим реквизит «ДолгаяИгра» с типом «Булево» (рис. 1.9.1). Если данный реквизит будет в положении «Истина», в колоде будет больше карт. Так игрок сможет выбрать между быстрой или долгой игрой.

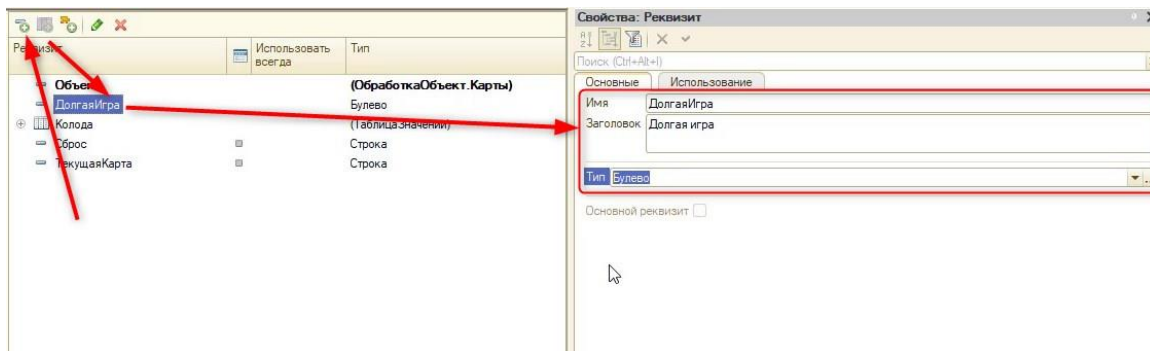


Рис. 1.9.1. Создание нового реквизита

Перенесём реквизит на форму, чтобы пользователь мог с ним взаимодействовать (рис. 1.9.2). Для того чтобы переместить элемент на форме вверх, воспользуйтесь синими стрелками «вверх» и «вниз».

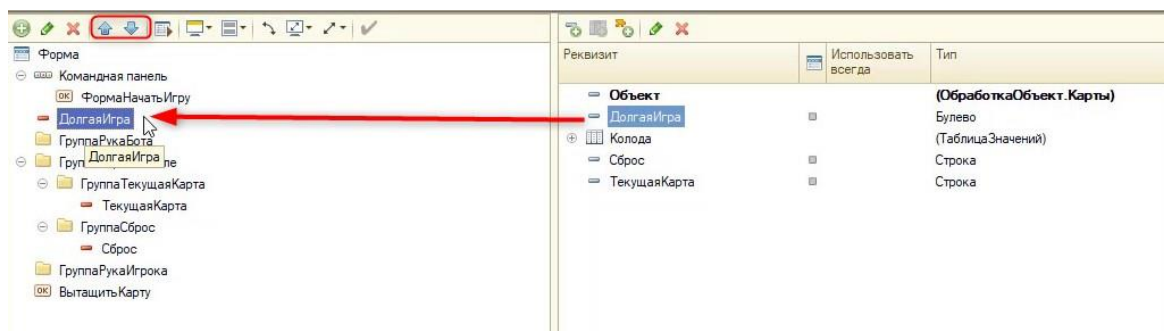


Рис. 1.9.2. Добавление элемента

Перейдем в модуль формы обработки. Необходимо выделить код (см. рис. 1.9.3) с запросом и перемешиванием. После этого вырезать его в буфер обмена (горячая клавиша «Ctrl+X») и удалить из процедуры.

Или же можно нажать по выделенному коду правой кнопкой мыши и выбрать «Вырезать», код автоматически сохранится в буфере обмена.

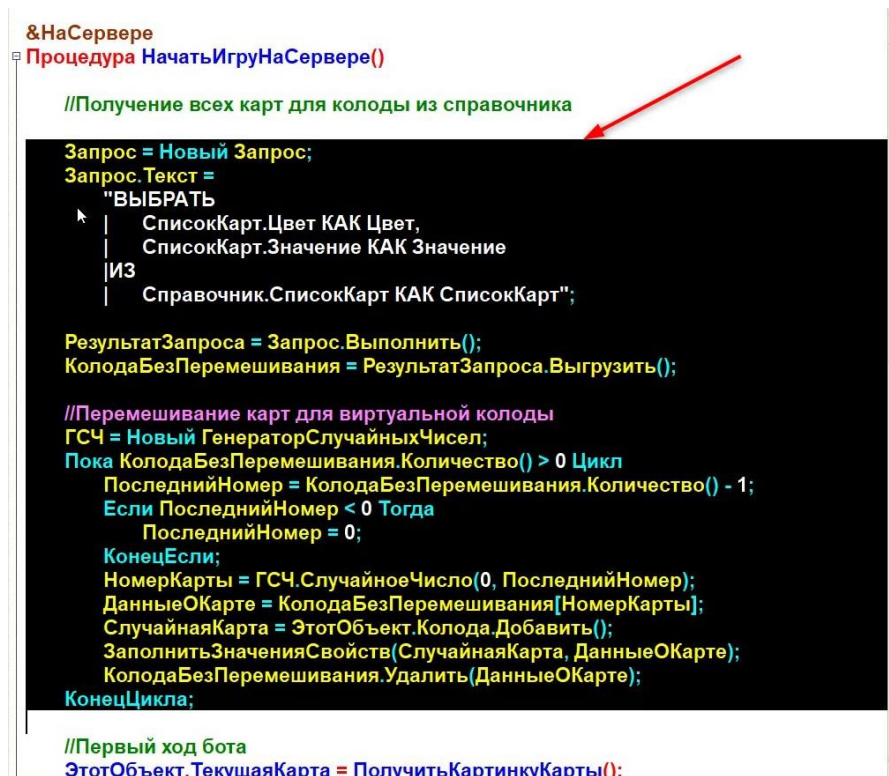


Рис. 1.9.3. Строки кода для переноса

После этого ниже определим новую серверную процедуру – «ПодготовитьКолоду» и подставим внутрь нее вырезанный код (с помощью горячей клавиши «Ctrl+V»).

Новая процедура будет выглядеть следующим образом:

```

&НаСервере
Процедура ПодготовитьКолоду()
    Запрос = Новый Запрос;
    Запрос.Текст =
        "ВЫБРАТЬ
        |     СписокКарт.Цвет КАК Цвет,
        |     СписокКарт.Значение КАК Значение
        |ИЗ
        |     Справочник.СписокКарт КАК СписокКарт";

    РезультатЗапроса = Запрос.Выполнить();
    КолодаБезПеремешивания = РезультатЗапроса.Выгрузить();
    //Перемешивание карт для виртуальной колоды
    ГСЧ = Новый ГенераторСлучайныхЧисел;
    Пока КолодаБезПеремешивания.Количество() > 0 Цикл
        ПоследнийНомер = КолодаБезПеремешивания.Количество() - 1;
        Если ПоследнийНомер < 0 Тогда
            ПоследнийНомер = 0;
        КонецЕсли;
        НомерКарты = ГСЧ.СлучайноеЧисло(0, ПоследнийНомер);
        ДанныеОКарте = КолодаБезПеремешивания[НомерКарты];
  
```

```

        СлучайнаяКарта = ЭтотОбъект.Колода.Добавить();
        ЗаполнитьЗначенияСвойств(СлучайнаяКарта, ДанныеОКарте);
        КолодаБезПеремешивания.Удалить(ДанныеОКарте);
    КонечЦикла;
КонечПроцедуры

```

Далее необходимо вызвать данную процедуру до первого хода бота (раздачи карт на руки) в процедуре «НачатьИгруНаСервере». Перед этим проверим, какую игру по продолжительности выбрал пользователь через условие «Если»:

&НаСервере

Процедура НачатьИгруНаСервере()

//Получение всех карт для колоды из справочника, вызываем новую процедуру

ПодготовитьКолоду();

//Проверка на выбор пользователя

Если ЭтотОбъект.ДолгаяИгра = Истина Тогда

ПодготовитьКолоду();

ПодготовитьКолоду();

КонецЕсли;

//Первый ход бота

ЭтотОбъект.ТекущаяКарта = ПолучитьКартинкуКарты();

Элементы.ТекущаяКарта.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;

ЭтотОбъект.Колода.Удалить(0);

//Заполнение руки игрока начальными картами

Для Счетчик = 1 По 7 Цикл

 ДобавитьКартуНаПоле(Счетчик, Элементы.ГруппаРукаИгрока);

КонецЦикла;

//Заполнение руки бота начальными картами

Для Счетчик = 8 По 14 Цикл

 ДобавитьКартуНаПоле(Счетчик, Элементы.ГруппаРукаБота);

КонецЦикла;

КонецПроцедуры

Обязательно проверим работоспособность кода в пользовательском режиме.

Теперь игра будет идти дольше при активном флаге (галочке) долгой игры (рис. 1.9.4).

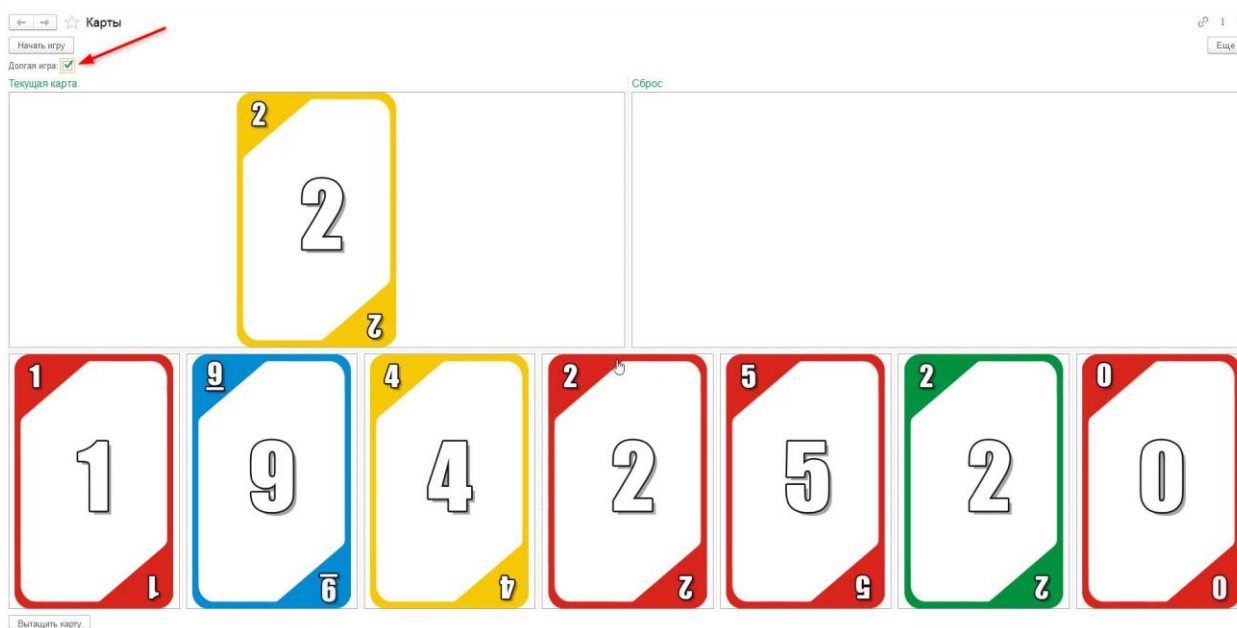


Рис. 1.9.4. Интерфейс игры с новым механизмом

На этом доработка с увеличением колоды закончена.

Следующее, что необходимо реализовать, – это контроль карт, которые были выданы на руку игроку или боту в самом начале. Ведь с увеличением колоды может получиться так, что первоначально у игрока вся рука будет состоять из карт одного цвета (рис. 1.9.5).

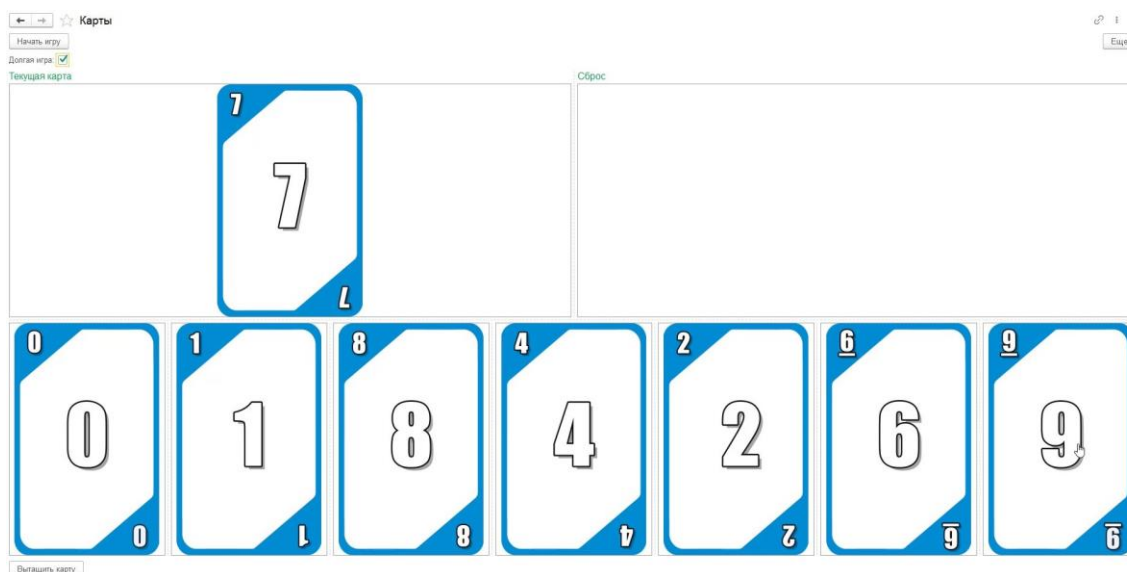


Рис. 1.9.5. Одинаковый цвет карт на руке

Необходимо это контролировать. В случае, если это произойдет, – все очищать и выводить уведомление пользователю.

В модуле формы, в процедуре «НачатьИгруНаСервере», после заполнения рук игрока и компьютера реализуем проверку с помощью отдельной процедуры. Предварительно вызовем ее для отдельной проверки после раздачи карт игроку и боту:

```
&НаСервере
```

```
Процедура НачатьИгруНаСервере()
```

```
    //Получение всех карт для колоды из справочника
```

```
    ПодготовитьКолоду();
```



```

Если ЭтотОбъект.ДолгаяИгра = Истина Тогда
    ПодготовитьКолоду();
    ПодготовитьКолоду();
КонецЕсли;
//Первый ход бота
ЭтотОбъект.ТекущаяКарта = ПолучитьКартинкуКарты();
Элементы.ТекущаяКарта.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;
ЭтотОбъект.Колода.Удалить(0);
//Заполнение руки игрока начальными картами
Для Счетчик = 1 По 7 Цикл
    ДобавитьКартуНаПоле(Счетчик, Элементы.ГруппаРукаИгрока);
КонецЦикла;
//Вызовем новую процедуру и передадим название группы для проверки:
ПроверитьРуку(Элементы.ГруппаРукаИгрока);
//Заполнение руки бота начальными картами
Для Счетчик = 8 По 14 Цикл
    ДобавитьКартуНаПоле(Счетчик, Элементы.ГруппаРукаБота);
КонецЦикла;
//Проверим руку бота:
ПроверитьРуку(Элементы.ГруппаРукаБота);
КонецПроцедуры

```

Новая процедура будет серверной, так как в ней будет использоваться таблица значений, взаимодействовать с которой можно только на сервере. **Таблица значений** – это многоуровневая универсальная коллекция значений, позволяющая обрабатывать огромные объемы данных. Пример такой таблицы показан на рис. 1.9.6.

НомерСтроки	Товар	Цена	Количество	Сумма
0	«Чайник»	100	2	200
1	«Ложка»	20	3	60
2	«Вилка»	19	3	57

Рис. 1.9.6. Таблица значений

У таблицы значений есть один очень важный метод – «Свернуть()». Он позволяет сгруппировать всю таблицу по определенной колонке. Например, по дублирующимся товарам в таблице на рис. 1.9.7.

Товар	Количество	Свернуть()	Товар	Количество
«Чайник»	2		«Чайник»	2
«Ложка»	3		«Ложка»	3
«Вилка»	3		«Вилка»	5
«Вилка»	2			

Рис. 1.9.7. Сворачивание таблицы значений

В нашей ситуации все будет просто, так как нужно сворачивать таблицу из одной колонки (рис. 1.9.8). И если колода будет одного цвета, то таблица значений после сворачивания будет иметь только одну строку.



Рис. 1.9.8. Сворачивание таблицы значений в программной логике игры

Создадим серверную процедуру «ПроверитьРуку» в модуле формы:

&НаСервере

Процедура ПроверитьРуку(Группа)

```

ТаблицаЦветов = Новый ТаблицаЗначений;
ТаблицаЦветов.Колонки.Добавить("Цвет");
//Переберем все карты из руки игроков и запишем в созданную таблицу значений:
Для Каждого Элемент Из Группа.ПодчиненныеЭлементы Цикл
    НомерПробела = СтрНайти(Элемент.Заголовок, " ");
    НазваниеЦвета = Лев(Элемент.Заголовок, НомерПробела - 1);
    НоваяСтрока = ТаблицаЦветов.Добавить();
    НоваяСтрока.Цвет = НазваниеЦвета;
КонецЦикла;
//Уберем все дубли в таблице:
ТаблицаЦветов.Свернуть("Цвет");

//Обработаем ситуацию, когда на руке один цвет карт:
Если ТаблицаЦветов.Количество() = 1 Тогда
    Сообщить("Колода у " + Группа.Заголовок + " состоит из одного цвета!");
    Сообщить("Начни игру для продолжения!");
    //Очистим форму и уберем видимость элементов:
    ОчиститьФорму();
    Элементы.ВытащитьКарту.Видимость = Ложь;
    Элементы.ГруппаИгровоеПоле.Видимость = Ложь;
    Элементы.ГруппаРукаИгрока.Видимость = Ложь;
КонецЕсли;

```

КонецПроцедуры

Для того чтобы протестировать данный механизм в пользовательском режиме, придется начать игру несколько раз.

На данный момент сообщение выглядит не совсем корректно. Так как пользователь не участвует в разработке и не понимает, что такое «Группа рука игрока» (рис. 1.9.9).

Сообщения:

- Колода у Группы рука игрока состоит из одного цвета!
- Начни игру для продолжения!

Рис. 1.9.9. Предупреждение

Отредактируем заголовки в режиме конфигурирования у группы руки бота и игрока (рис. 1.9.10 – 1.9.11).

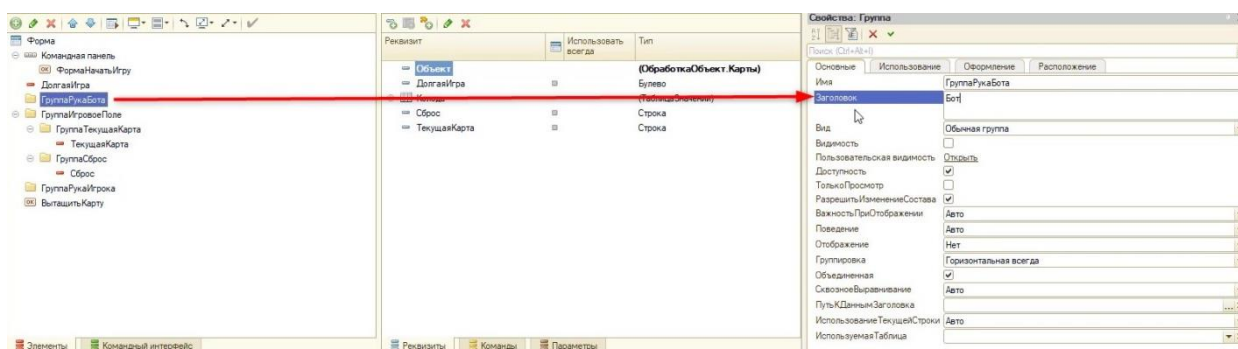


Рис. 1.9.10. Смена заголовка руки бота

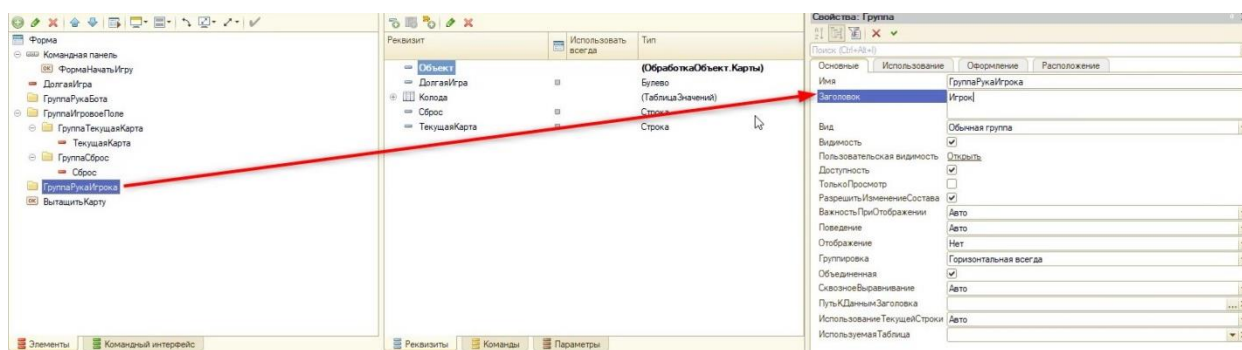


Рис. 1.9.11. Смена заголовка руки игрока

Теперь сообщение в пользовательском режиме будет выглядеть иначе.

Немаловажная доработка, которую необходимо сделать, – пропуск хода игрока. Реализуем ее в виде кнопки снизу.

Для этого создадим новую команду «ПропуститьХод» и перенесем ее на форму (рис. 1.9.12).

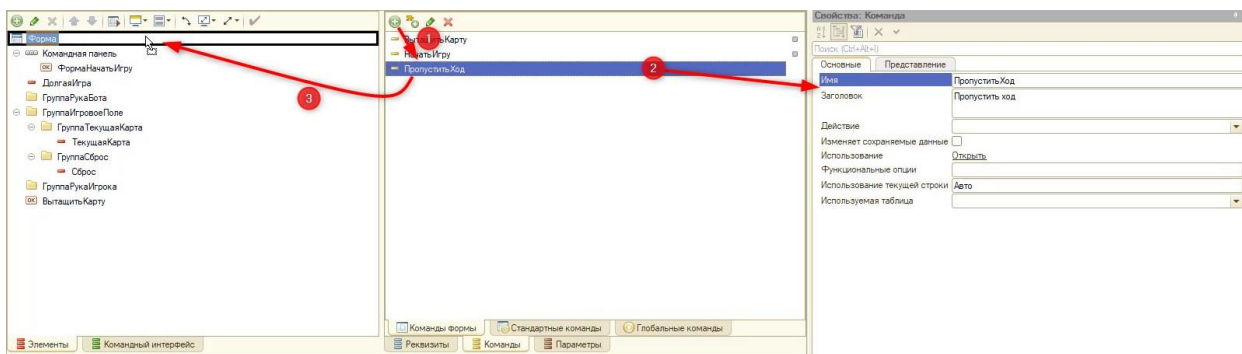


Рис. 1.9.12. Создание новой команды

Изначально отключим видимость кнопки (рис. 1.9.13).

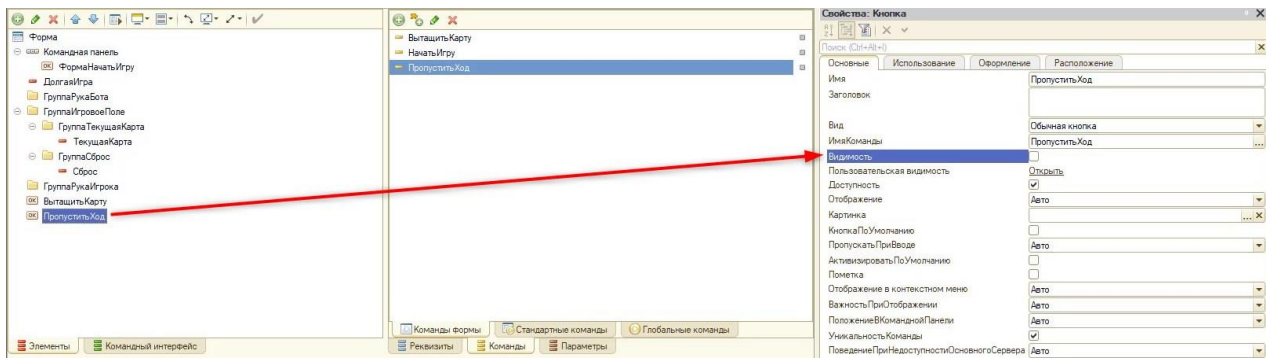


Рис. 1.9.13. Отключение видимости

Кнопка будет появляться только при старте игры. В связи с этим добавим включение видимости кнопки в процедуру «НачатьИгру» в модуле формы обработки и очистим все сообщения, которые были выведены ранее:

&НаКлиенте

Процедура НачатьИгру(Команда)

ОчиститьСообщения();

ОчиститьФорму();

ЭтотОбъект.Сброс = "";

Элементы.ГруппаРукаИгрока.Доступность = Истина;

Элементы.ГруппаИгровоеПоле.Доступность = Истина;

Элементы.ВытащитьКарту.Видимость = Истина;

Элементы.ГруппаИгровоеПоле.Видимость = Истина;

Элементы.ПропуститьХод.Видимость = Истина;

НачатьИгруНаСервере();

КонецПроцедуры

Опишем действие для новой команды «Пропустить ход» (рис. 1.9.14).

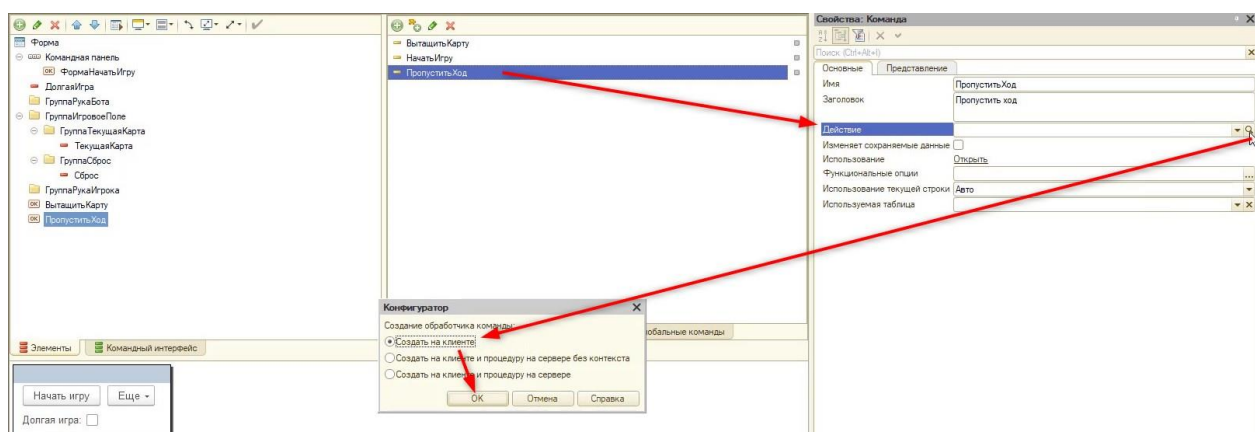


Рис. 1.9.14. Создание обработчика команды

Код команды будет выглядеть следующим образом:

&НаКлиенте

Процедура ПропуститьХод(Команда)

//Выключим видимость кнопки, пока бот не сделает ход:

Элементы.ПропуститьХод.Видимость = Ложь;

//Вызовем процедуру "Ход бота" с задержкой:

ПодключитьОбработчикОжидания("ХодБота", 1, Истина);
КонецПроцедуры

Теперь необходимо подкорректировать процедуру «ХодБота»:

&НаКлиенте

Процедура ХодБота()

ХодБыл = Ложь;

МассивДанныхОКартеНаСтоле = СтрРазделить(Элементы.ТекущаяКарта.Заголовок, " ", Ложь);

//Считаем количество карт в руке бота

КартУБота = 0;

Для Каждого Элемент Из Элементы.ГруппаРукаБота.ПодчиненныеЭлементы Цикл

Если Элемент.Видимость = Истина Тогда

КартУБота = КартУБота + 1;

КонецЕсли;

КонецЦикла;

//Перебор карт из руки

Для Каждого Карта Из Элементы.ГруппаРукаБота.ПодчиненныеЭлементы Цикл

Если Карта.Видимость = Истина Тогда

МассивДанныхОКартеБота = СтрРазделить(Карта.Заголовок, " ", Ложь);

//Если может сыграть карту

Если МассивДанныхОКартеБота[0] = МассивДанныхОКартеНаСтоле[0] ИЛИ
МассивДанныхОКартеБота[1] = МассивДанныхОКартеНаСтоле[1] Тогда

СыгратьКарту(Карта.Имя);

Карта.Видимость = Ложь;

КартУБота = КартУБота - 1;

ЭтотОбъект.Заголовок = "Бот: Готово!";

//Включим видимость кнопки после того, как бот сделал свой ход:

Элементы.ПропуститьХод.Видимость = Истина;

//Если выложил последнюю карту из руки

Если КартУБота = 0 Тогда

Элементы.ГруппаИгровоеПоле.Доступность = Ложь;

Элементы.ГруппаРукаИгрока.Доступность = Ложь;

ПредупреждениеАсинх("Бот: Я выиграл!");

//Выключим видимость кнопки, если бот выиграл:

Элементы.ПропуститьХод.Видимость = Ложь;

КонецЕсли;

ХодБыл = Истина;

Прервать;

Иначе

//Если карта перебора не подошла

ЭтотОбъект.Заголовок = "Бот: Думаю...";

КонецЕсли;

```

        КонечЕсли;
КонечЦикла;
//Если ни одна карта не подошла
Если ХодБыл = Ложь Тогда
    //Если бот не ходил, то пытается взять карту из колоды
    Если ВытащитьКартуНаСервере(Элементы.ГруппаРукаБота.Имя) = Истина Тогда
        Сообщить("Бот: Возьму еще карту!");
        ХодБота();
    Иначе //Если не получается - проигрыш бота
        Элементы.ГруппаИгровоеПоле = Доступность = Ложь;
        Элементы.ГруппаРукаИгрока = Доступность = Ложь;
        ПредупреждениеАсинх("Бот: Карты в колоде закончились! Я проиграл!");
        КонечЕсли;
    КонечЕсли;
КонечПроцедуры

```

Добавим последний механизм, который даст возможность игроку подглядеть карты, которые находятся у бота. Чтобы игрок не злоупотреблял этим, сделаем такую возможность разовой за всю игру и только на 5 секунд.

Создадим новую команду – «ПодглядетьКарты» (рис. 1.9.15) и перенесем ее на форму.

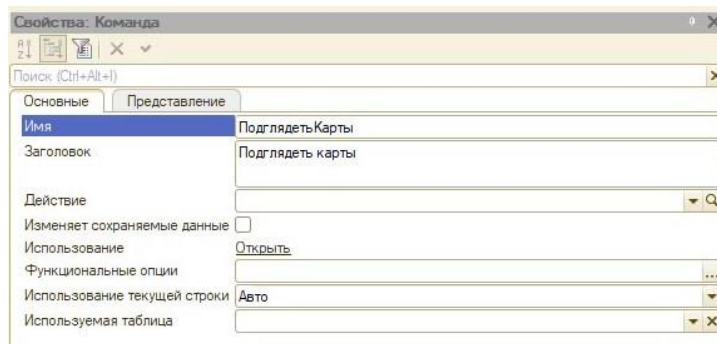


Рис. 1.9.15. Создание новой команды

Изначально выключим видимость новой кнопки на форме (рис. 1.9.16).

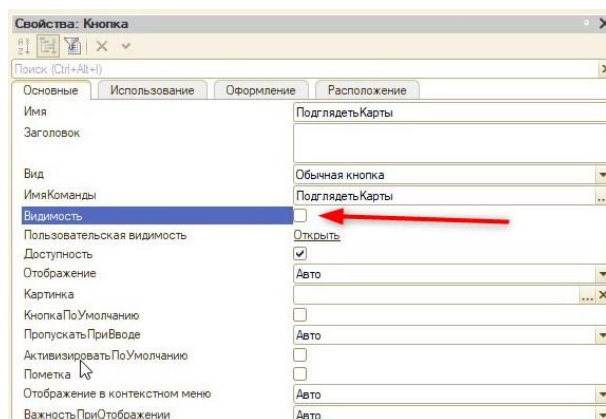


Рис. 1.9.16. Выключение видимости у кнопки

В процедуре «НачатьИгру» включим видимость кнопки:

```

&НаКлиенте

```

```

Процедура НачатьИгру(Команда)
    ОчиститьСообщения();
    ОчиститьФорму();
    ЭтотОбъект.Сброс = "";
    Элементы.ГруппаРукаИгрока.Доступность = Истина;
    Элементы.ГруппаИгровоеПоле.Доступность = Истина;
    Элементы.ВытащитьКарту.Видимость = Истина;
    Элементы.ГруппаИгровоеПоле.Видимость = Истина;
    Элементы.ПропуститьХод.Видимость = Истина;
    Элементы.ПодглядетьКарты.Видимость = Истина;
    НачатьИгруНаСервере();
КонецПроцедуры

```

Опишем действие для новой команды, как делали это ранее:

```

&НаКлиенте
Процедура ПодглядетьКарты(Команда)
    //Выключим видимость кнопки:
    Элементы.ПодглядетьКарты.Видимость = Ложь;
    //Включим видимость руки бота:
    Элементы.ГруппаРукаБота.Видимость = Истина;
    //Создадим новую процедуру и вызовем её через 5 секунд (она будет выключать
    видимость руки бота)
    ПодключитьОбработчикОжидания("СпрятатьКарты", 5, Истина);
КонецПроцедуры

```

Добавим новую процедуру, которую вызвали с задержкой в обработчике команды:

```

&НаКлиенте
Процедура СпрятатьКарты()
    Элементы.ГруппаРукаБота.Видимость = Ложь;
КонецПроцедуры

```

Готово! Осталось протестировать новые механизмы на наличие ошибок и можно играть.

Занятие 10 – Игра «Квест»

Следующая на очереди игра, которую мы создадим, – это квест. Квестовая игра – это интерактивная история с главным героем, которым управляет игрок. Важнейшими элементами игры в жанре квеста являются повествование и исследование мира, а ключевая роль в игровом процессе отводится наличию выбора, который влияет на сюжет.

Квест, созданный в этом модуле, будет настраиваемым. Пользователь сам сможет настраивать картинки, текст и точку выбора с её разветвлениями в настройках.

Первостепенно нужно реализовать механизм настройки квеста, т.е. «Каталог историй», где пользователь сможет добавлять собственный сюжет и настраивать его.

Для начала создадим новый справочник с названием «Каталог историй» (рис. 1.10.1).

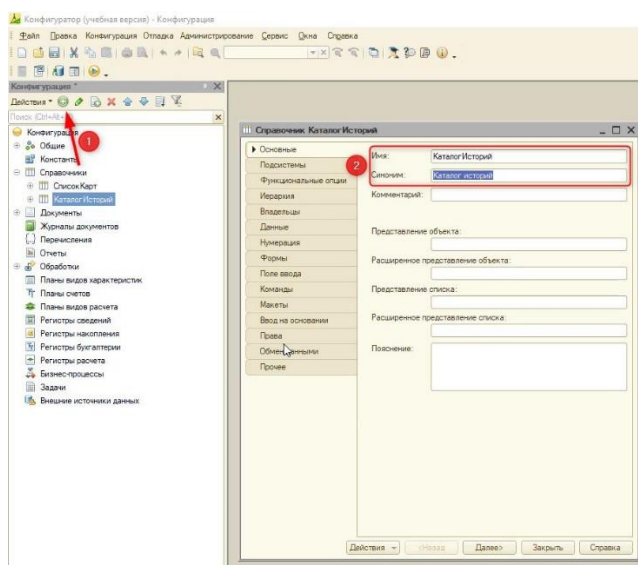


Рис. 1.10.1. Создание нового справочника

Далее на вкладке «Данные» увеличим длину наименования (рис. 1.10.2).

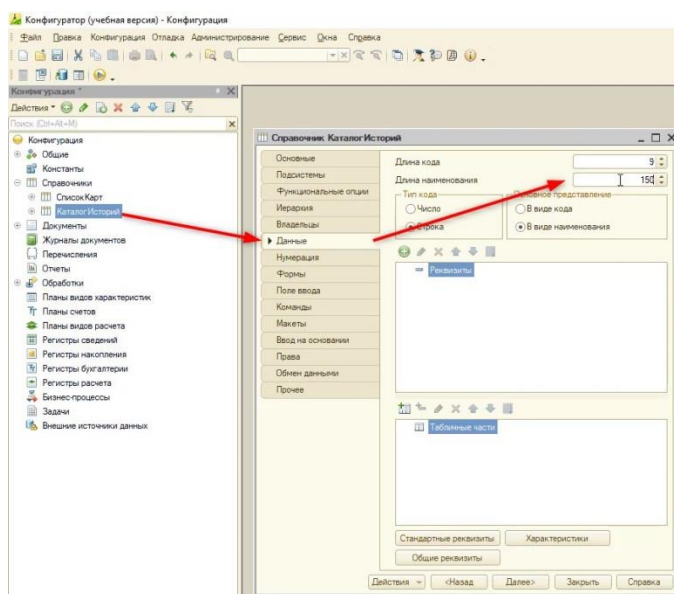


Рис. 1.10.2. Увеличение длины наименования

Для более удобного отображения можно задать тип кода – «Число» (рис. 1.10.3).

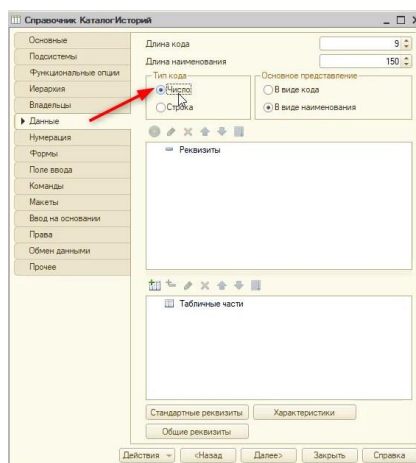


Рис. 1.10.3. Смена типа кода

Далее необходимо добавить реквизиты в наш справочник. Реквизиты будут хранить ответы для победного и проигрышного исхода, а также изображения и текст для этапов. Этапов всего будет 5, но при желании можно увеличить это количество.

Создадим два реквизита, которые будут хранить названия вариантов:

- 1) «ВариантДляПобеды», тип: строка, неограниченная длина (рис. 1.10.5);

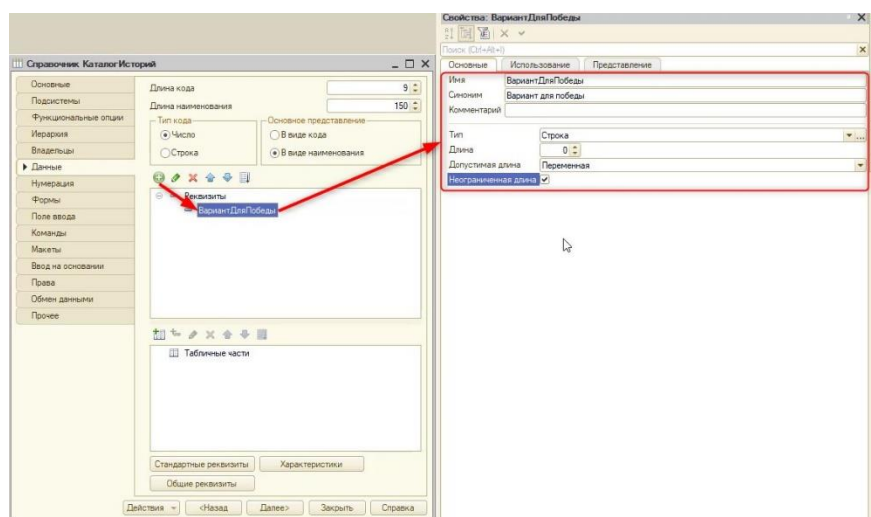


Рис. 1.10.5. Создание нового реквизита

- 2) «ВариантДляПоражения», тип: строка, неограниченная длина (рис. 1.10.6);

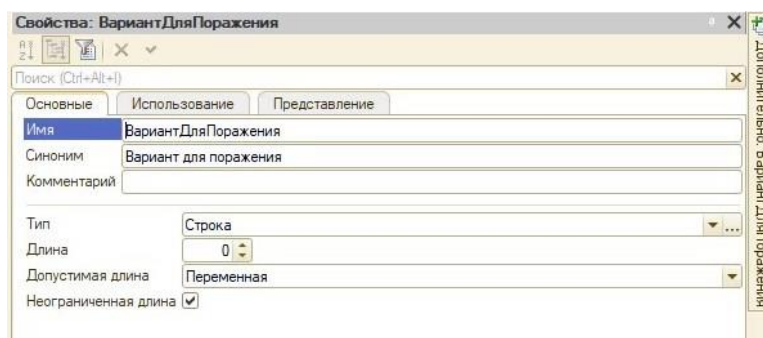


Рис. 1.10.6. Свойства нового реквизита

Для удобства можно создавать копии реквизитов с помощью правой кнопки мыши и выбора «Скопировать» (рис. 1.10.7). Скопированный реквизит будет иметь аналогичные свойства.

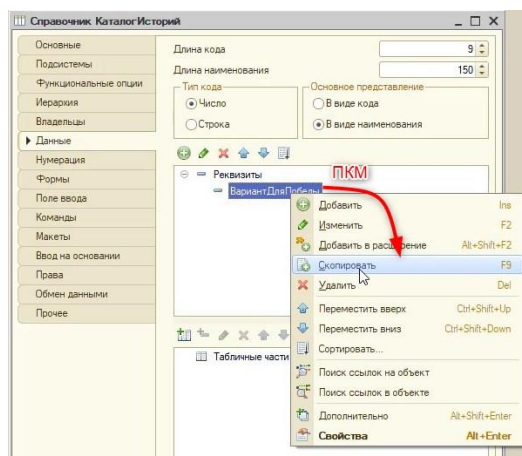


Рис. 1.10.7. Копирование реквизита

Далее нужно создать 5 реквизитов, в которых будут храниться истории (сюжеты) этапов:

- 1) «НачалоИсторииКомментарий»;
- 2) «ПродолжениеИсторииКомментарий»;
- 3) «ВыборВИсторииКомментарий»;
- 4) «ЗавершениеИсторииКомментарий»;
- 5) «ПлохоеЗавершениеИсторииКомментарий».

Все новые реквизиты будут иметь тип «Строка» и неограниченную длину.

На каждом этапе квеста должны быть картинки, для их хранения тоже необходимо создать реквизиты, но уже с типом значения «ХранилищеЗначения». Создадим новые реквизиты:

- 1) «НачалоИстории»;
- 2) «ПродолжениеИстории»;
- 3) «ВыборВИстории»;
- 4) «ЗавершениеИстории»;
- 5) «ПлохоеЗавершениеИстории».

Структура справочника готова и выглядит следующим образом (см. рис. 1.10.8).

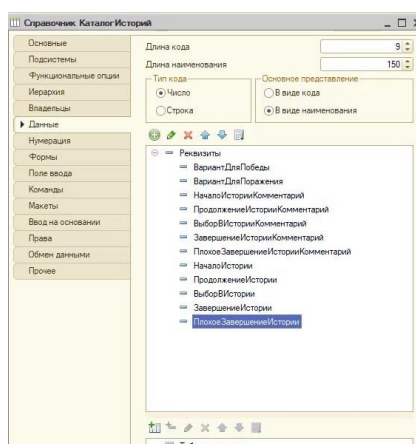


Рис. 1.10.8. Структура справочника «Каталог историй»

На данный момент справочник в режиме пользователя выглядит непрактично (рис. 1.10.9).

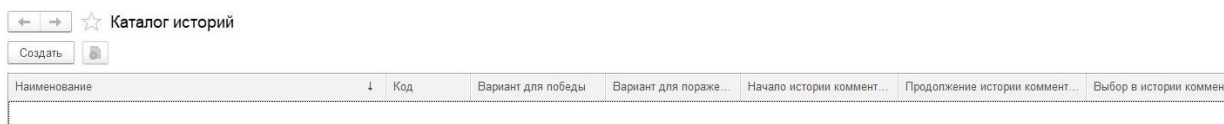


Рис. 1.10.9. Справочник в пользовательском режиме

Форма в режиме «1С:Предприятие», которая открывается первоначально при нажатии на название справочника, называется формой списка.

Отредактируем форму списка справочника, а именно уберем все реквизиты, кроме «Код» и «Наименование». Для этого на вкладке «Формы» создадим основную форму списка (рис. 1.10.10). Основная форма – это форма, которая будет открываться по умолчанию. Если создать не основную форму, то для того, чтобы ее открыть, нужно будет прописывать дополнительное условие кодом.

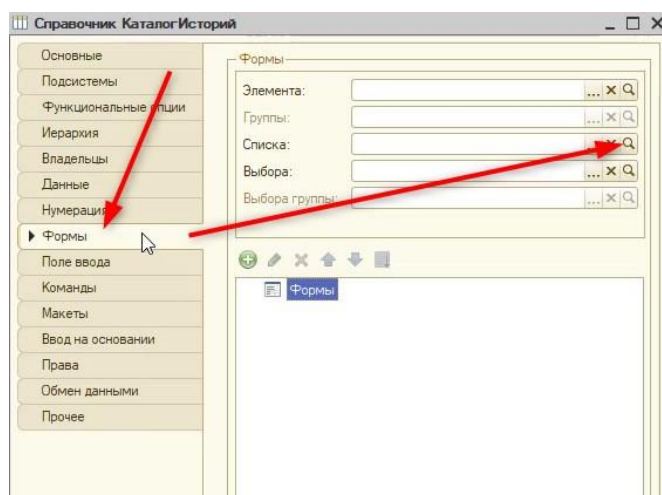


Рис. 1.10.10. Создание основной формы списка

В открывшемся окне конструктора нажмем кнопку «Далее» (рис. 1.10.11).

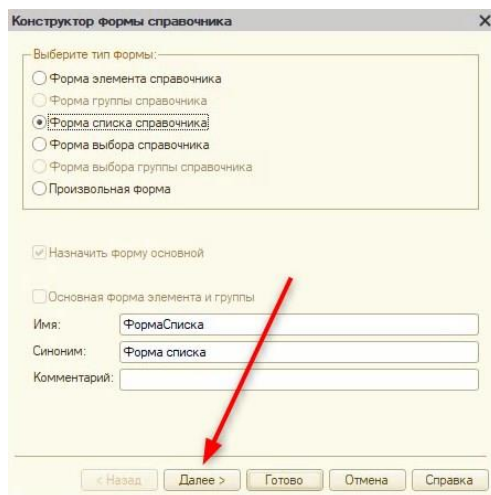


Рис. 1.10.11. Конструктор формы

Далее укажем, какие поля необходимо отобразить, а именно «Наименование» и «Код» (рис. 1.10.12), и нажмем кнопку «Готово».

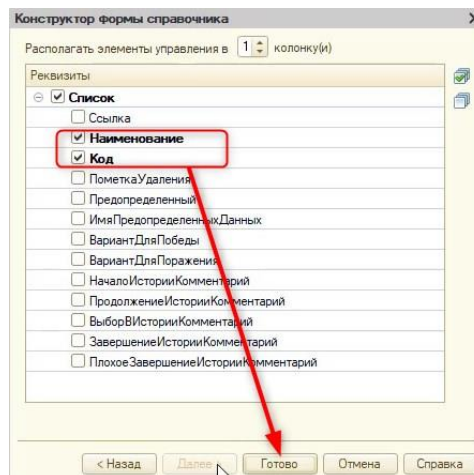


Рис. 1.10.12. Выбор реквизитов для отображения

Теперь в режиме пользователя можно увидеть, что форма списка изменилась.

Если открыть форму самой истории (форму элемента справочника), то она будет выглядеть некорректно, в ней нельзя прикрепить картинки (рис. 1.10.13).

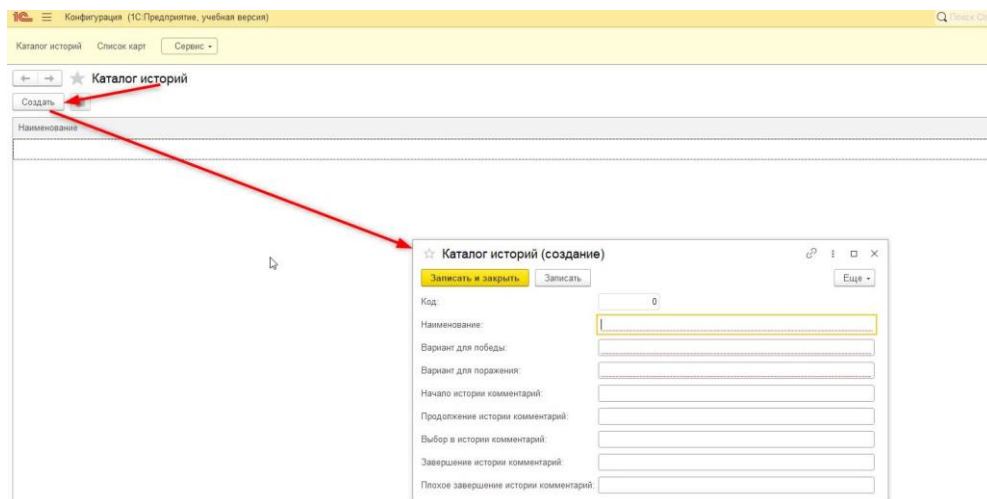


Рис. 1.10.13. Форма элемента

Чтобы исправить это, создадим основную форму элемента (рис. 1.10.14).

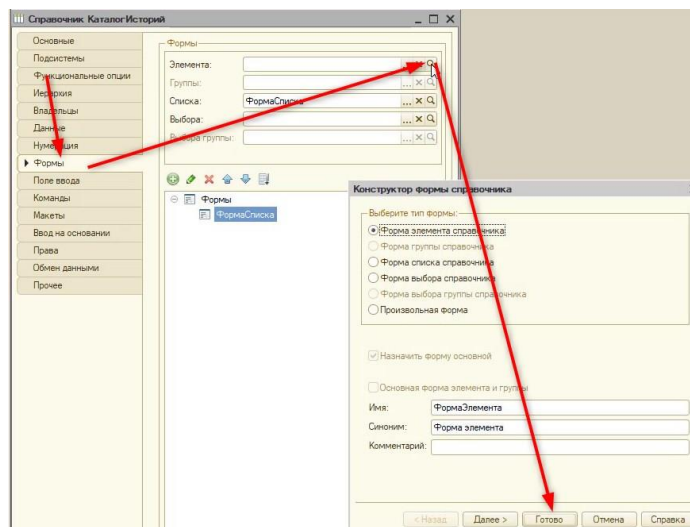


Рис. 1.10.14. Создание формы элемента

В окне редактирования формы добавим группу «Страницы» (рис. 1.10.15), она позволит вмещать в себя отдельные страницы.

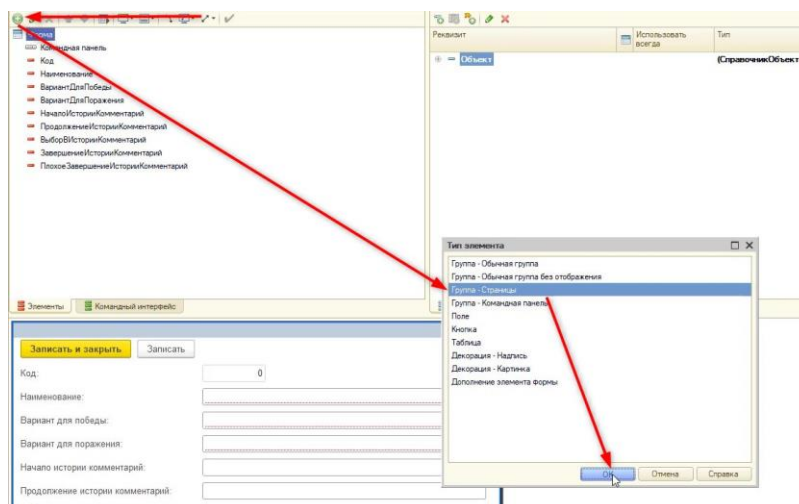


Рис. 1.10.15. Создание группы «Страницы»

Новую группу переименуем в «ГруппаСтраницы». В эту группу добавим группу «Страница» (рис. 1.10.16 – 1.10.17).

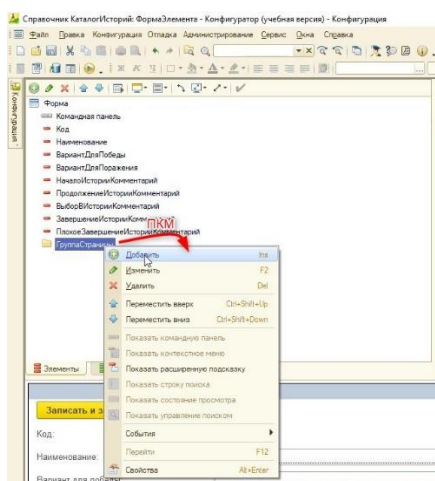


Рис. 1.10.16. Создание новой группы

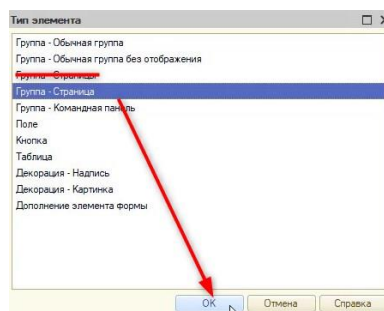


Рис. 1.10.17. Создание группы «Страница»

Новую группу назовем «ГруппаИстория» с заголовком «История». Далее перенесем элементы «Код», «Наименование», «ВариантДляПобеды» и «ВариантДляПоражения» в новую группу (рис. 1.10.18).

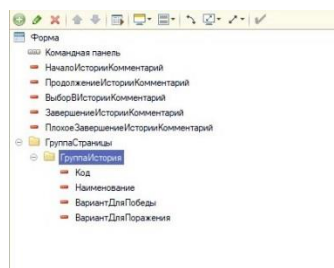


Рис. 1.10.18. Состав новой группы

Включим отображение заголовков справа. Для этого обратимся к свойствам «ГруппаСтраницы» и на вкладке «Использование» укажем следующее (см. рис. 1.10.19).

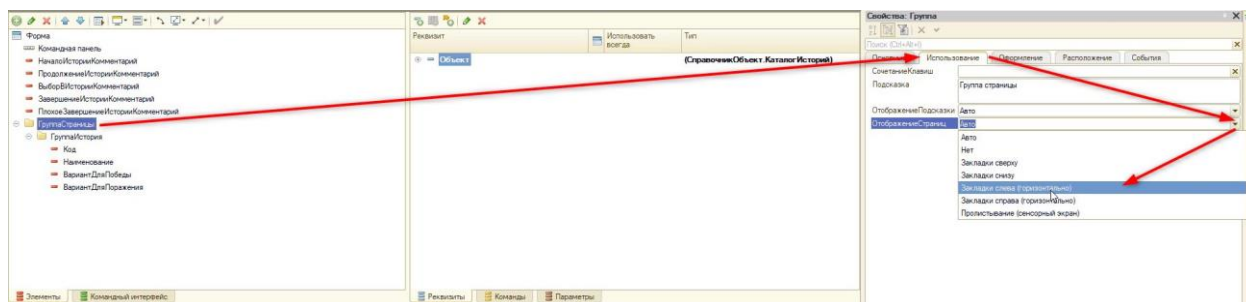


Рис. 1.10.19. Изменение отображения страниц

Следующая страница, которую необходимо добавить, – это «ГруппаНачалоИстории» с заголовком «Начало истории». Она будет содержать элемент «НачалоИсторииКомментарий».

По аналогии добавим еще ряд групп, чтобы по итогу получилась следующая картина (см. рис. 1.10.20).

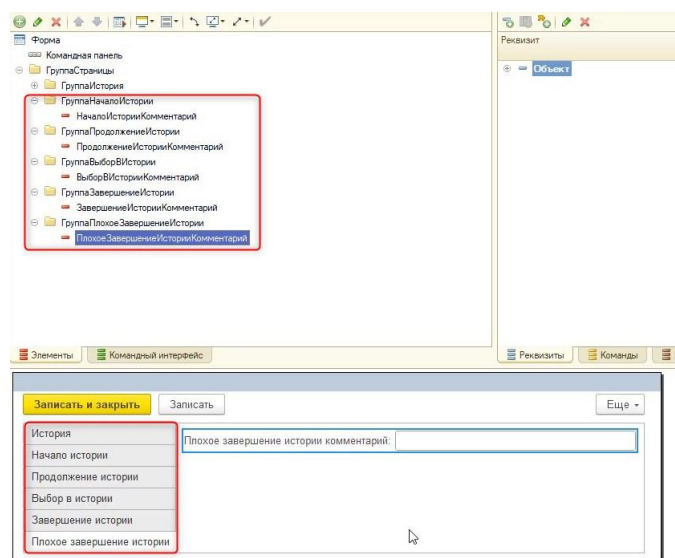


Рис. 1.10.20. Расположение групп и элементов новой формы

Выделим элементы, которые находятся в новых группах, с помощью клавиши «Ctrl» и укажем свойство «Многострочный режим» в положение «Да» (рис. 1.10.21).

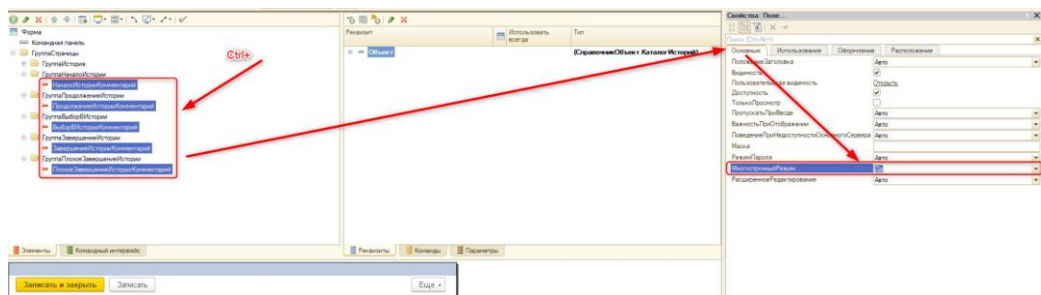


Рис. 1.10.21. Включение многострочного режима

Для того чтобы эти поля занимали всю форму, на вкладке «Расположение» выключим свойства «АвтоМаксимальнаяШирина» и «АвтоМаксимальнаяВысота».

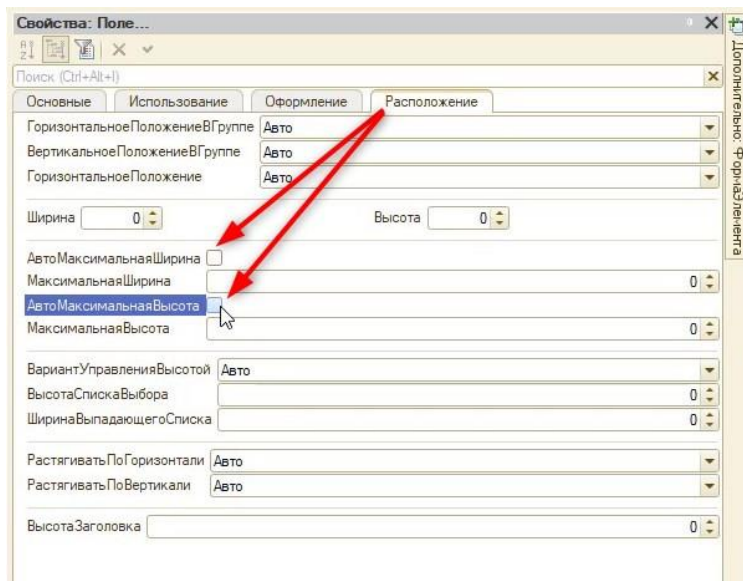


Рис. 1.10.22. Выключение авто максимальной ширины и высоты

Также сделаем шрифт для ввода текста покрупнее на вкладке «Оформление» (рис. 1.10.23).

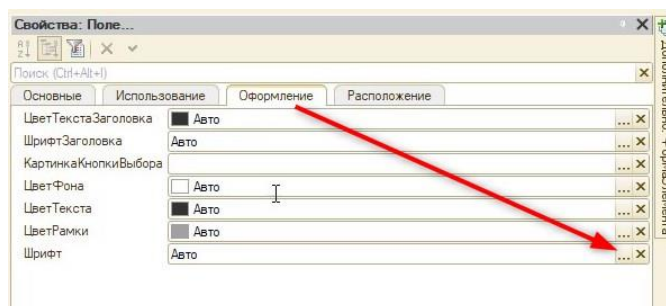


Рис. 1.10.23. Редактирование шрифта

Предварительно выключим галочку «Из стиля» и укажем размер шрифта «14» (рис. 1.10.24).

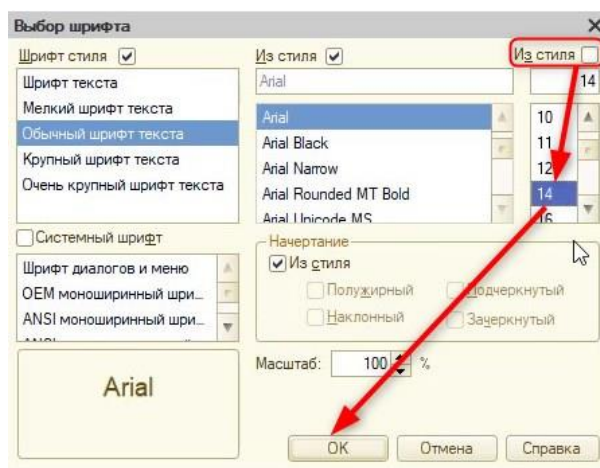


Рис. 1.10.24. Увеличение размера шрифта

Теперь необходимо отобразить картинки в этом интерфейсе. Чтобы это сделать, создадим реквизиты формы под каждую картинку с типом «Строка» (рис. 1.10.25).

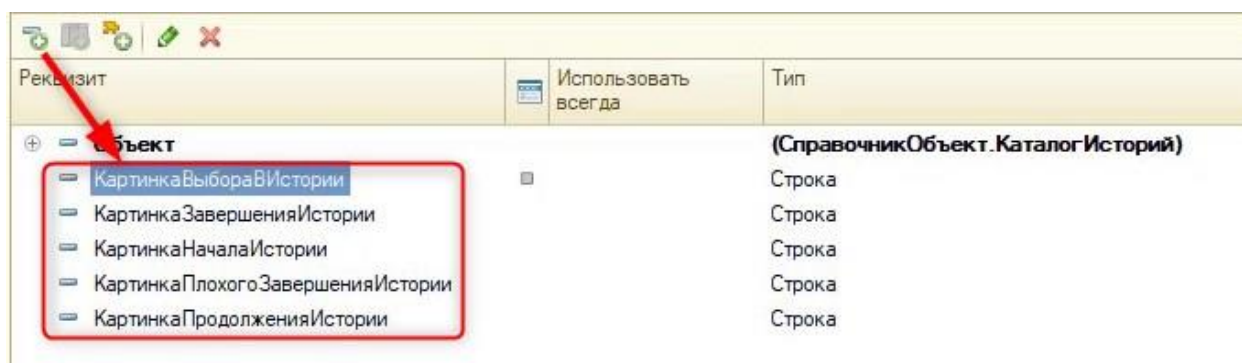


Рис. 1.10.25. Создание реквизитов под хранение картинок

Далее, как делали ранее, перенесем эти реквизиты на форму в соответствующую группу и укажем у каждого элемента свойство «Вид» – «Поле картинки».

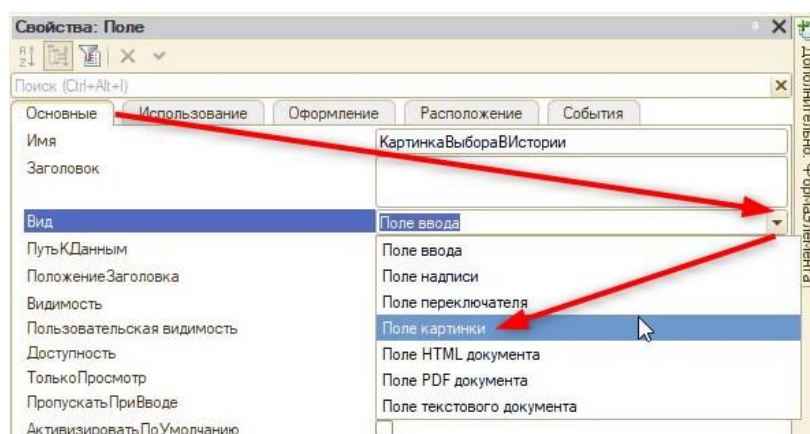


Рис. 1.10.26. Изменение отображения элемента

После этого окно редактирования формы выглядит следующим образом (см. рис. 1.10.27).

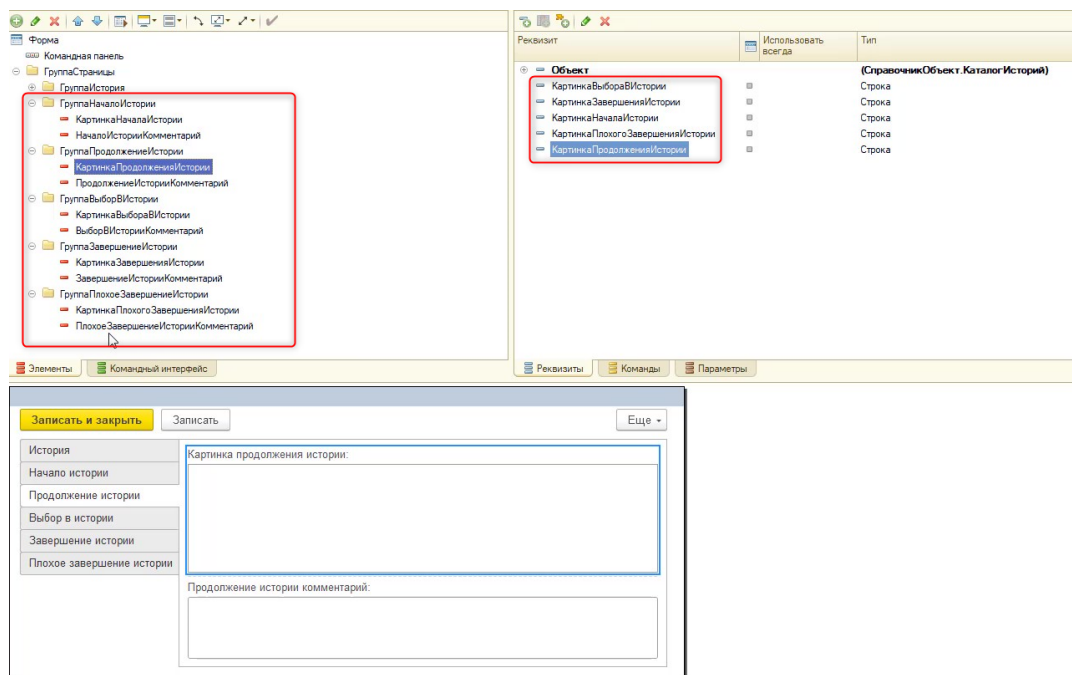


Рис. 1.10.27. Форма после изменений

Уберем у всех элементов с картинками заголовков (рис. 1.10.28).

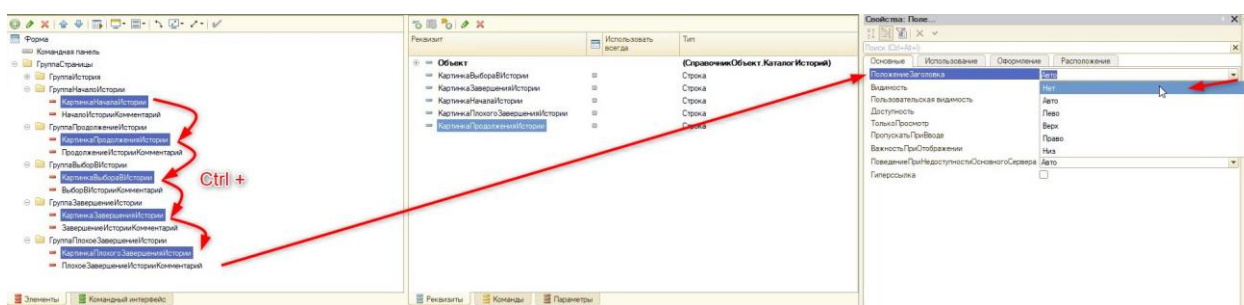


Рис. 1.10.28. Отключение заголовка

Дополнительно включим свойство «Гиперссылка» для этих элементов (рис. 1.10.29).

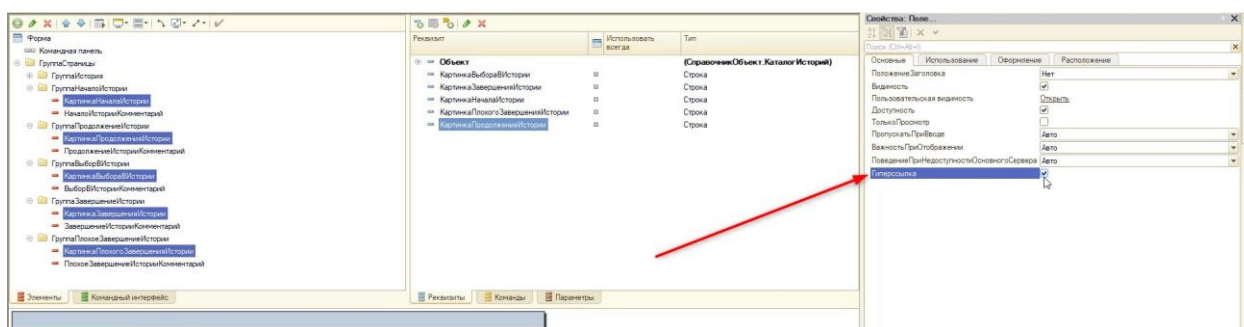


Рис. 1.10.29. Включение гиперссылки

Это свойство нужно для того, чтобы в дальнейшем пользователь смог нажать на окно с изображением и прикрепить нужную картинку.

Чтобы будущая картинка не растягивала окно, обратимся к свойству «Размер картинки» и укажем «Пропорционально» (рис. 1.10.30).

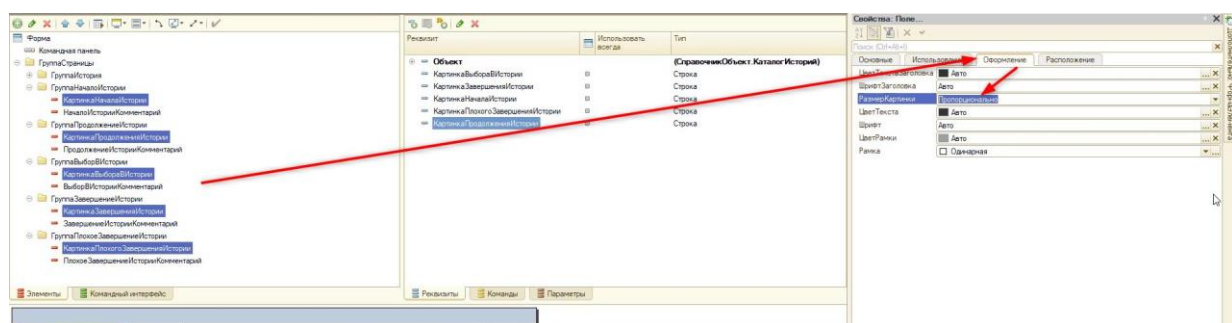


Рис. 1.10.30. Пропорциональное отображение картинки в окне

Готово! Теперь в программе есть форма, при помощи которой пользователь сможет создавать собственный сюжет для квеста.

Занятие 11 – Продолжение разработки игры «Квест»

Следующее, что необходимо сделать, – это реализовать механизм загрузки изображений для каждого этапа истории.

Сделать это можно двумя способами:

- 1) Создать один обработчик и привязать его ко всем элементам картинки;
- 2) Создать для каждой картинки обработчик события.

Естественно, самым логичным и практичным является первый вариант. Создадим для одной из картинок обработчик события «Нажатие» (рис. 1.11.1).

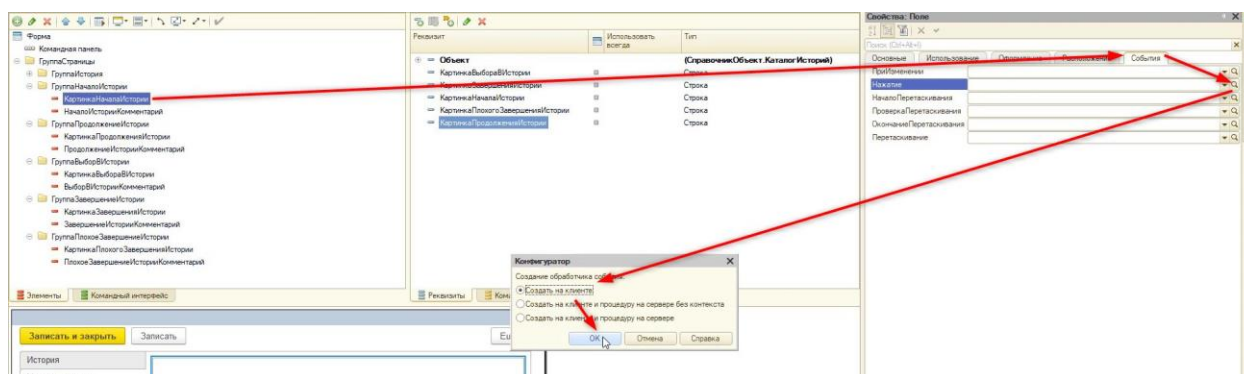


Рис. 1.11.1. Создание обработчика события

Чтобы в дальнейшем не запутаться, поменяем имя процедуры на «КартинкаЭтапаИсторииНажатие». Т. к. привязывать мы будем процедуру ко всем картинкам, ей нужно дать более обезличенное название.

Важно!

Если изменить название привязанного к элементу обработчика (вкладка «События»), то эта привязка пропадет, и ее придется настраивать заново.

Заранее для каждого элемента картинки привяжем новую процедуру для события «Нажатие» (рис. 1.11.2).

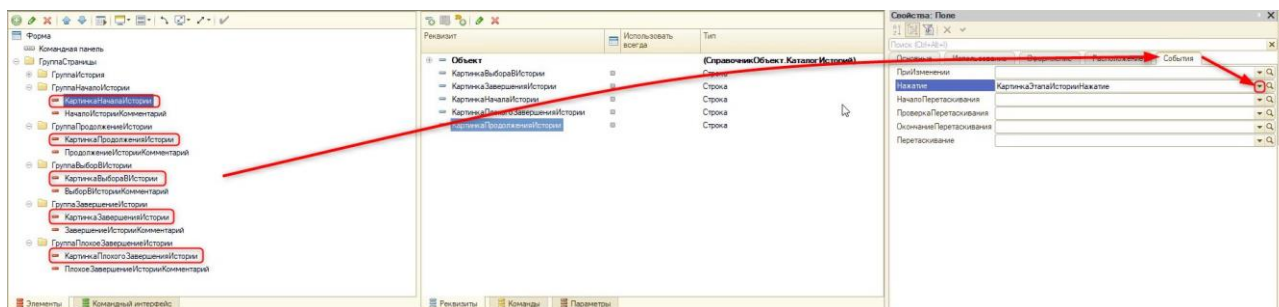


Рис. 1.11.2. Привязка существующего обработчика событий

Далее вернемся в модуль формы и приступим к программированию загрузки изображений:

&НаКлиенте

Процедура КартинкаЭтапаИсторииНажатие(Элемент, СтандартнаяОбработка)

//Отключим окна с предупреждениями:

СтандартнаяОбработка = Ложь;

```

        //Обработкаем выбор файла картинки
        //Так как прикреплять нужно не одну картинку, а несколько, создадим отдельную
процедуру.
        //ОписаниеОповещения(Имя процедуры, Модуль в котором она находится ,
Дополнительные параметры);
        //Дополнительные параметры – это значение с произвольным типом данных
        //В этот параметр потребуется поместить значения имени элемента (на которое нажал
пользователь)
        //Это нужно для того, чтобы потом знать в какое поле нужно поместить картинку
(Элемент.Имя)

        Оповещение = Новый ОписаниеОповещения("ОбработатьВыборФайла", ЭтотОбъект,
Элемент.Имя);
        //С помощью метода НачатьПомещениеФайла(ОписаниеОповещения, , ,
Интерактивно,УникальныйИдентификаторФормы)
        //Описание оповещения – это процедура, которая будет вызвана после завершения
помещения файла.
        //Интерактивно - включение диалога выбора файла
        НачатьПомещениеФайла(Оповещение, , , Истина, УникальныйИдентификатор);
        КонецПроцедуры

```

Далее опишем новую клиентскую процедуру «ОбработатьВыборФайла», которую вызвали в обработчике события:

```

&НаКлиенте
//ДополнительныеПараметры - содержит имя элемента, в который нужно поместить картинку
//Экспорт, чтобы процедуру можно было вызвать через оповещение
Процедура ОбработатьВыборФайла(Результат, Адрес, ПомещаемыйФайл,
ДополнительныеПараметры) Экспорт
    //Выбрал ли пользователь файл?
    Если Результат = Ложь Тогда
        //Пользователь не выбрал картинку
        Возврат;
    КонецЕсли;
    //Так как точное название элемента, в который нужно поместить картинку точно не знаем
обращаемся через квадратные скобки
    //В результате получится (к примеру) ЭтотОбъект.КартинкаПродолженияИстории
    ЭтотОбъект[ДополнительныеПараметры] = Адрес;
    Модифицированность = Истина;
    КонецПроцедуры

```

Адрес, который мы подставили в реквизит формы, является адресом временного хранилища и на данный момент нигде не сохраняется. Временное хранилище – это строковая переменная, в которую помещен адрес, где хранятся данные. Для того чтобы сохранять адрес, необходимо описать обработчик для события «ПередЗаписьюНаСервере».

Для создания новой процедуры обратимся к свойствам самой формы на вкладке «События». На этой вкладке необходимо найти событие «ПередЗаписьюНаСервере» и создать для него новый обработчик (рис. 1.11.2).

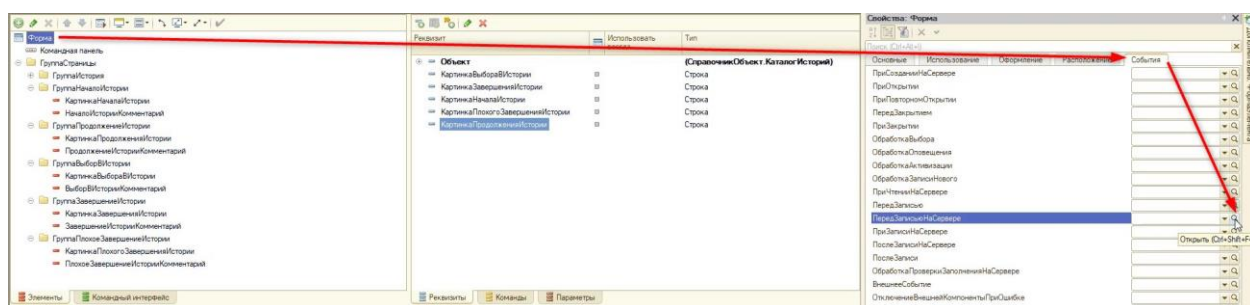


Рис. 1.11.2. Создание нового обработчика события

Так как это событие происходит только на сервере, программа не предлагает нам выбор между клиентом и сервером как раньше.

Приступим к написанию кода. В этой процедуре будет происходить запись адреса из временного хранилища в реквизит. Так как таких реквизитов несколько, то придется программно получить нужные реквизиты формы (рис. 1.11.3) и объекта (рис. 1.11.4) с помощью новых функций.

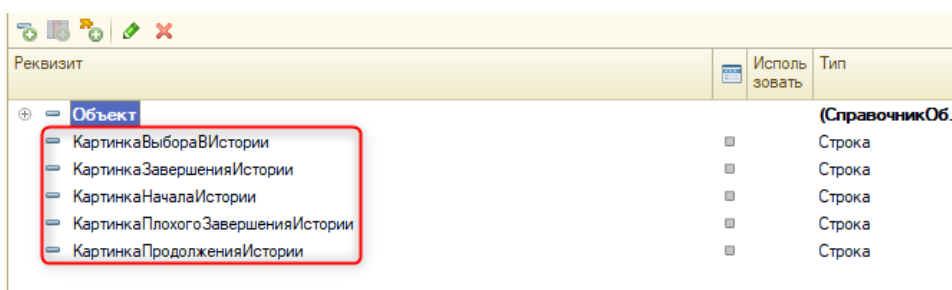


Рис. 1.11.3. Реквизиты формы

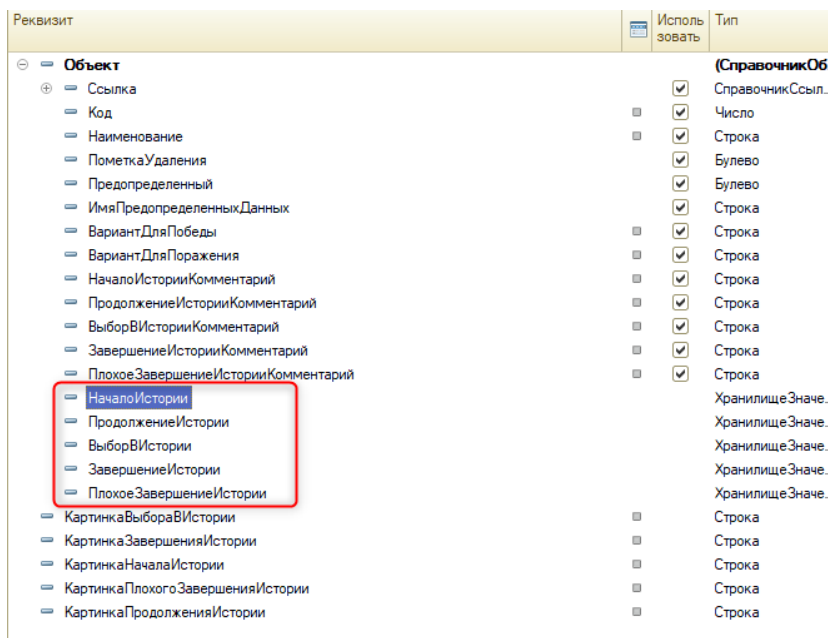


Рис. 1.11.4. Реквизиты объекта, которые необходимо записать в массив

Важно!

Реквизит формы хранит информацию до тех пор, пока открыта форма.

Для начала создадим массив и присвоим ему результат функции:

```
&НаСервере  
Процедура ПередЗаписьюНаСервере(Отказ, ТекущийОбъект, ПараметрыЗаписи)  
    МассивРеквизитовФормы = ПолучитьМассивРеквизитовФормы();  
КонецПроцедуры
```

Теперь создадим функцию, которую вызвали:

```
&НаСервере  
Функция ПолучитьМассивРеквизитовФормы()  
    //Создаем новый массив  
    Массив = Новый Массив;  
    //Добавим в массив названия всех реквизитов формы:  
    Массив.Добавить("КартинкаНачалаИстории");  
    Массив.Добавить("КартинкаПродолженияИстории");  
    Массив.Добавить("КартинкаВыбораВИстории");  
    Массив.Добавить("КартинкаЗавершенияИстории");  
    Массив.Добавить("КартинкаПлохогоЗавершенияИстории");  
    //Сделаем возврат массива со значениями  
    Возврат Массив;  
КонецФункции
```

Далее нужно получить массив реквизитов, которые находятся в самой структуре справочника (были созданы через вкладку «Данные»). Для начала вызовем новую функцию в обработчике «ПередЗаписьюНаСервере»:

```
&НаСервере  
Процедура ПередЗаписьюНаСервере(Отказ, ТекущийОбъект, ПараметрыЗаписи)  
    МассивРеквизитовФормы = ПолучитьМассивРеквизитовФормы();  
    МассивРеквизитовОбъекта = ПолучитьМассивРеквизитовОбъекта();  
КонецПроцедуры
```

Создадим новую функцию:

```
&НаСервере  
Функция ПолучитьМассивРеквизитовОбъекта()  
    //По аналогии с предыдущей функцией  
    //Заполняем названиями реквизитов самого справочника  
    Массив = Новый Массив;  
    Массив.Добавить("НачалоИстории");  
    Массив.Добавить("ПродолжениеИстории");  
    Массив.Добавить("ВыборВИстории");  
    Массив.Добавить("ЗавершениеИстории");  
    Массив.Добавить("ПлохоеЗавершениеИстории");  
    Возврат Массив;  
КонецФункции // ПолучитьМассивРеквизитовОбъекта()
```

Важно!

Порядок реквизитов объекта должен быть такой, чтобы потом их можно было соотнести с реквизитами формы. Т. е. КартинкаНачалаИстории – НачалоИстории, КартинкаПродолженияИстории – ПродолжениеИстории и т. д.

Далее сохраним адрес картинки:

```
&НаСервере
Процедура ПередЗаписьюНаСервере(Отказ, ТекущийОбъект, ПараметрыЗаписи)
    МассивРеквизитовФормы = ПолучитьМассивРеквизитовФормы();
    МассивРеквизитовОбъекта = ПолучитьМассивРеквизитовОбъекта();

    //Обойдём все элементы массива, которые содержит название реквизитов объекта
    //Обход идёт с 0, так как индексация массива начинается с 0
    Для Счетчик = 0 По МассивРеквизитовОбъекта.ВГраница() Цикл
        //Создадим новые переменные и присвоим им значения из массива:
        //К примеру, МассивРеквизитовФормы[0] = "КартинкаНачалаИстории"
        РеквизитФормы = МассивРеквизитовФормы[Счетчик];
        РеквизитОбъекта = МассивРеквизитовОбъекта[Счетчик];
        //Опишем алгоритм сжатия, чтобы уменьшать "вес" картинок.
        //Тогда игру можно будет запустить на более слабом компьютере
        АлгоритмСжатия = Новый СжатиеДанных(9);
        //Если в реквизите формы есть адрес временного хранилища, тогда:
        Если ЭтоАдресВременногоХранилища(ЭтотОбъект[РеквизитФормы]) Тогда
            //Получим адрес из временного хранилища:
            Значение = ПолучитьИзВременногоХранилища(ЭтотОбъект[РеквизитФормы]);
            //Запишем его в реквизит формы с помощью метода ХранилищеЗначения(Значение,
            АлгоритмСжатияДанных)
            //Сожмем картинку с помощью описанного ранее алгоритма и поместим в реквизит:
            ТекущийОбъект[РеквизитОбъекта] = Новый ХранилищеЗначения(Значение,
            АлгоритмСжатия);
        КонецЕсли;
    КонецЦикла;
КонецПроцедуры
```

Теперь можно прикрепить изображение, и оно запишется в базу. Однако оно не отобразится при повторном открытии. Чтобы это исправить, необходимо будет описать дополнительно обработчик события «ПриСозданииНаСервере».

Для начала создадим этот обработчик с помощью свойств форм, как делали ранее (рис. 1.11.5).

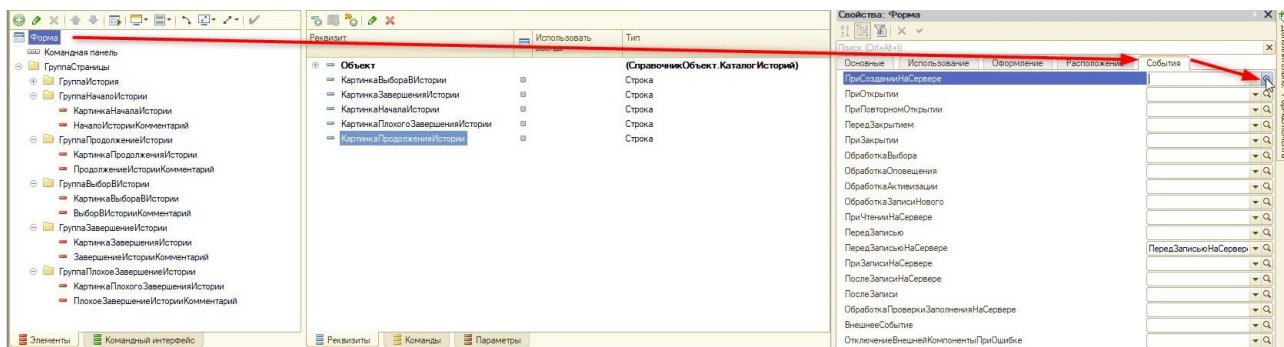


Рис. 1.11.5. Создание обработчика события «При создании на сервере»

В новой процедуре опишем следующую программную логику:

&НаСервере

Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

//Для начала получим названия реквизитов объекта и формы, с помощью функций, которые создали ранее:

МассивРеквизитовФормы = ПолучитьМассивРеквизитовФормы();

МассивРеквизитовОбъекта = ПолучитьМассивРеквизитовОбъекта();

//Обойдём массив

Для Счетчик = 0 По МассивРеквизитовОбъекта.ВГраница() Цикл

//Присвоим переменным значения из двух массивов:

РеквизитФормы = МассивРеквизитовФормы[Счетчик];

РеквизитОбъекта = МассивРеквизитовОбъекта[Счетчик];

//Получим навигационную ссылку:

ЭтотОбъект[РеквизитФормы] = ПолучитьНавигационнуюСсылку(Объект.Ссылка, РеквизитОбъекта);

КонецЦикла;

КонецПроцедуры

Готово! Теперь можем протестировать прикрепление картинок в пользовательском режиме (рис. 1.11.6).

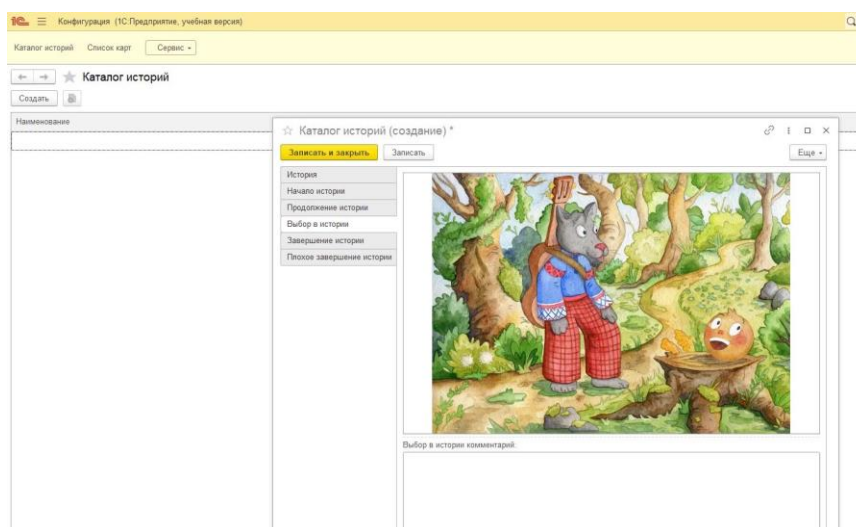


Рис. 1.11.6. Прикрепление картинки по нажатию

Занятие 12 – Создание этапов квеста

Следующее, что необходимо реализовать, – это автоматическое создание этапов истории. Чтобы продвижение по этапам определенного события (к примеру, продажи товара) происходило автоматически, необходимо создать в системе бизнес-процесс. Бизнес-процессы (см. рис. 1.12.1) – это прикладной объект конфигурации, который содержит в себе карту маршрута (см. рис. 1.12.2).

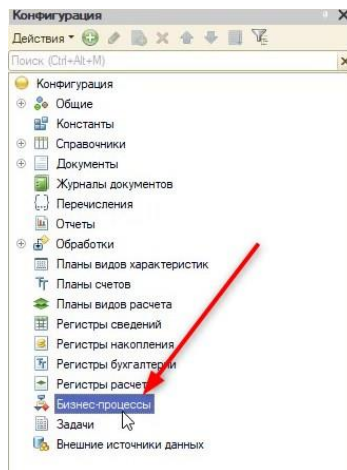


Рис. 1.12.1. Бизнес-процесс в дереве конфигурации

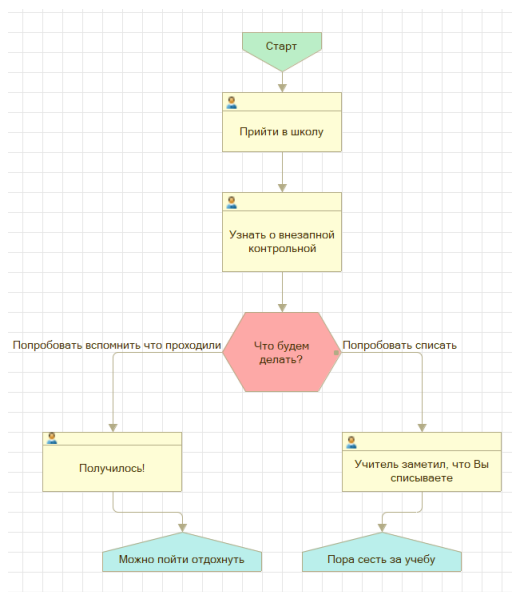


Рис. 1.12.2. Карта маршрута

Каждый бизнес-процесс тесно связан с объектом «Задачи». С точки зрения механизма, задача – это один из этапов процесса. Процесс представляет собой схему (карту маршрута). Бизнес-процесс описывает эту «схему», а в задачах происходит их этапное выполнение. Первоначально необходимо в программе создать прикладной объект «Задачи» с наименованием «ЭтапыИстории» (рис. 1.12.3).

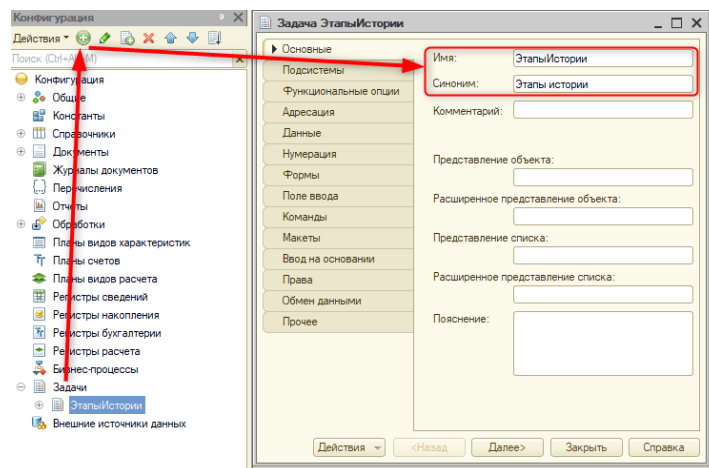


Рис. 1.12.3. Создание задачи

Задачи – это список, похожий на справочник, но с отличительной чертой – помимо записи, там есть кнопка «Выполнить» (рис. 1.12.4).

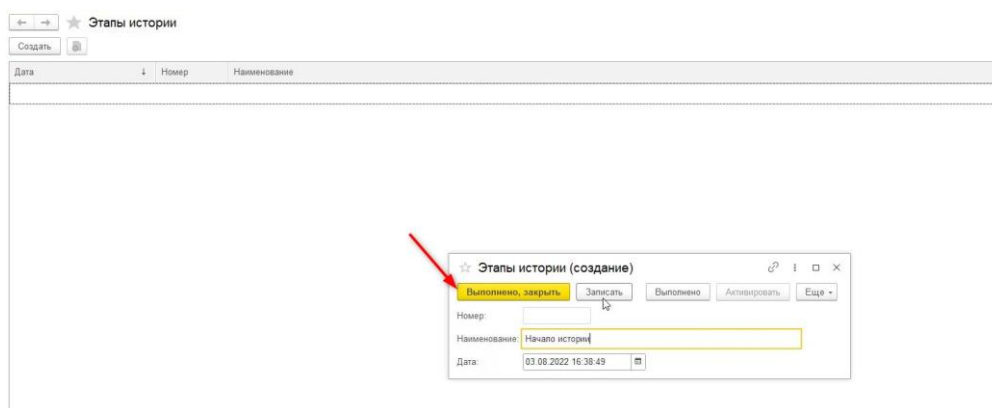


Рис. 1.12.4. Прикладной объект «Задачи» в пользовательском режиме

Отметка «Выполнено» означает, что данный этап пройден и можно автоматически создать следующий этап. Автоматически создаются последующие этапы при наличии связи с бизнес-процессом (в котором содержится схема с последовательностью задач).

Перед тем как создать эту «связь» и новый бизнес-процесс, откорректируем настройки задач. Увеличим длину наименования на «150» и укажем тип номера – «Число».

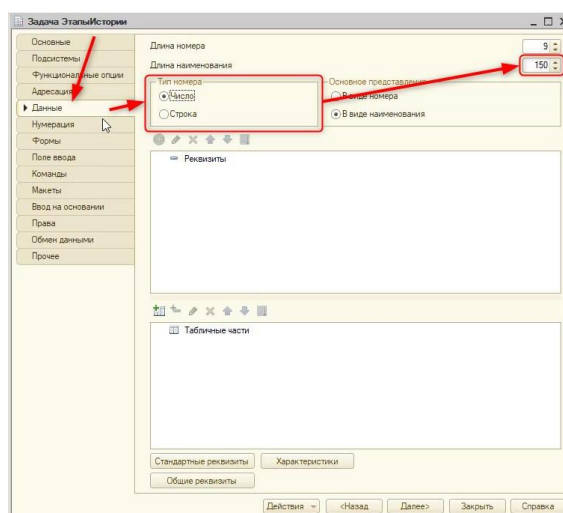


Рис. 1.12.5. Настройка задач

Далее откорректируем форму задачи, а именно уберем все стандартные поля с помощью вкладки «Формы» (рис. 1.12.6 – 1.12.7).

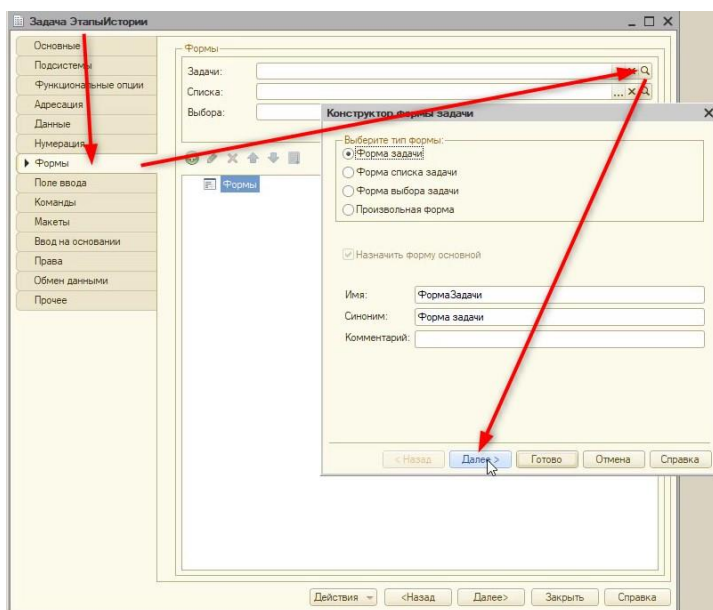


Рис. 1.12.6. Настройка формы задачи

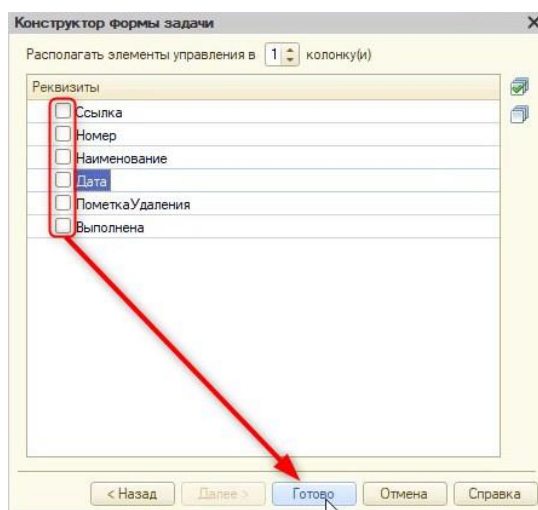


Рис. 1.12.7. Убираем все реквизиты

Далее обратимся к свойствам формы в конструкторе и отключим командную панель, поскольку стандартные команды, которые существуют у задачи, использоваться не будут.

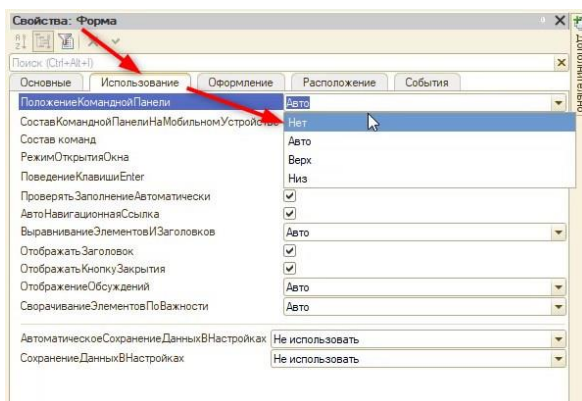


Рис. 1.12.8. Скрытие командной панели

Так или иначе игроку нужны кнопки для взаимодействия с игрой. К примеру, кнопка закрытия, после которой начнется следующий этап. Для этого найдем стандартную команду «Выполнено, закрыть» и вынесем ее на форму (рис. 1.12.9).

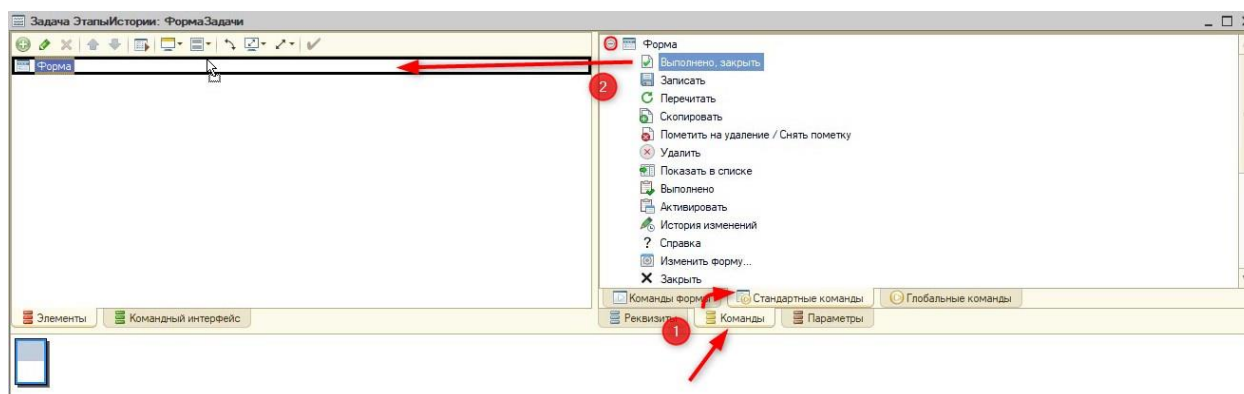


Рис. 1.12.9. Скрытие командной панели

Для игрока значение «Выполнить и закрыть» неясно, поэтому поменяем заголовок кнопки на «Дальше». Вдобавок сделаем кнопку более яркой с помощью включения свойства «Кнопка по умолчанию» (рис. 1.12.10).

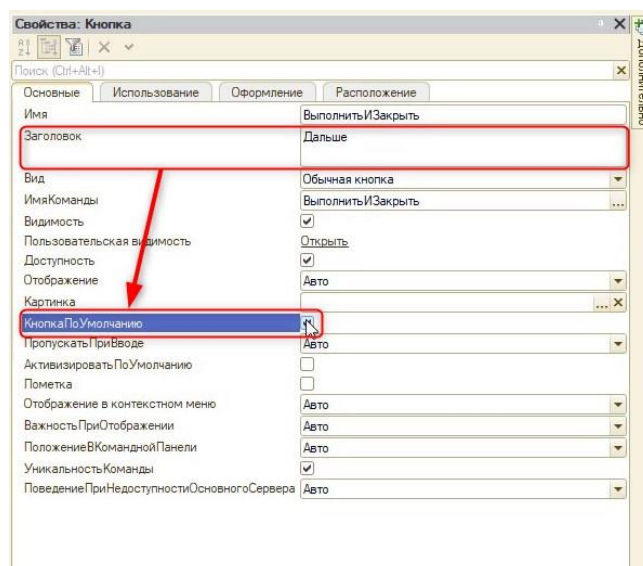


Рис. 1.12.10. Изменение свойств кнопки

Также расположим кнопку по центру формы (рис. 1.12.11).

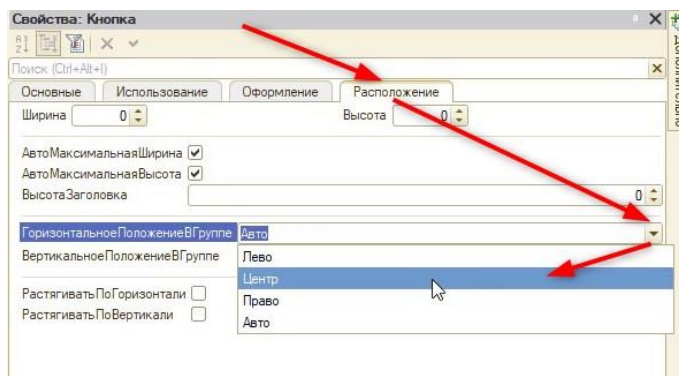


Рис. 1.12.11. Смена расположения кнопки

Приступим к настройке связи между задачами и бизнес-процессом. Для этого создадим новый бизнес-процесс «НашиИстории» и в графе «Задачи» укажем «ЭтапыИстории» (рис.

1.12.12). Дополнительно можно указать представление объекта «Наша история». Это название будет высвечиваться при открытии одной из историй.

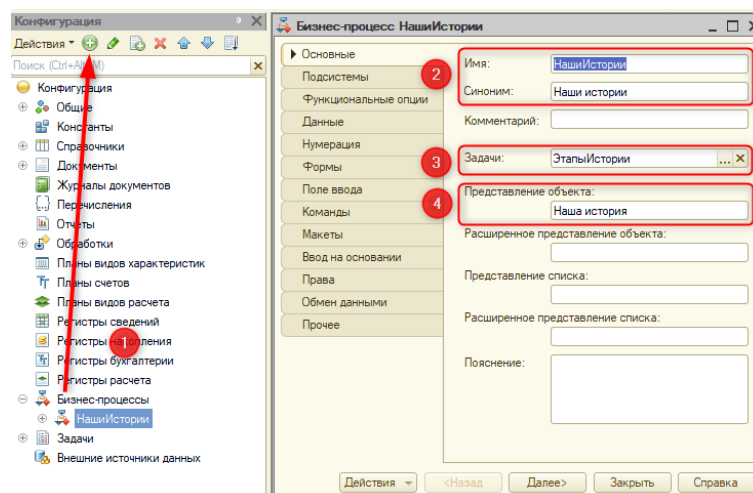


Рис. 1.12.12. Настройка бизнес-процесса

Далее нужно создать карту маршрута (схема, где будет отображена последовательность задач). Для этого откроем вкладку «Прочее» – «Карта маршрута» (рис. 1.12.13).

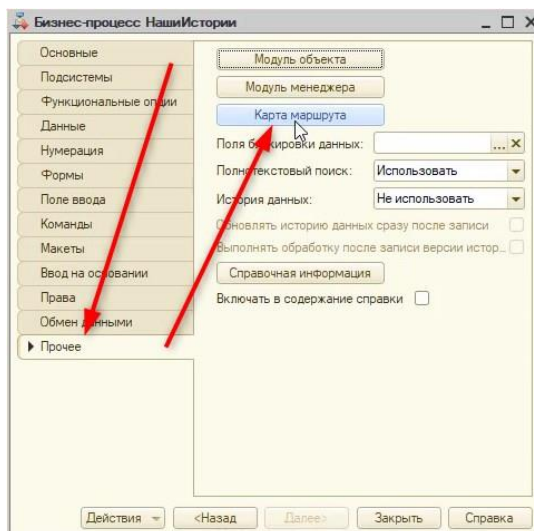


Рис. 1.12.13. Открытие карты маршрута

Карта маршрута состоит из точек навигации, которые расположены в нижней части окна или в пункте главного меню «Карта маршрута» (рис. 1.12.14).

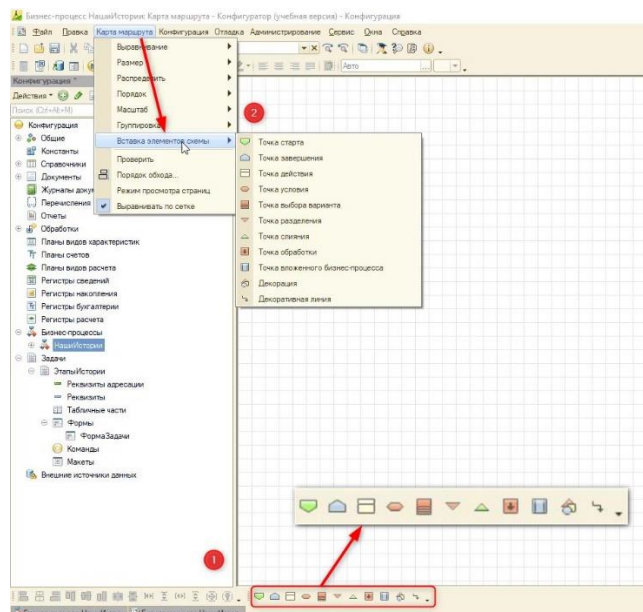


Рис. 1.12.14. Точки навигации

Для того чтобы определить для системы, когда начинается или заканчивается сценарий, необходимо использовать точку старта и завершения.

Выберем точку старта, используя нижнюю панель или главное меню, и поместим с помощью удерживания левой кнопки мыши на схему (рис. 1.12.15).

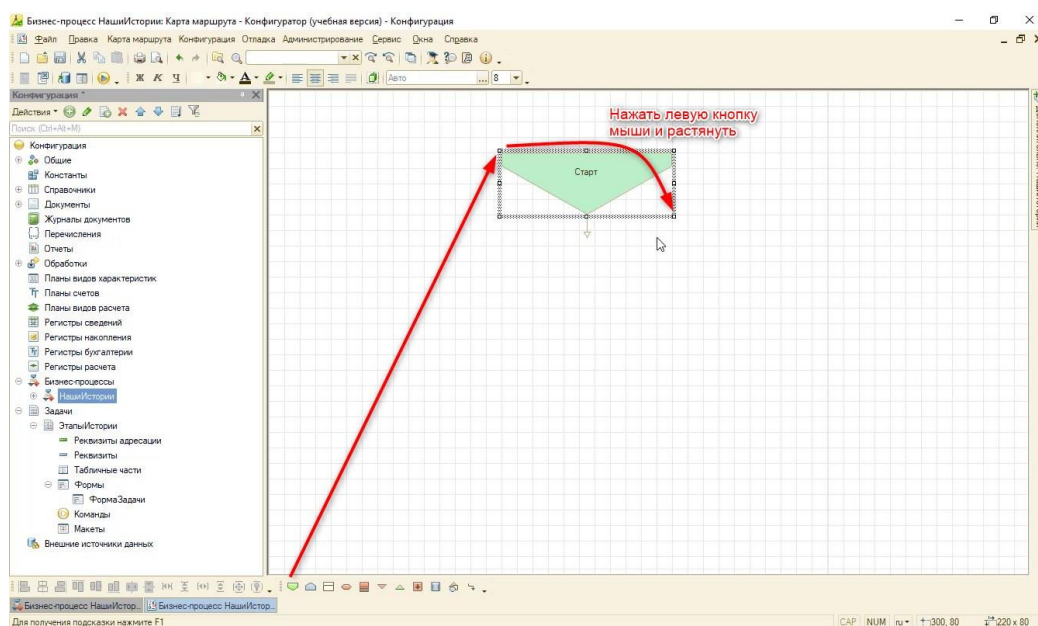


Рис. 1.12.14. Точки навигации

Добавленную точку можно перемещать на схеме после добавления с помощью левой кнопки мыши или клавиатуры.

Из точек маршрута выходят линии со стрелками – это и является последовательностью, которая выстраивается на схеме.

Точка старта и завершения не содержит в себе задачу (этап истории). Для добавления задач необходимо использовать точки действия. Каждая точка действия будет отражать этап истории квеста.

Для начала создадим линейную историю. Добавим точку действия «НачалоИстории» (рис. 1.12.15).

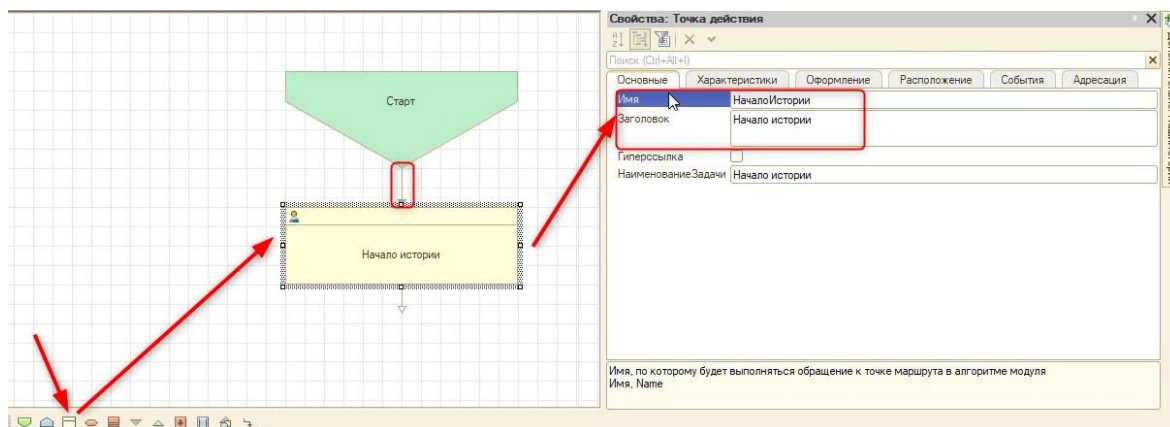


Рис. 1.12.15. Добавление точки действия

По аналогии создадим точки действия «ПродолжениеИстории» и «ЗавершениеИстории» (рис. 1.12.16).

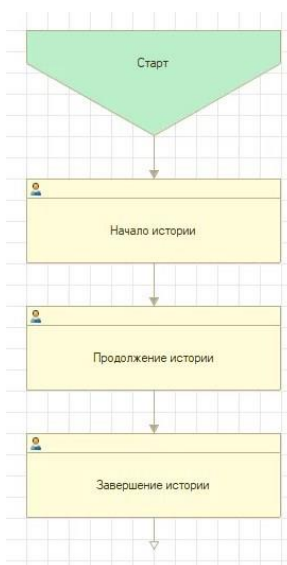


Рис. 1.12.16. Обновленная карта маршрута

В низ схемы поместим точку завершения (рис. 1.12.17).

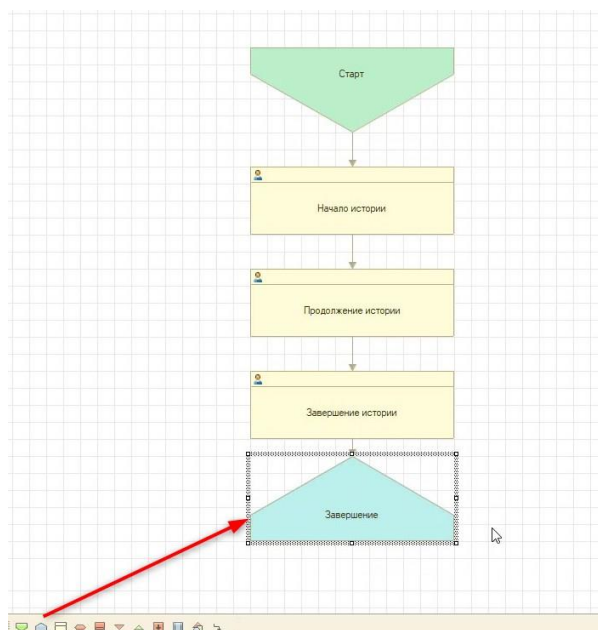


Рис. 1.12.17. Точка завершения

Теперь, если создать в пользовательском режиме бизнес-процесс и начать его, то автоматически будут появляться новые задачи. Создадим нашу историю (рис. 1.12.18).

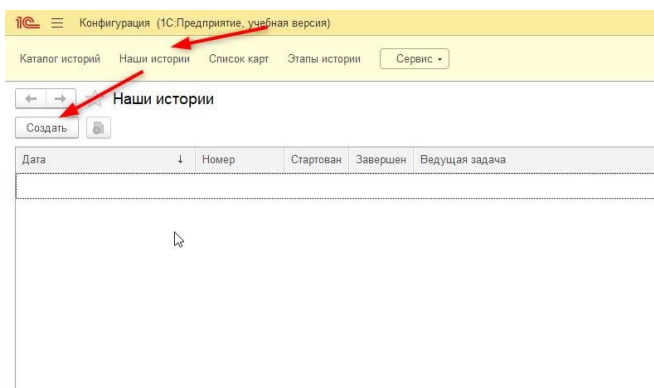


Рис. 1.12.18. Создание бизнес-процесса

На форме объекта (рис. 1.12.19) находится достаточно много лишних полей, в дальнейшем их потребуется убрать, чтобы не нагружать игрока лишней информацией. Как видно на форме, функционал бизнес-процесса позволяет отслеживать старт, завершение и ведущую задачу.

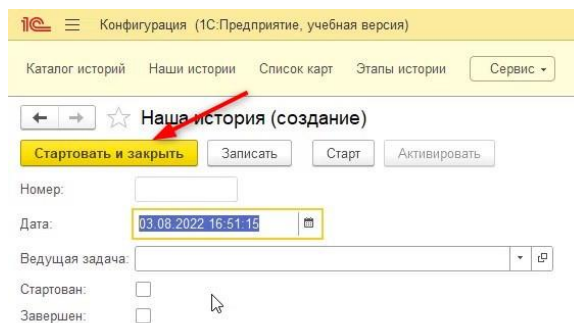


Рис. 1.12.19. Старт бизнес-процесса

Идея бизнес-процесса заключается в том, что старт работы карты маршрута зависит от того, был ли этот процесс начат. После нажатия кнопки «Стартовать и закрыть» перейдем в задачи «Этапы истории».

Если эта форма была открыта ранее, ее потребуется обновить, чтобы появились задачи. Используем горячую клавишу «F5» или кнопки «Еще» – «Обновить» (рис. 1.12.20).

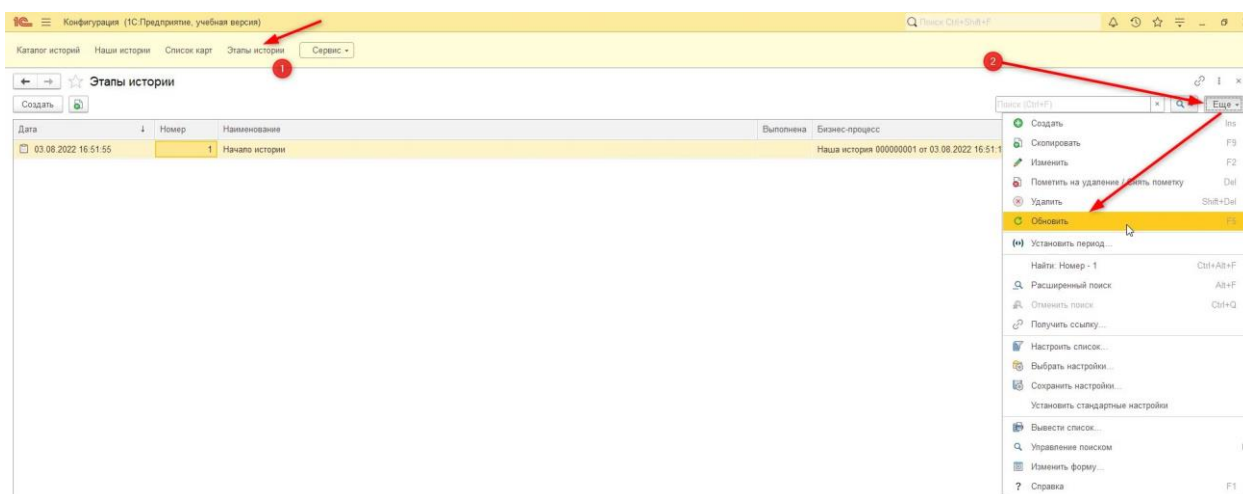


Рис. 1.12.20. Старт бизнес-процесса

Откроем форму задачи и нажмем кнопку «Дальше», после этого форма закроется, и автоматически появится новая задача в списке (рис. 1.12.21).

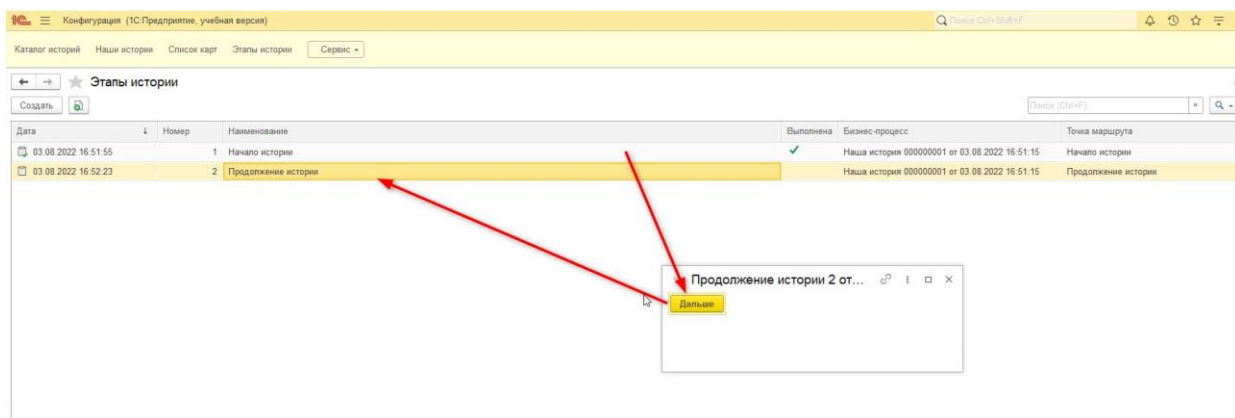


Рис. 1.12.21. Старт бизнес-процесса

Новые задачи перестали появляться после завершения истории, так как процесс прохождения карты маршрута окончен. Если вернуться обратно к бизнес-процессам на форму, то флага в виде галочки в колонке «Завершен» не будет. Связано это с отсутствием команд на обновление формы. «Перечитать» форму можно с помощью клавиши «F5», как было показано, тогда все будет выглядеть корректно (рис. 1.12.22).

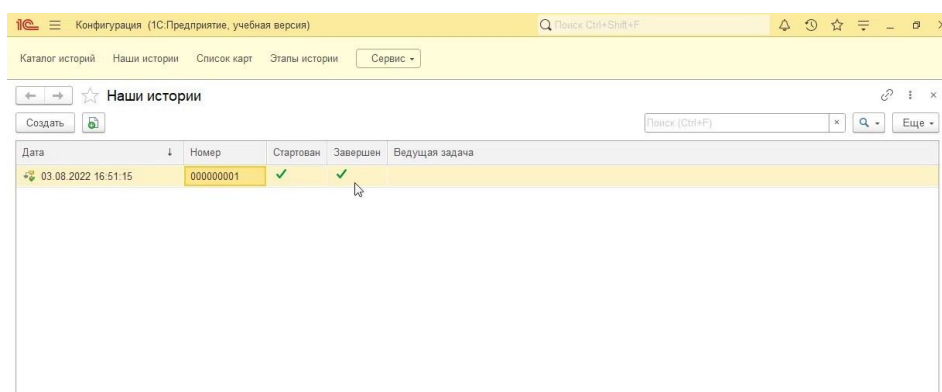


Рис. 1.12.22. Завершение бизнес-процесса

В дальнейшем этот механизм будет универсальным, и игрок сможет создавать собственные истории на разные тематики.

Если запустить новый бизнес-процесс, то новая задача появится вместе со старыми. Так как в дальнейшем у нас будет много историй, работать с таким списком будет неудобно (рис. 1.12.23).

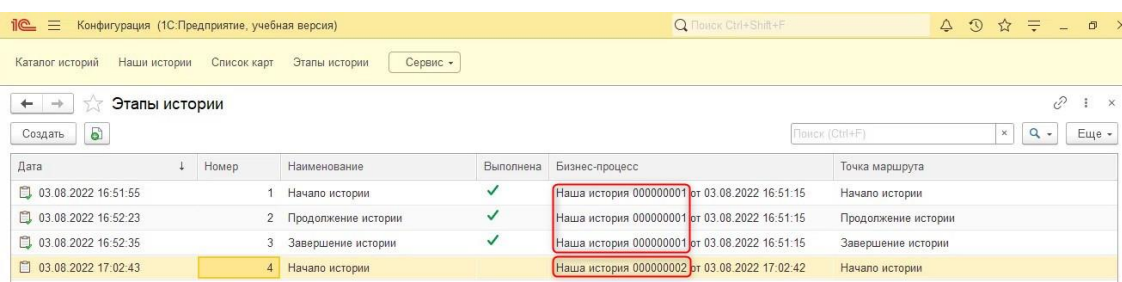


Рис. 1.12.23. Задачи после запуска еще одного бизнес-процесса

Для того чтобы это исправить, уберем видимость в интерфейсе задач и выведем список к каждому бизнес-процессу.

Вернемся в режим разработчика и обратимся к командному интерфейсу с помощью нажатия правой кнопки мыши по «Конфигурации» в дереве (рис. 1.12.24).

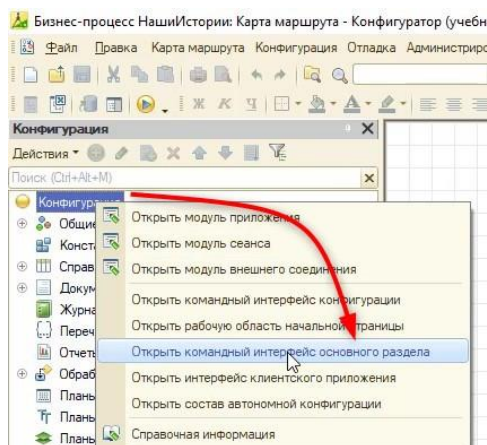


Рис. 1.12.24. Открытие командного интерфейса

Важно!

Командный интерфейс – это основное средство навигации пользователя по функциям конфигурации.

Выключим видимость у задач (рис. 1.12.25).

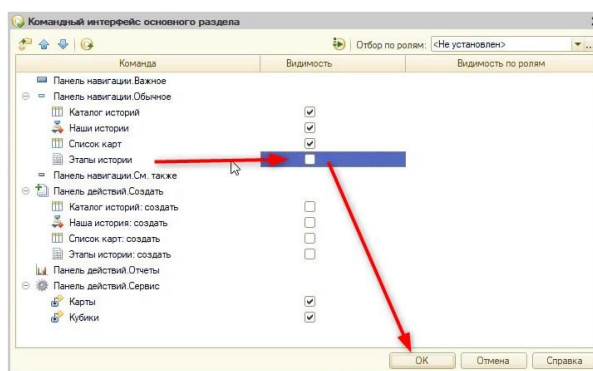


Рис. 1.12.25. Скрытие задач в интерфейсе

Далее обратимся к настройкам бизнес-процесса. У бизнес-процесса необходимо добавить один реквизит – «История» (рис. 1.12.26), с помощью которого игрок будет выбирать историю, которую хочет пройти.

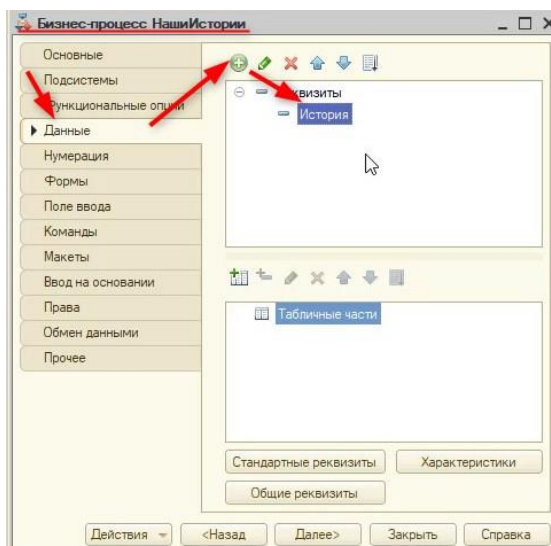


Рис. 1.12.26. Создание реквизита

Тип нового реквизита будет ссылка на справочник «КаталогИсторий» (рис. 1.12.27).

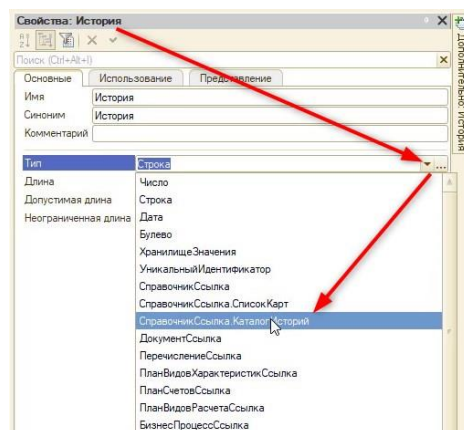


Рис. 1.12.27. Выбор типа реквизита

Данный реквизит должен быть обязательным к заполнению, включить это можно на вкладке «Представление» в свойстве «Проверка заполнения» (рис. 1.12.28).

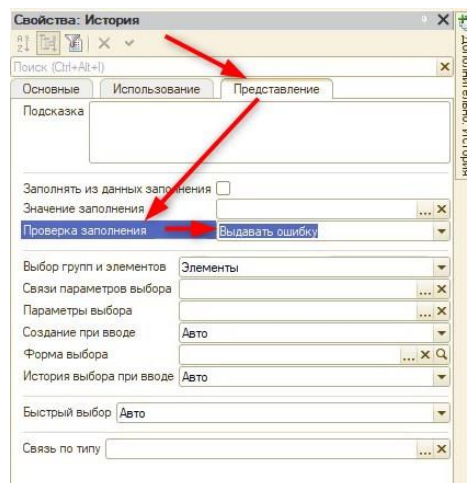


Рис. 1.12.28. Включение проверки заполнения

Нужно создать формы списка и бизнес-процесса. Для начала разберемся с формой списка и создадим ее на вкладке «Формы» (рис. 1.12.29).

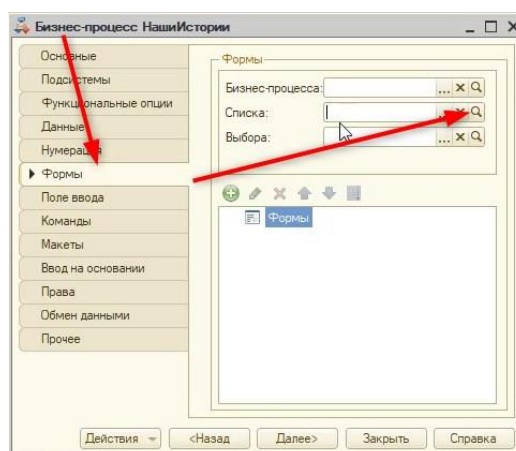


Рис. 1.12.29. Создание формы списка

Включим отображения поля «История» и уберем видимость поля «ВедущаяЗадача» (рис. 1.12.30).

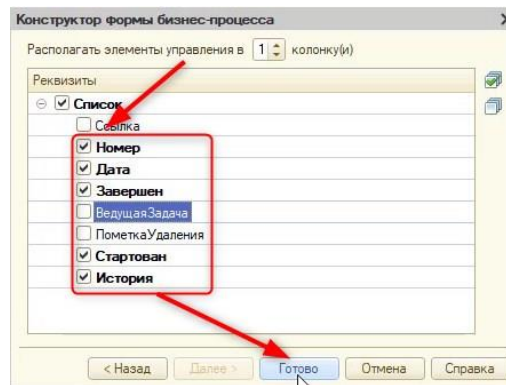


Рис. 1.12.30. Выбор отображения полей

В конструкторе поменяем порядок элементов, как показано на скриншоте 1.12.31, с помощью стрелок «Вверх» и «Вниз».

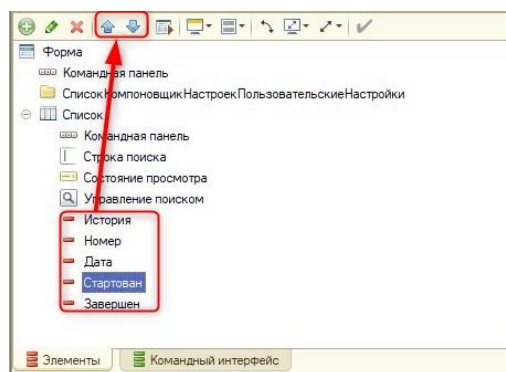


Рис. 1.12.31. Смена расположения элементов

Поменяем заголовки у элементов «Стартован» и «Завершен» на «Начата» и «Завершена» соответственно.

Далее сделаем форму самого бизнес-процесса. В этой форме на вкладке конструктора включим отображение только у поля «История». Форма будет выглядеть следующим образом (рис. 1.12.32).

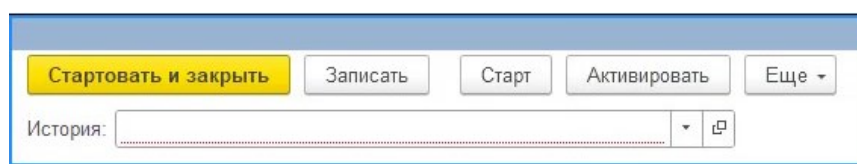


Рис. 1.12.32. Форма элемента бизнес-процесса

Выключим командную панель на форме, как делали ранее – через свойства самой формы (рис. 1.12.33).

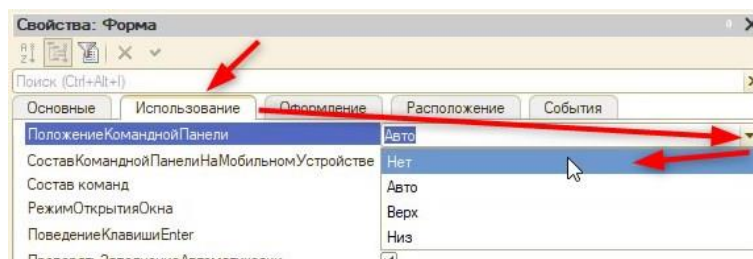


Рис. 1.12.33. Выключение командной панели

Добавим стандартную команду «Стартовать и закрыть» на форму (рис. 1.12.34).

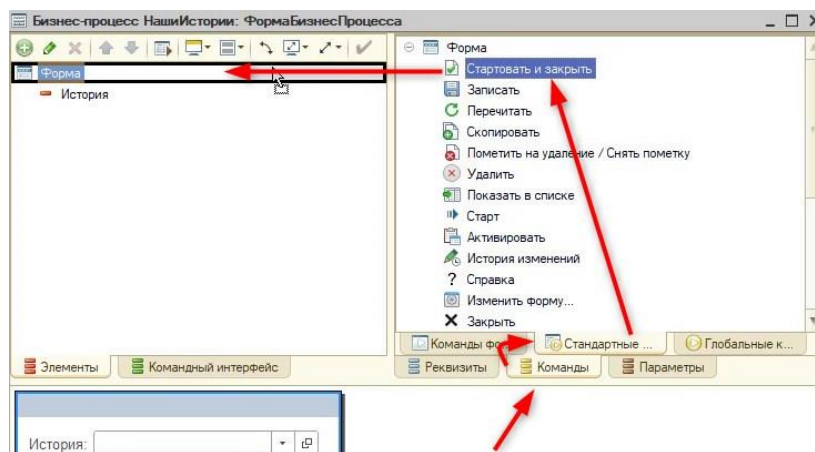


Рис. 1.12.34. Добавление команды на форму

Чтобы отобразить те этапы, которые относятся к текущему процессу, на вкладке «Командный интерфейс» нужно включить видимость кнопки перехода (рис. 1.12.35).

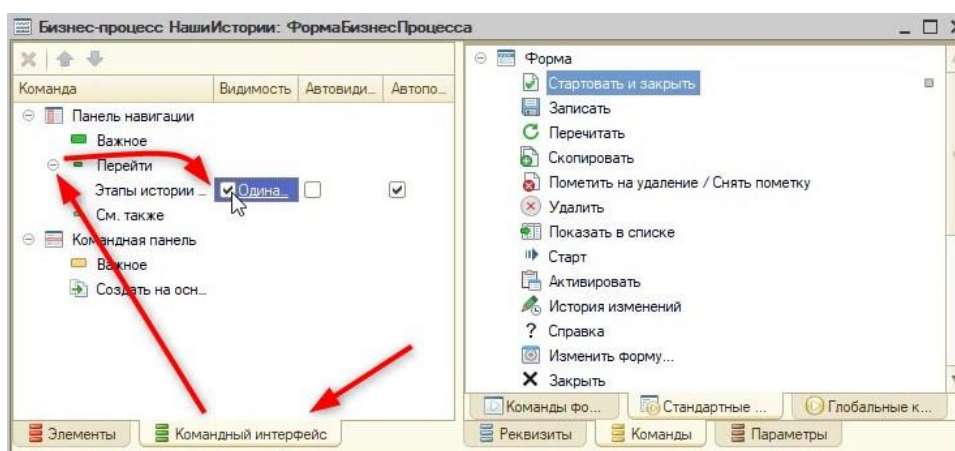


Рис. 1.12.35. Включение видимости задач

Таким образом, на дополнительной вкладке будут открываться те этапы, которые отфильтрованы по принадлежности к текущему бизнес-процессу. В пользовательском режиме это будет выглядеть следующим образом (рис. 1.12.36 – 1.12.37).

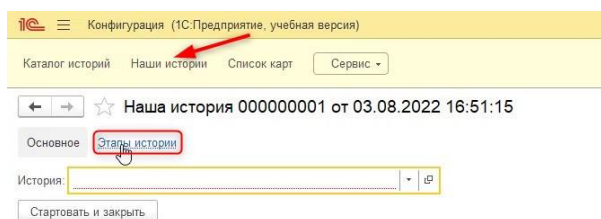


Рис. 1.12.36. Открытие этапов истории бизнес-процесса

Дата	Номер	Наименование	Выполнена	Бизнес-процесс	Точка маршрута
03.08.2022 16:51:55	1	Начало истории	✓	Наша история 000000001 от 03.08.2022 16:51:15	Начало истории
03.08.2022 16:52:23	2	Продолжение истории	✓	Наша история 000000001 от 03.08.2022 16:51:15	Продолжение истории
03.08.2022 16:52:35	3	Завершение истории	✓	Наша история 000000001 от 03.08.2022 16:51:15	Завершение истории

Рис. 1.12.37. Этапы истории

Далее необходимо создать механизм, который по кнопке «Стартовать и закрыть» будет открывать форму первой задачи. Для этого понадобится «отловить» событие «ПриЗакрытии». Создадим новый обработчик (рис. 1.12.38) через свойства самой формы бизнес-процесса.

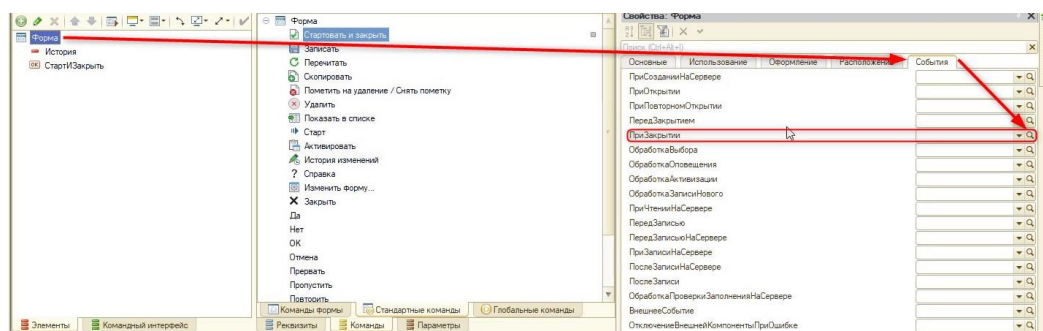


Рис. 1.12.38. Создание обработчика события «ПриЗакрытии»

Приступим к написанию кода:

&НаКлиенте

//Параметр "ЗавершениеРаботы" отвечает за то, как закрывается форма

//Через закрытие программы = Истина, через "крестик" формы = Ложь

Процедура ПриЗакрытии(ЗавершениеРаботы)

 //Если бизнес-процесс не завершен и программу не закрыли тогда

 Если Объект.Завершен = Ложь И ЗавершениеРаботы = Ложь Тогда

 //Далее необходимо получить ссылку на новый этап

 //Для этого создадим новую функцию "ПолучитьНовыйЭтап()"

 НовыйЭтапСсылка = ПолучитьНовыйЭтап();

 КонецЕсли;

КонецПроцедуры

Создадим новую функцию, которую вызвали в процедуре. Так как информацию нужно будет получить из базы данных, функция будет серверной. Создав «тело» функции, воспользуемся конструктором запроса с обработкой результата (рис. 1.12.39).

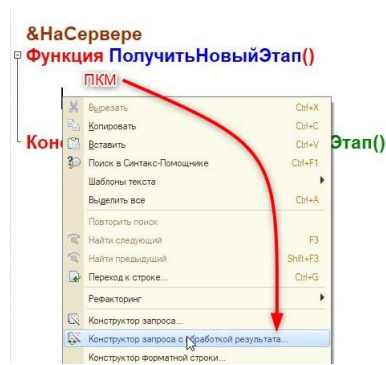


Рис. 1.12.39. Вызов конструктора запроса

Типом обработки будет обход результата. Необходимо среди этапов истории получить реквизит «Ссылка» (рис. 1.12.40).

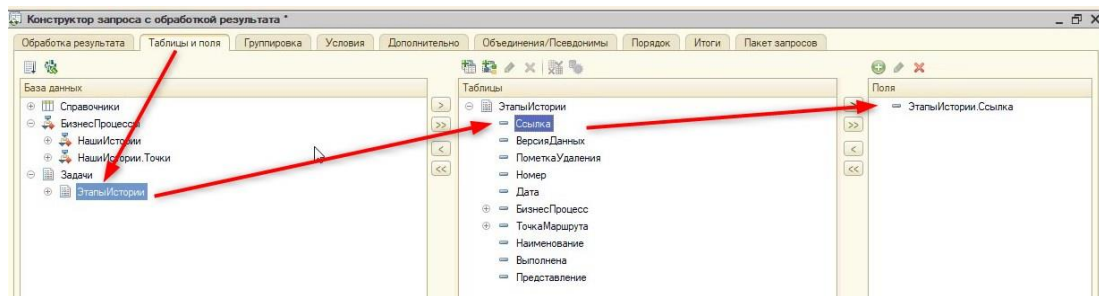


Рис. 1.12.40. Вкладка «Таблицы и поля»

Извлекать эту ссылку нужно при двух условиях (рис. 1.12.41):

- 1) Новая задача не выполнена;
- 2) Бизнес-процесс равен какому-то значению.

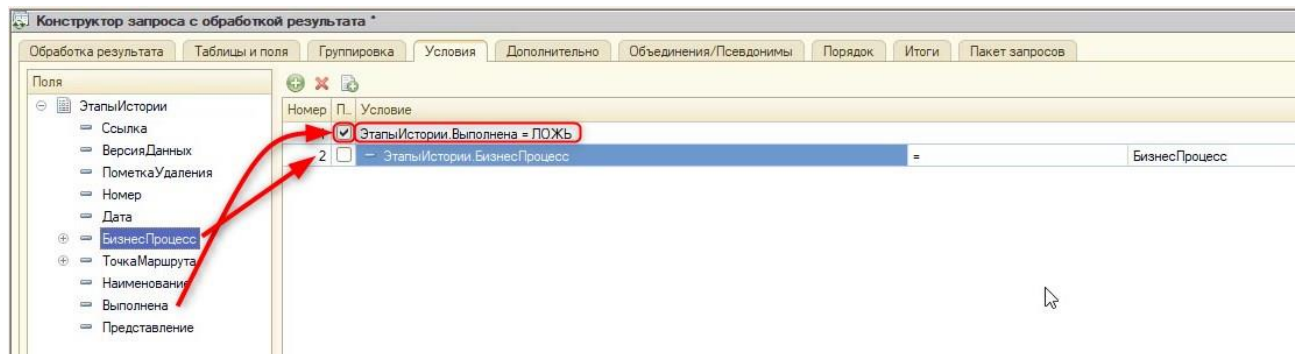


Рис. 1.12.41. Вкладка «Условия»

Нажмем «ОК». В результате получится следующий код с помощью конструктора, который необходимо изменить:

```
&НаСервере
Функция ПолучитьНовыйЭтап()
    Запрос = Новый Запрос;
    Запрос.Текст =
        "ВЫБРАТЬ
        |     ЭтапыИстории.Ссылка КАК Ссылка
        |ИЗ
        |     Задача.ЭтапыИстории КАК ЭтапыИстории
        |ГДЕ
```

```
|      ЭтапыИстории.Выполнена = ЛОЖЬ
|      И ЭтапыИстории.БизнесПроцесс = &БизнесПроцесс";
```

```
Запрос.УстановитьПараметр("БизнесПроцесс", Объект.Ссылка);
РезультатЗапроса = Запрос.Выполнить();
ВыборкаДетальныеЗаписи = РезультатЗапроса.Выбрать();
//Выборку не нужно прокручивать в цикле, поэтому уберём его, оставив:
ВыборкаДетальныеЗаписи.Следующий();
//Возвратим значения
Возврат ВыборкаДетальныеЗаписи.Ссылка;
КонецФункции // ПолучитьНовыйЭтап()
```

Далее вернемся в процедуру и обработаем результат:

```
&НаКлиенте
Процедура ПриЗакрытии(ЗавершениеРаботы)
    Если Объект.Завершен = Ложь И ЗавершениеРаботы = Ложь Тогда
        НовыйЭтапСсылка = ПолучитьНовыйЭтап();
        //Если ссылка получена
        Если НовыйЭтапСсылка <> Неопределено Тогда
            //Создадим ключ для поиска формы по ссылке:
            СтруктураОтбора = Новый Структура("Ключ", НовыйЭтапСсылка);
            //Получим имя формы:
            Форма = ПолучитьФорму("Задача.ЭтапыИстории.ФормаОбъекта",
СтруктураОтбора);
            //Откроем форму с помощью метода "ОткрытьФормы"
            ОткрытьФорму(Форма);
        КонецЕсли;
    КонецЕсли;
КонецПроцедуры
```

Готово! Следующее, что необходимо реализовать – это открытие этапов друг за другом.

Занятие 13 – Автоматическое открытие форм для игры «Квест»

Далее необходимо реализовать открытие формы следующей задачи по кнопке «Дальше» (рис. 1.13.1). Для этого будет использоваться событие «ПриЗакрытии», поэтому важно будет реализовывать проверку того, что форма закрылась именно с помощью кнопки, а не «Крестика».

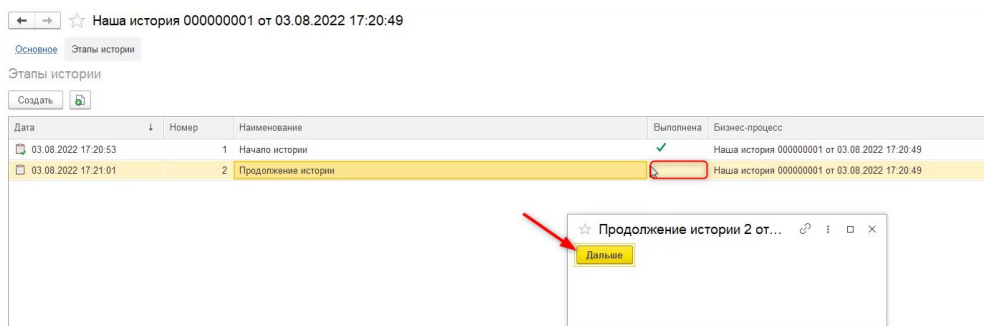


Рис. 1.13.1. Задача «Продолжение истории»

Перед этим создадим собственную форму списка с помощью дерева конфигурации (рис. 1.13.2).

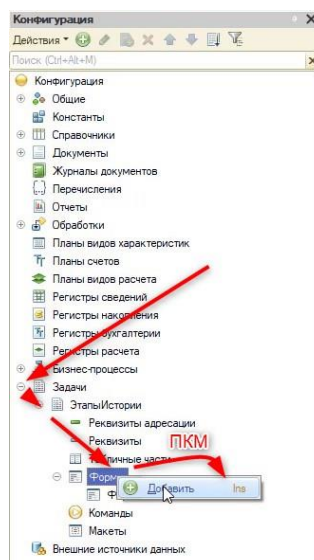


Рис. 1.13.2. Добавление формы через дерево конфигурации

Далее укажем тип формы «Форма списка задачи», обязательно проверим включенный флаг «Назначить форму основной» и нажмем «Далее» (рис. 1.13.3).

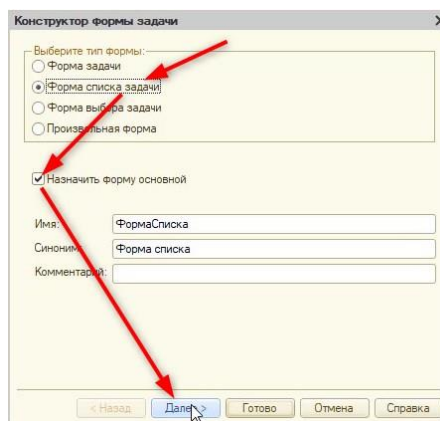


Рис. 1.13.3. Конструктор формы задачи

Включим видимость только реквизитов «Наименование», «Дата» и «Выполнена» (рис. 1.13.4).

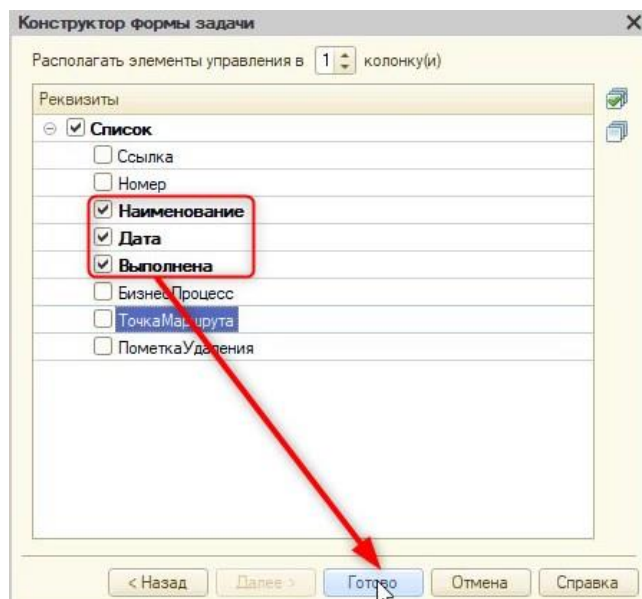


Рис. 1.13.4. Настройка реквизитов

Обратимся к реквизиту «БизнесПроцесс» и вынесем из него на форму реквизит «История» (рис. 1.13.5).

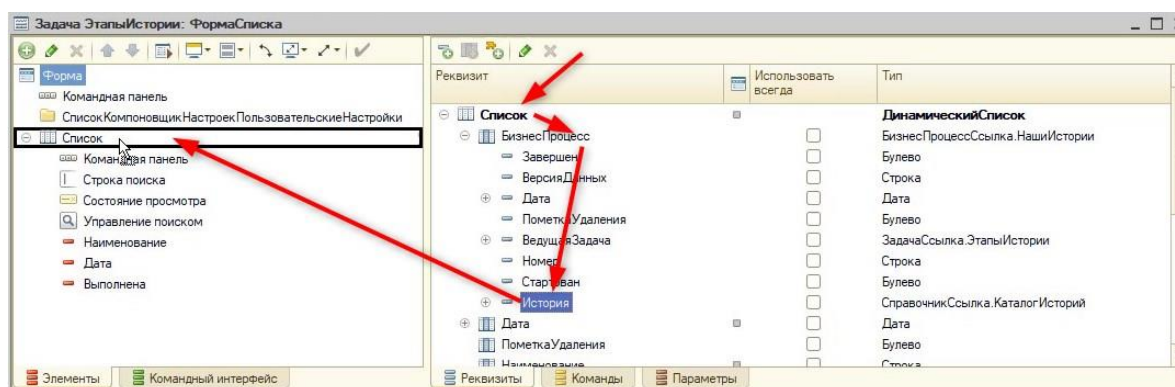


Рис. 1.13.5. Добавление элемента «История»

В результате последовательность элементов настроим следующим образом (рис. 1.13.6).

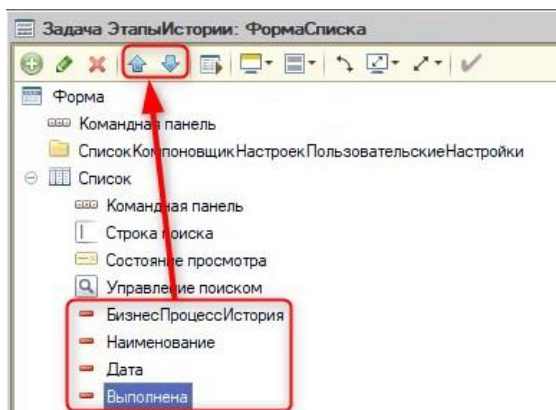


Рис. 1.13.6. Добавление элемента «История»

Назначим заголовок «Этап» для элемента «Наименование» (рис. 1.13.7).

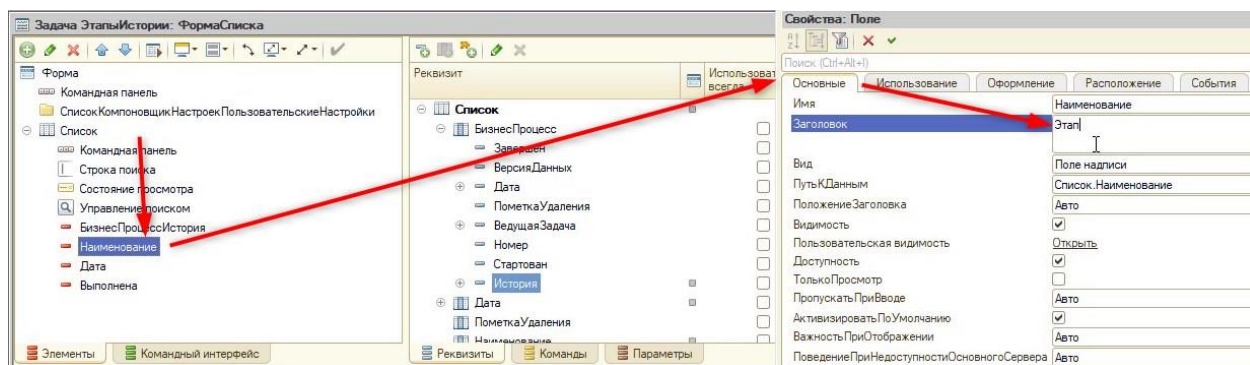


Рис. 1.13.7. Смена заголовка

Теперь приступим к настройке формы задачи, откроем ее через дерево конфигурации (рис. 1.13.8).

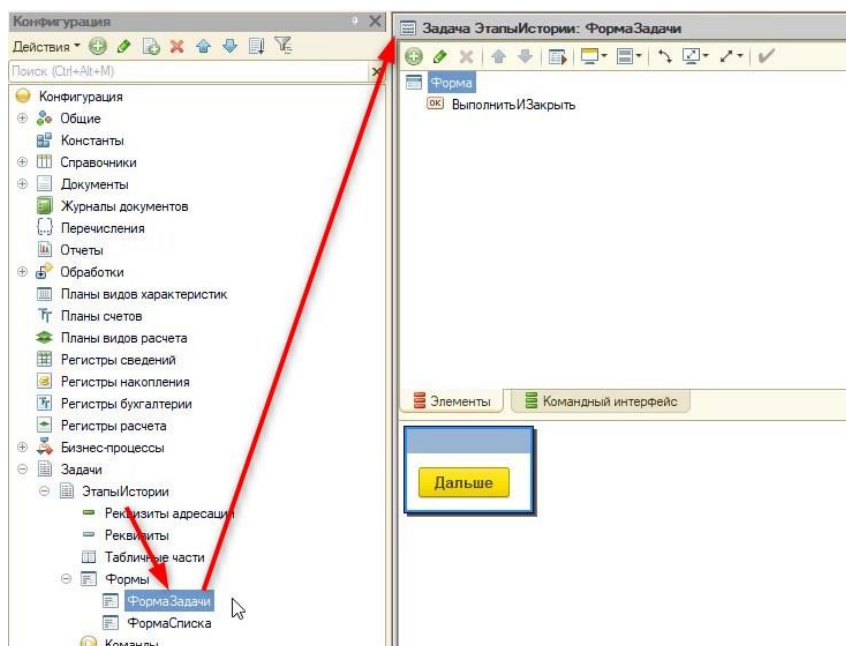


Рис. 1.13.8. Форма задачи

Необходимо создать обработчик события формы «ПриЗакрытии» (рис. 1.13.9).

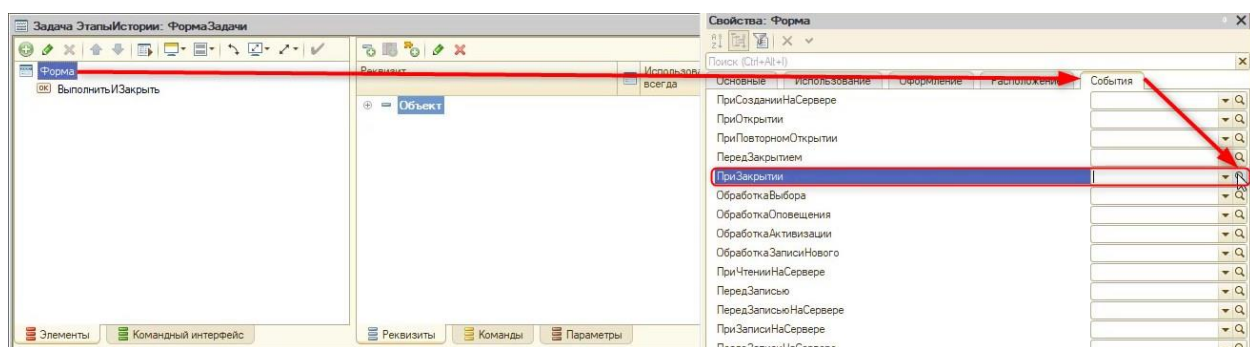


Рис. 1.13.9. Создание обработчика «ПриЗакрытии»

Определим его на клиенте и приступим к программированию:

&НаКлиенте

Процедура ПриЗакрытии(ЗавершениеРаботы)

//Получим ссылку на этап, как делали ранее

//С Помощью новой серверной функции «ПолучитьНовыйЭтап»

НовыйЭтапСсылка = ПолучитьНовыйЭтап();

Далее создадим серверную функцию и вызовем конструктор запроса с обработкой результата (рис. 1.13.10).

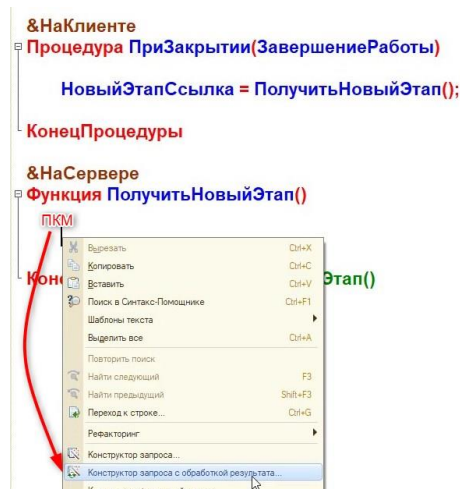


Рис. 1.13.10. Открытие конструктора

Тип обработки оставляем «Обход результата» и переходим на вкладку «Таблицы и поля». Из списка задач достанем реквизит «Ссылка» (рис. 1.13.11).

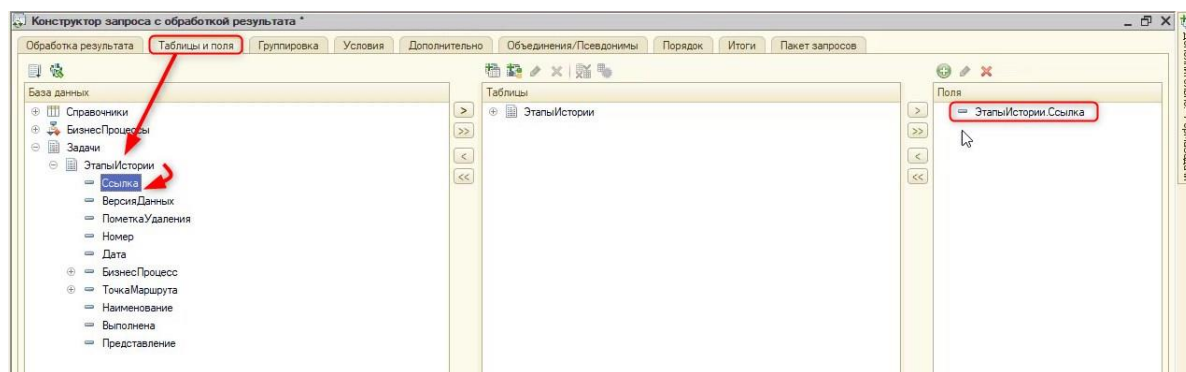


Рис. 1.13.11. Вкладка «Таблицы и поля»

Нужна ссылка с условием (рис. 1.13.12), что:

- 1) Ссылка должна быть не равной («<>») ссылке задачи, которая в данный момент открыта;
- 2) Задача не должна быть выполнена (выполнена равняется значению «Ложь»);
- 3) Задача должна относиться к текущему бизнес-процессу.

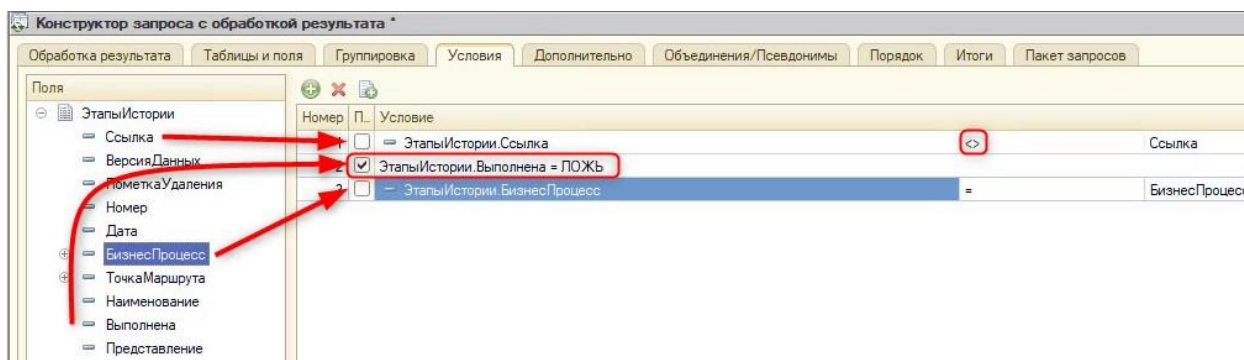


Рис. 1.13.12. Вкладка «Условия»

Нажмем «ОК» и отредактируем код конструктора:

&НаСервере

Функция ПолучитьНовыйЭтап()

Запрос = Новый Запрос;

Запрос.Текст =

 "ВЫБРАТЬ

 | ЭтапыИстории.Ссылка КАК Ссылка

 |ИЗ

 | Задача.ЭтапыИстории КАК ЭтапыИстории

 |ГДЕ

 | ЭтапыИстории.Ссылка <> &Ссылка

 | И ЭтапыИстории.Выполнена = ЛОЖЬ

 | И ЭтапыИстории.БизнесПроцесс = &БизнесПроцесс";

Запрос.УстановитьПараметр("БизнесПроцесс", **Объект.БизнесПроцесс**);

Запрос.УстановитьПараметр("Ссылка", **Объект.Ссылка**);

РезультатЗапроса = Запрос.Выполнить();

ВыборкаДетальныеЗаписи = РезультатЗапроса.Выбрать();

ВыборкаДетальныеЗаписи.Следующий();

Возврат ВыборкаДетальныеЗаписи.Ссылка;

КонецФункции

Вернемся и проанализируем результат запроса в процедуре:

&НаКлиенте

Процедура ПриЗакрытии(ЗавершениеРаботы)

 НовыйЭтапСсылка = ПолучитьНовыйЭтап();

 //Ссылка на новый этап заполнена, и работа завершена не через закрытие программы

Если НовыйЭтапСсылка <> Неопределено И ЗавершениеРаботы = Ложь Тогда

 //Аналогично как делали ранее получаем ссылку на форму:

СтруктураОтбора = Новый Структура("Ключ", НовыйЭтапСсылка);

Форма = ПолучитьФорму("Задача.ЭтапыИстории.ФормаОбъекта",
СтруктураОтбора);

ОткрытьФорму(Форма);

КонецЕсли;

КонецПроцедуры

Проверим код в пользовательском режиме.

Если нужно, можно удалить лишние бизнес-процессы. Но перед этим обязательно удалим все задачи, связанные с этим бизнес-процессом, с помощью горячей клавиши «Shift+Del» или кнопок «Еще» – «Удалить» (рис. 1.13.13).

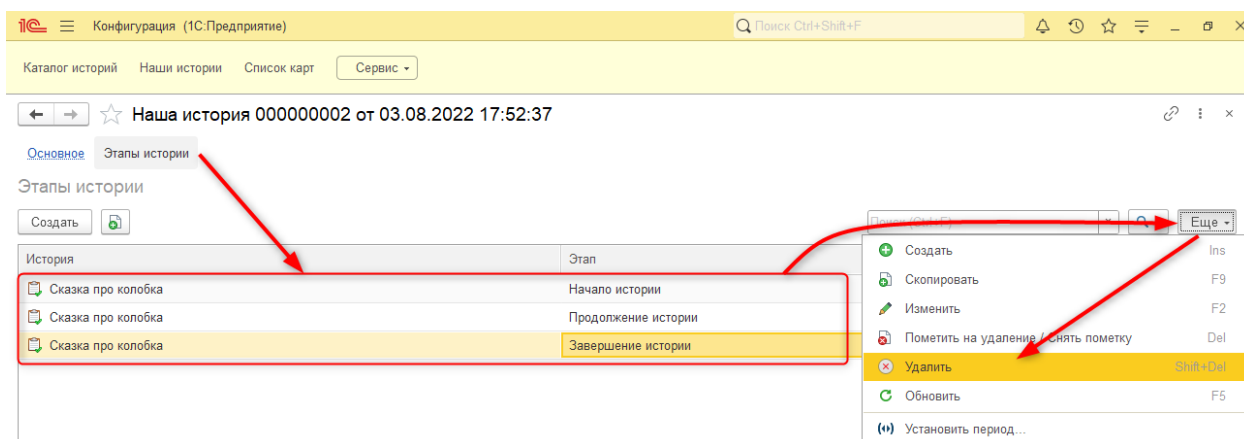


Рис. 1.13.13. Удаление задач бизнес-процесса

Если этого не сделать, то в системе возникнут битые ссылки «<Объект не найден>». Если не исправить такую ситуацию, то возникнут сложности при обмене данными.

После удаления задач бизнес-процесс можно удалить аналогичным способом.

На данный момент форма пустая, и необходимо добавить изображение и текст. Для этого на форме добавим два новых реквизита формы – «Картинка» и «Комментарий» с типом «Строка» (рис. 1.13.14).

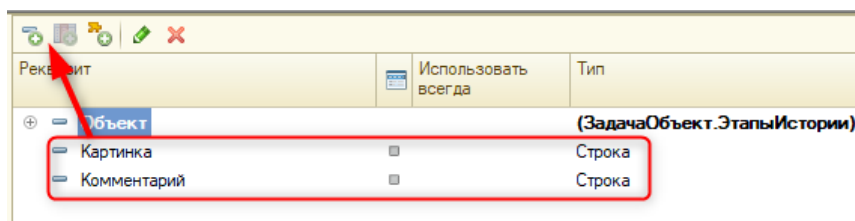


Рис. 1.13.14. Новые реквизиты формы

Переместим данные реквизиты на форму над кнопкой «Дальше» (рис. 1.13.15).

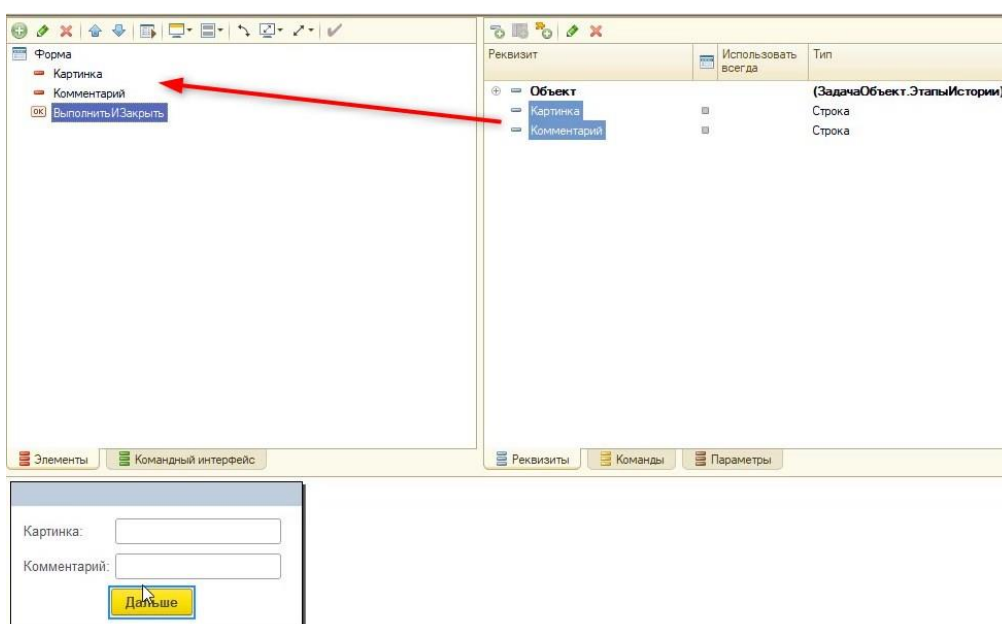


Рис. 1.13.15. Создание элементов для новых реквизитов

У этих полей выключим заголовок в свойствах (рис. 1.13.16).

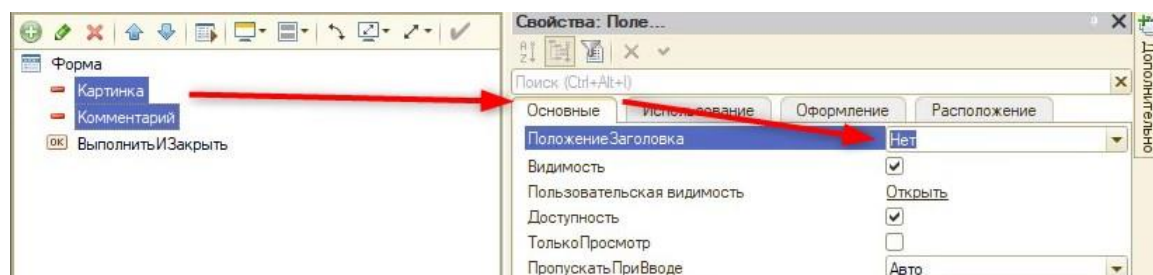


Рис. 1.13.16. Выключение заголовка

Для элемента «Картинка» зададим вид – «Поле картинки» (рис. 1.13.17).

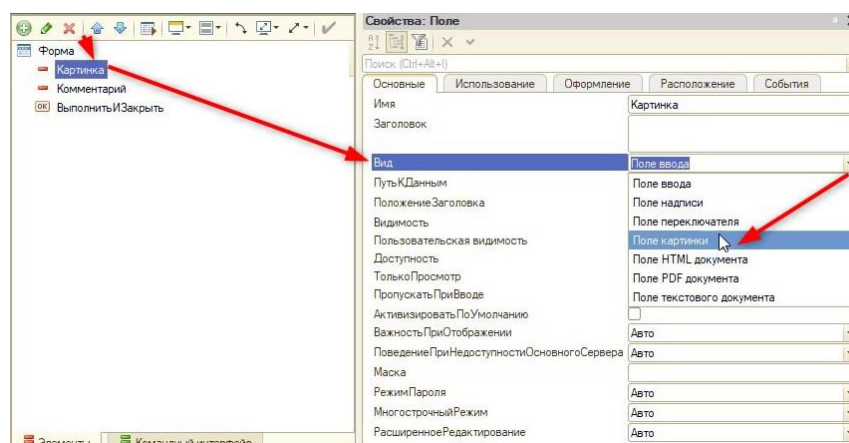


Рис. 1.13.17. Смена вида поля

Поскольку это изображение, то сразу нужно указать пропорциональный размер (рис. 1.13.18).

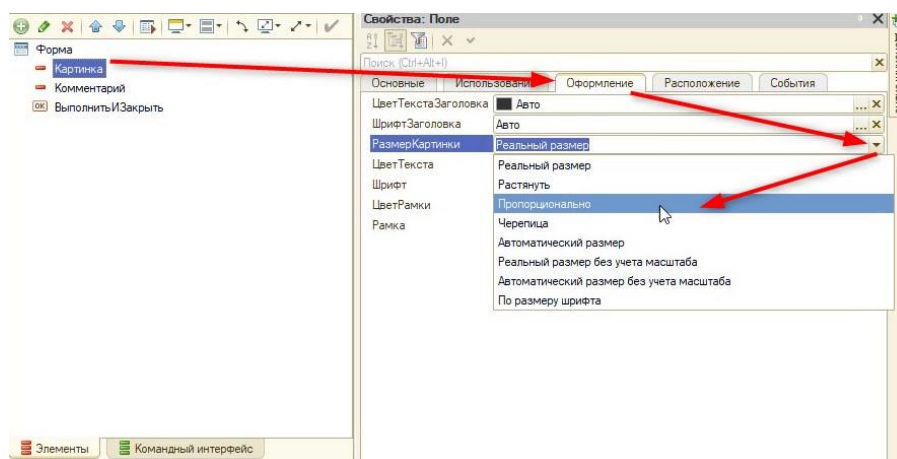


Рис. 1.13.18. Смена вида поля

Для поля «Комментарий» включим многострочный режим (рис. 1.13.19), так как описание истории может быть большим.

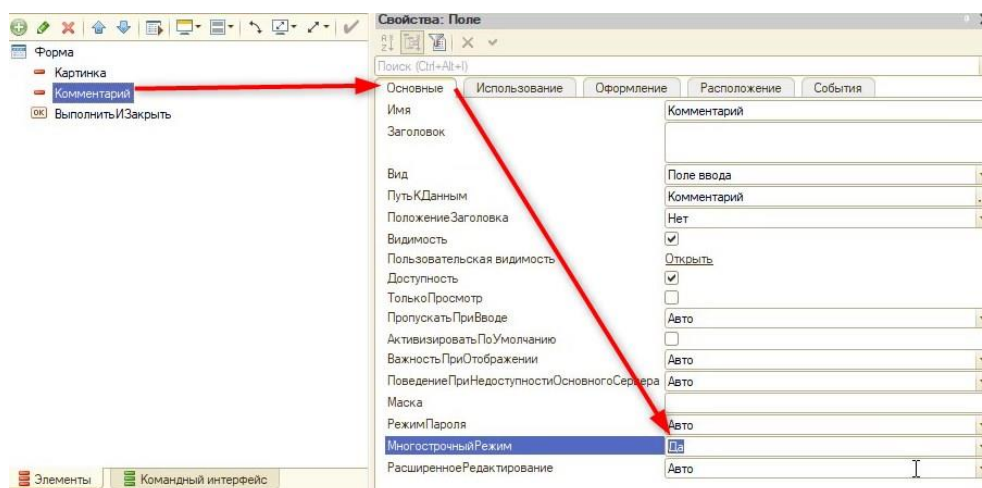


Рис. 1.13.19. Многострочный режим

Чтобы игрок не смог менять текст этапа, включим только просмотр для элемента «Комментарий» (рис. 1.13.20).

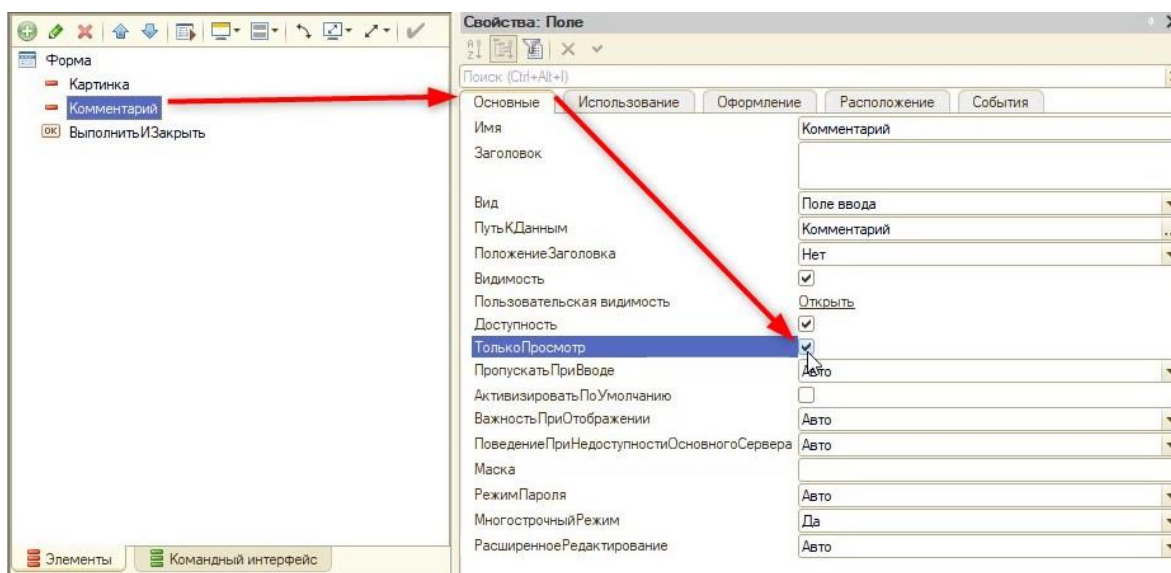


Рис. 1.13.20. Только просмотр

Чтобы поле отображалось исходя из количества текста, выключим автоматические максимальные ширину и высоту (рис. 1.13.21).

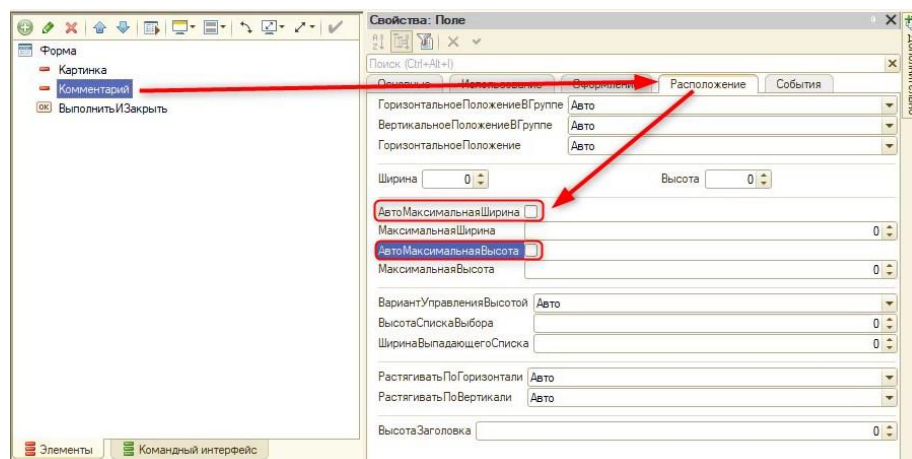


Рис. 1.13.21. Выключение автоматических ширины и высоты

Последнее, что необходимо сделать – это реализовать обработчик события, при котором происходит чтение картинки и текста для нужного этапа и последующая постановка их на форму. Для этого нужно событие «ПриСозданииНаСервере».

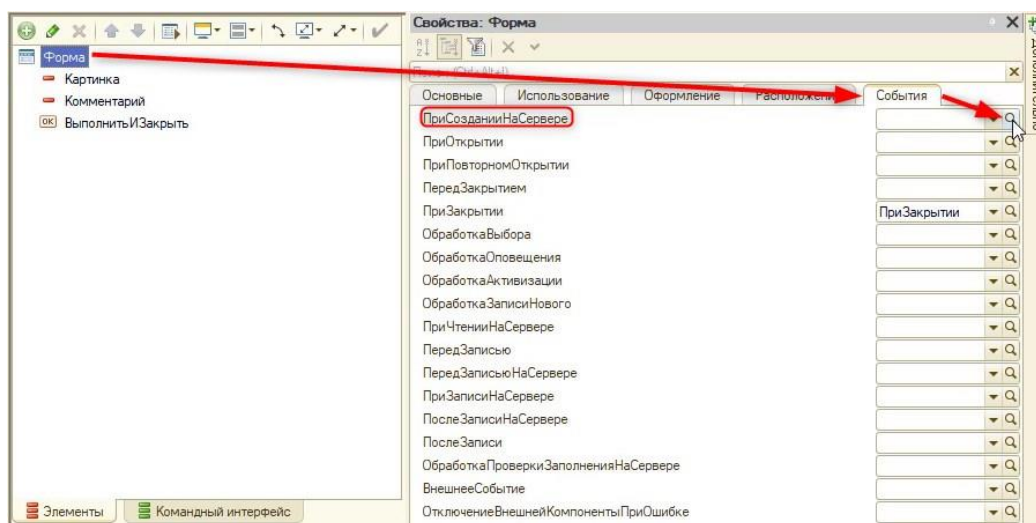


Рис. 1.13.22. Создание новой процедуры

Код процедуры будет выглядеть следующим образом:

&НаСервере

Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

//Получим ссылку на историю, которую в данный момент проходит игрок:

История = Объект.БизнесПроцесс.История;

//Получим имя этапа истории:

ЭтапИстории = Объект.ТочкаМаршрута.Имя;

//Получим навигационную ссылку картинки этапа по ссылке истории и названию этапа

//И поместим реквизит формы:

ЭтотОбъект.Картинка = ПолучитьНавигационнуюСсылку(История, ЭтапИстории);

//Получим комментарий к картинке через название

ЭтотОбъект.Комментарий = История[ЭтапИстории + "Комментарий"];

//Выключим видимость кнопки при условии, если задача выполнена была ранее

//Если задача выполнена:

Если Объект.Выполнена = Истина Тогда

 //Выключим видимость кнопки

 Элементы.ВыполнитьИЗакреть.Видимость = Ложь;

КонецЕсли;

//Сменим название кнопки, если происходит последний этап истории:

//Если происходит последний этап:

Если ЭтапИстории = "ЗавершениеИстории" Тогда

 //Сменим название кнопки на "Завершить историю"

 Элементы.ВыполнитьИЗакреть.Заголовок = "Завершить историю";

КонецЕсли;

КонецПроцедуры

Отлично! Осталось протестировать игру на ошибки.

Практикум №1

Создать собственную историю с картинками и описанием. Если понадобится, подкорректировать размер и стиль шрифта текста (рис. 1.13.23).

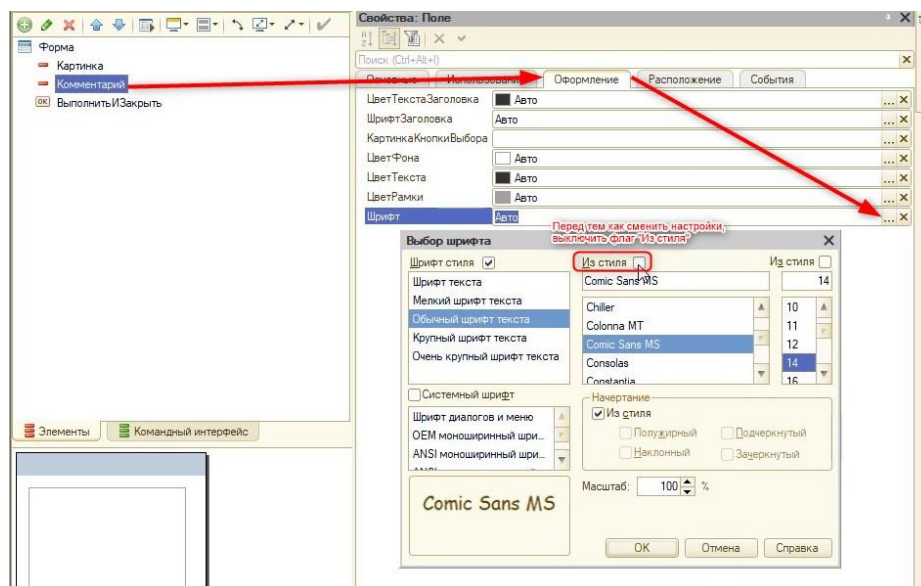


Рис. 1.13.23. Смена стиля шрифта

Занятие 14 – Возможность выбора в игре «Квест»

Во всех квестовых играх всегда есть возможность выбора, которая приводит к разным концовкам. На данный момент такого в созданной игре нет. Реализуем это.

Для этого откроем карту маршрута бизнес-процесса «Наши истории» (рис. 1.14.1).

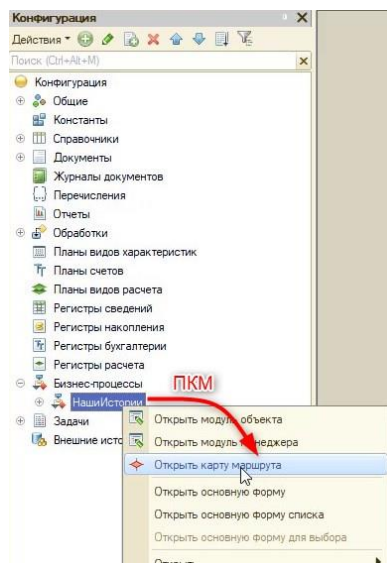


Рис. 1.14.1. Открытие карты маршрута

Как и было сказано ранее, квест получился линейным, но при этом справочник с описанием историй содержит в себе 2 разветвления.

В карте маршрута есть два механизма, которые могут создать «развилку» для квеста:

- 1) Точка условия;
- 2) Точка выбора варианта.

Больше для реализации подойдет точка выбора варианта, так как с точкой условия у игрока будет выбор только между «Да» или «Нет».

Создадим точку выбора на карте маршрута (рис. 1.14.2).

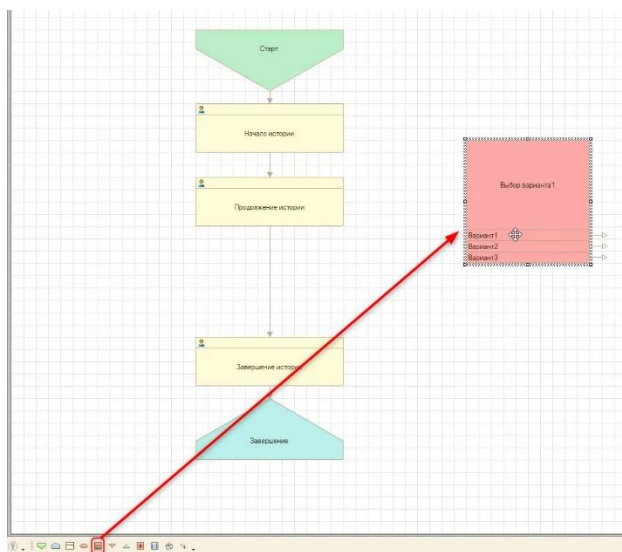


Рис. 1.14.2. Создание точки выбора варианта

В квесте будет только два варианта развития, поэтому третий необходимо удалить (рис. 1.14.3).

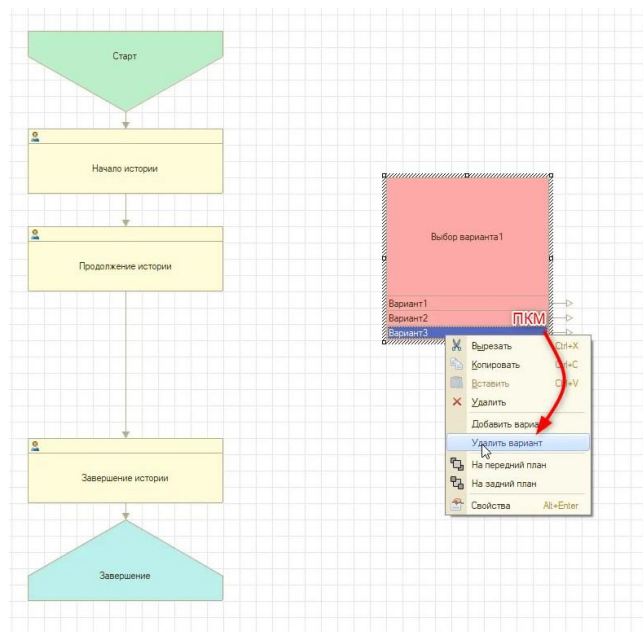


Рис. 1.14.3. Удаление варианта в точке выбора

Назовем точку выбора варианта «ВариантыВыбора» (рис. 1.14.4).

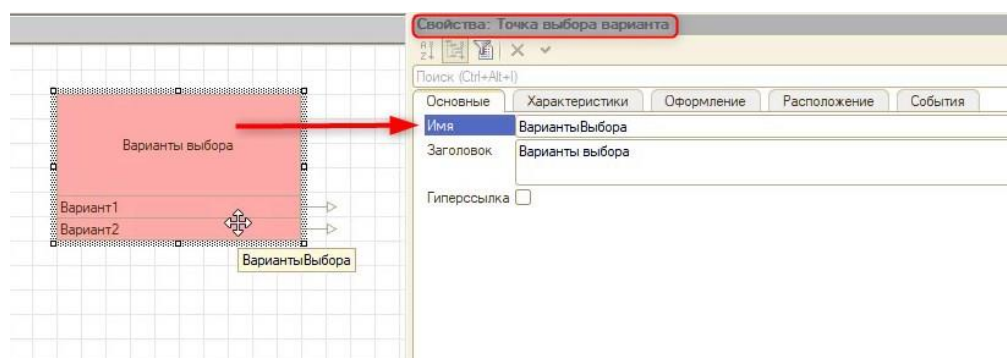


Рис. 1.14.4. Свойства точки выбора варианта

Далее переименуем первый вариант в «ВариантПобеды» (рис. 1.14.5) через вкладку «Вариант». Если изменить название через вкладку «Основные», то поменяется название самой точки выбора.

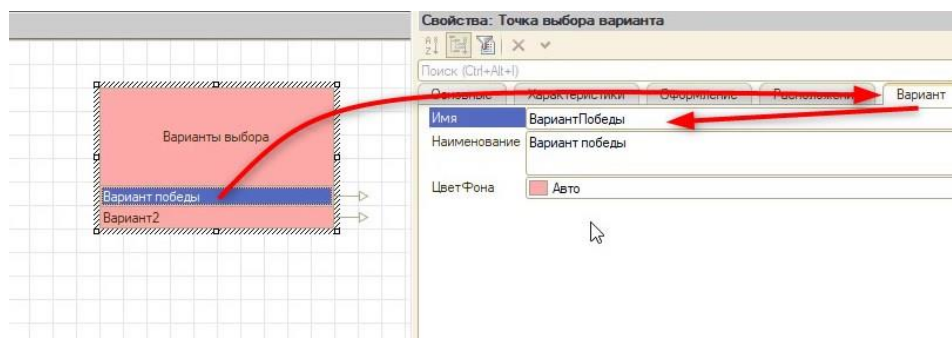


Рис. 1.14.5. Удаление варианта в точке выбора

Второй вариант, также через вкладку варианты, переименуем в «Вариант Проигрыша» (рис. 1.14.7).

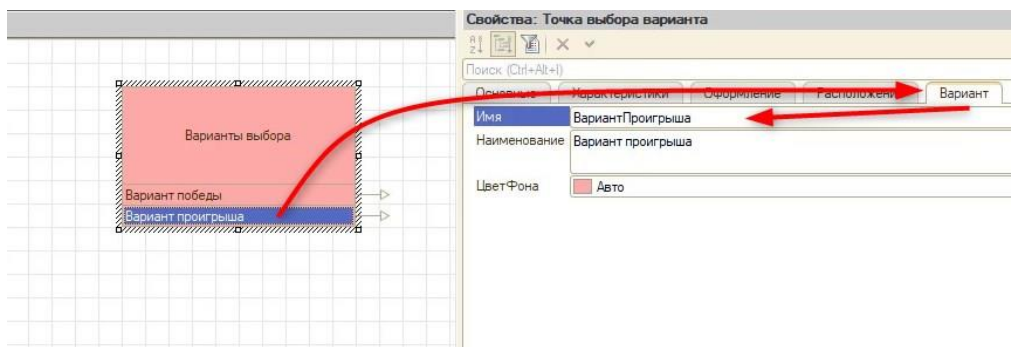


Рис. 1.14.7. Смена названия варианта

Вариант победы должен вести к завершению истории, поэтому стрелку из варианта победы соединяем с «ЗавершениеИстории» (рис. 1.14.8).

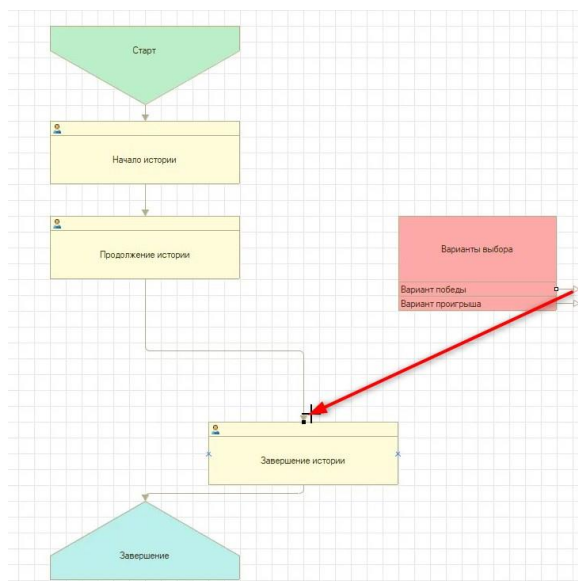


Рис. 1.14.8. Соединение

Добавим на карту маршрута новую точку действия «ВыборВИстории» и соединим ее с «Продолжение истории» (рис. 1.14.9).

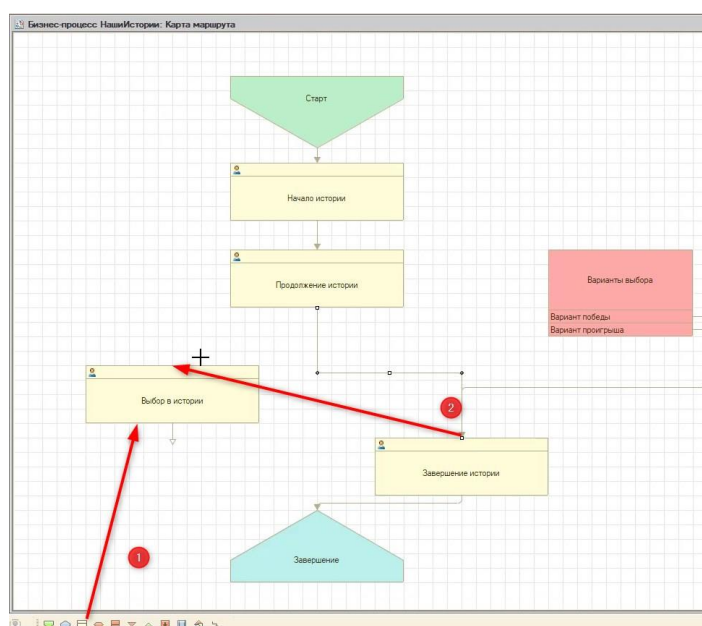


Рис. 1.14.9. Создание новой точки действия и соединение

После этого соединим «Выбор в истории» с «Вариантами выбора» (рис. 1.14.10).

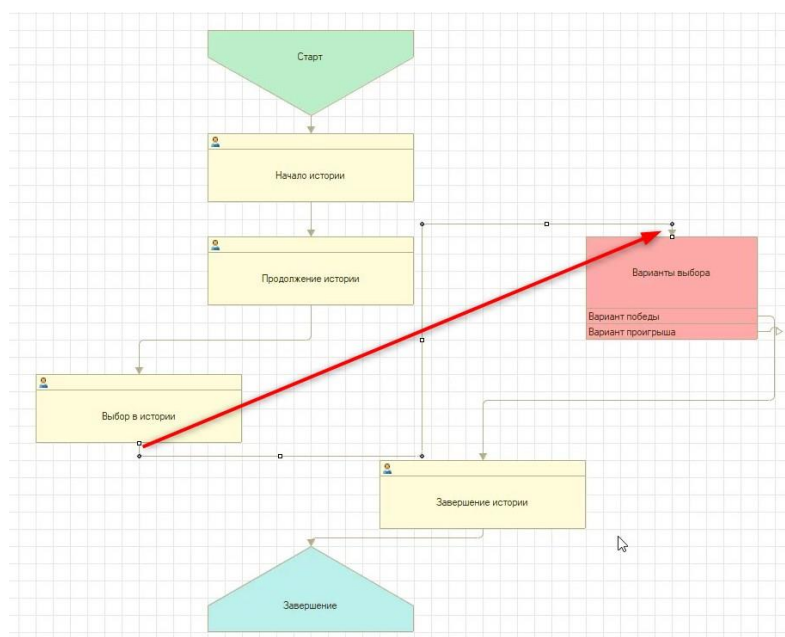


Рис. 1.14.10. Соединение точки действия с точкой выбора

Передвинем точки на схеме, чтобы она выглядела лучше и понятнее (рис. 1.14.11).

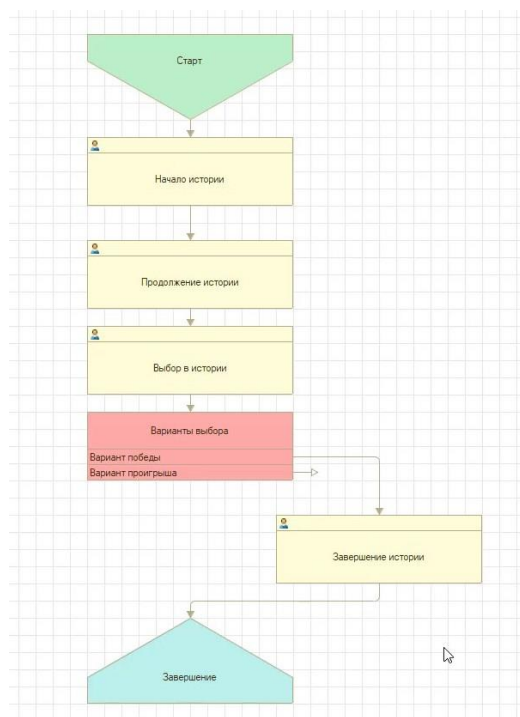


Рис. 1.14.11. Схема маршрута

У стрелок, которые соединяют элементы схемы, есть две точки для взаимодействия (рис. 1.14.12). С помощью красной точки можно соединить элементы, а с помощью белой – перенести начало стрелки.

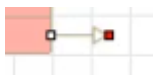


Рис. 1.14.12. Стрелка для соединения элементов

С зажатием левой кнопки мыши на белой точке переместим стрелку в левую часть варианта выбора (рис. 1.14.13).

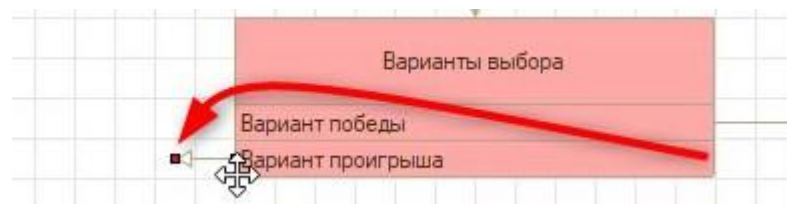


Рис. 1.14.13. Перемещение стрелки

Добавим новую точку действия «ПлохоеЗавершениеИстории» и соединим ее с точкой выбора и завершения, чтобы схема выглядела следующим образом (рис. 1.14.14).

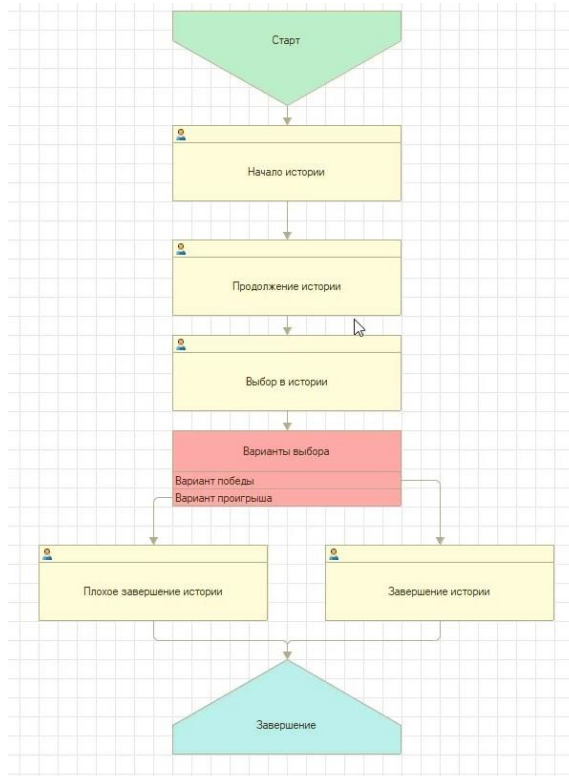


Рис. 1.14.14. Итоговая схема маршрута

Для того чтобы система понимала, как считывать выбор игрока, понадобится обработчик выбора варианта. Создать его можно через свойства точки выбора варианта (рис. 1.14.15).

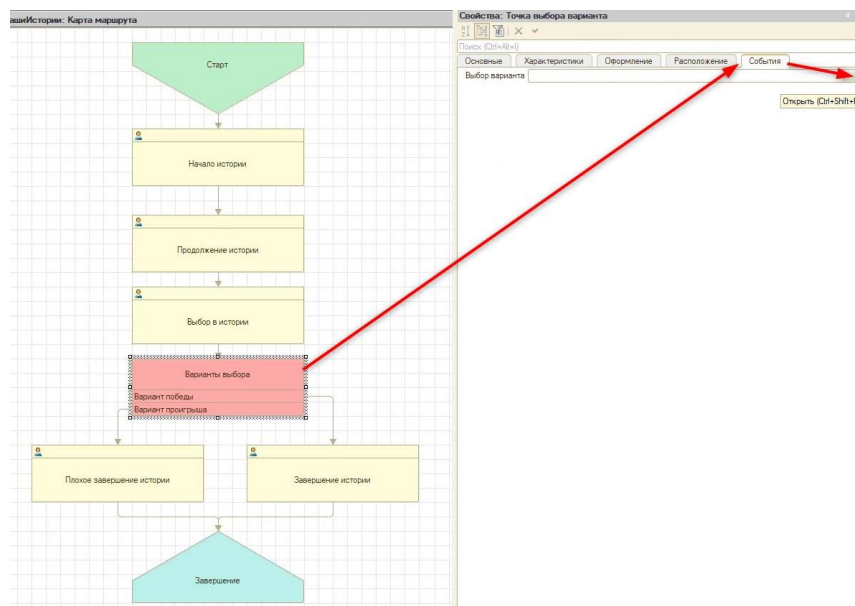


Рис. 1.14.15. Добавление обработчика события «Выбор варианта»

Перед написанием кода добавим бизнес-процессу реквизит «ОтветНаВопрос» с типом «Строка» неограниченной длины через дерево конфигурации (рис. 1.14.16).

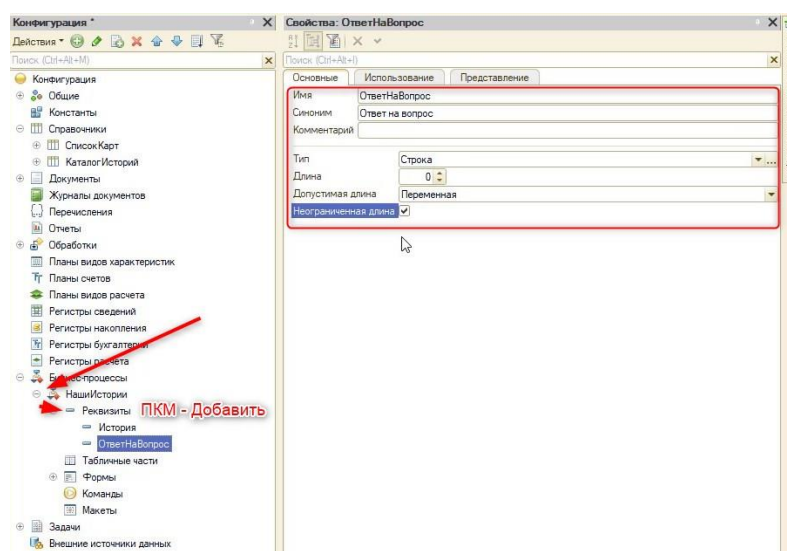


Рис. 1.14.16. Создание нового реквизита

Этот реквизит в дальнейшем будем считывать в новом обработчике исход от сделанного выбора:

```

Процедура ВариантыВыбораОбработкаВыбораВарианта(ТочкаВыбораВарианта, Результат)
    //Если новый реквизит равен значению "ВариантДляПобеды" из справочника "КаталогИсторий"

    Если ЭтотОбъект.ОтветНаВопрос = ЭтотОбъект.История.ВариантДляПобеды Тогда
        //Результат – это параметр, который отвечает за то в какую точку попадёт пользователь
        после выбора
        //Приводим игрока в точку хорошего завершения истории
        Результат = ТочкаВыбораВарианта.Варианты.ВариантПобеды;
    //Обрабатываем вариант проигрыша аналогично
    ИначеЕсли ЭтотОбъект.ОтветНаВопрос = ЭтотОбъект.История.ВариантДляПоражения Тогда
    
```

```
        Результат = ТочкаВыбораВарианта.Варианты.ВариантПроигрыша;  
    КонецЕсли;  
КонецПроцедуры
```

Далее необходимо подредактировать код, написанный в модуле формы задачи. Так как, помимо «ЗавершениеИстории», появился другой этап – «ПлохоеЗавершениеИстории»:

```
&НаСервере
```

```
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)
```

```
    История = Объект.БизнесПроцесс.История;
```

```
    ЭтапИстории = Объект.ТочкаМаршрута.Имя;
```

```
    ЭтотОбъект.Картинка = ПолучитьНавигационнуюСсылку(История, ЭтапИстории);
```

```
    ЭтотОбъект.Комментарий = История[ЭтапИстории + "Комментарий"];
```

```
    Если Объект.Выполнена = Истина Тогда
```

```
        Элементы.ВыполнитьИЗаккрыть.Видимость = Ложь;
```

```
    КонецЕсли;
```

```
    Если ЭтапИстории = "ЗавершениеИстории" ИЛИ ЭтапИстории =  
"ПлохоеЗавершениеИстории" Тогда
```

```
        Элементы.ВыполнитьИЗаккрыть.Заголовок = "Завершить историю";
```

```
    КонецЕсли;
```

```
КонецПроцедуры
```

Готово! На следующем занятии мы создадим вариант выбора уже в пользовательском режиме.

Занятие 15 – Создание интерактивных ответов для игры «Квест»

Теперь нужно добавить взаимодействие с выбором игрока. Для этого на форме задачи у этапов истории добавим две команды – «ОтветПлохой» и «ОтветХороший» (рис. 1.15.1).

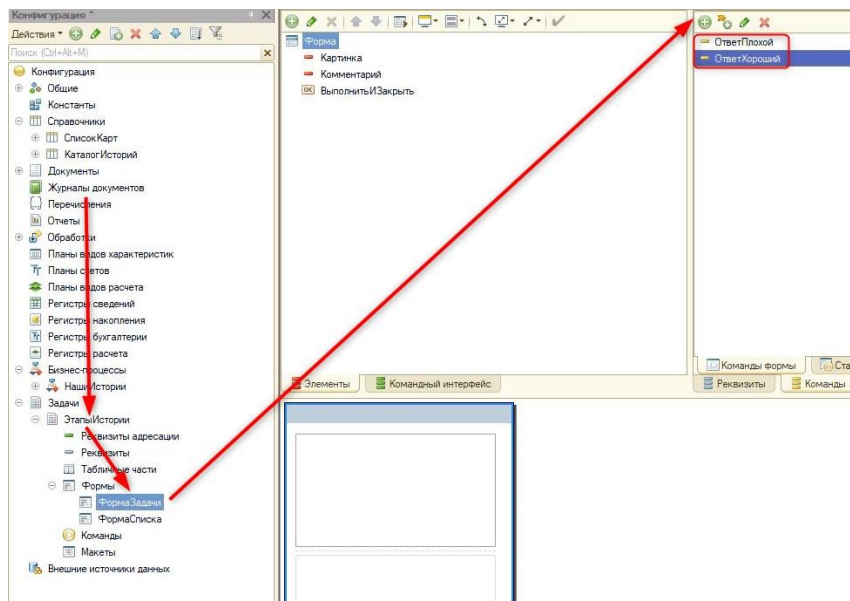


Рис. 1.15.1. Создание команд

Так как происходит разработка общего механизма, нужно учитывать, что необходимо будет менять название кнопок. Вне зависимости от названий, известно, что в квесте есть плохой вариант событий и хороший. В зависимости от выбранной кнопки, нужно будет вести игрока к плохому или хорошему исходу.

Перенесем команды на форму. Как видно на рис. 1.15.2, в таком виде оставлять кнопки нельзя.

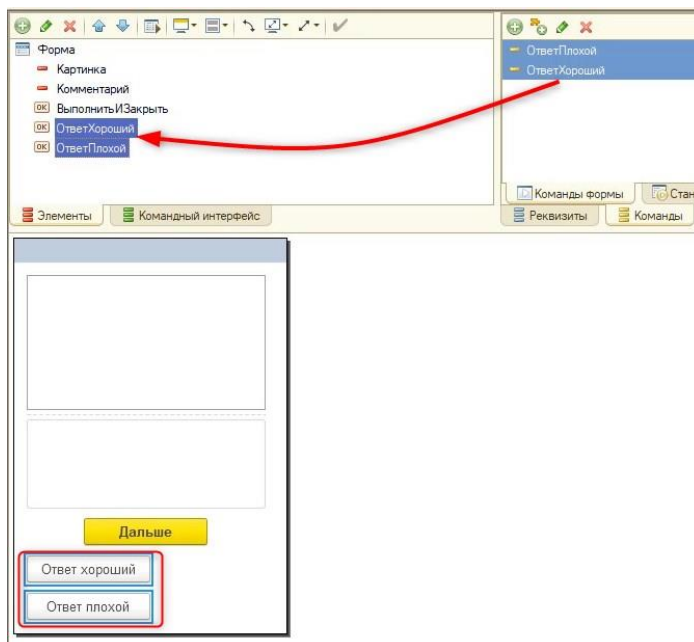


Рис. 1.15.2. Перенос команд на форму

Для более грамотного отображения добавим группу с типом «Группа – Обычная группа без отображения» (рис. 1.15.3).

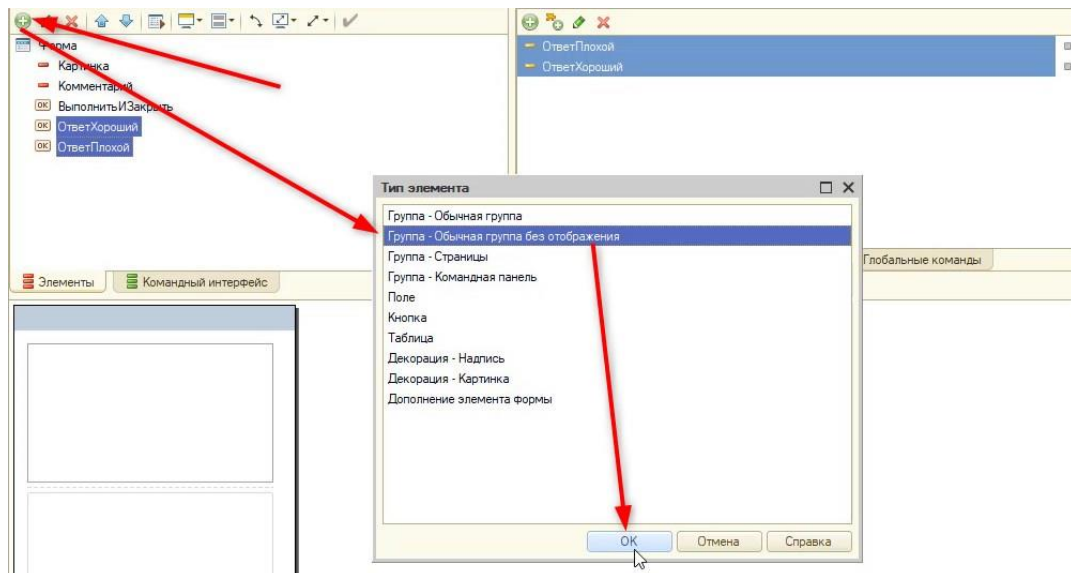


Рис. 1.15.3. Добавление группы

Новая группа будет иметь название «ГруппаОтветы». Далее в эту группу поместим новые кнопки (рис. 1.15.4).

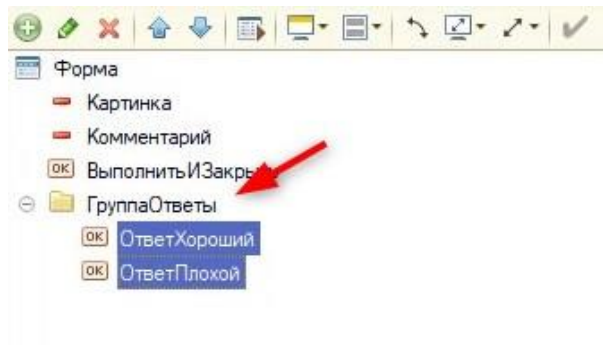


Рис. 1.15.4 Кнопки внутри группы

Далее нам понадобится механизм с наследованием наименований кнопок, которые указали в справочнике с историей. Для этого перейдем в модуль формы и дополним процедуру «ПриСозданииНаСервере»:

&НаСервере

Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

История = Объект.БизнесПроцесс.История;

ЭтапИстории = Объект.ТочкаМаршрута.Имя;

ЭтотОбъект.Картинка = ПолучитьНавигационнуюСсылку(История, ЭтапИстории);

ЭтотОбъект.Комментарий = История[ЭтапИстории + "Комментарий"];

Если Объект.Выполнена = Истина Тогда

 Элементы.ВыполнитьИЗакреть.Видимость = Ложь;

КонецЕсли;

Если ЭтапИстории = "ЗавершениеИстории" ИЛИ ЭтапИстории = "ПлохоеЗавершениеИстории" Тогда

```
Элементы.ВыполнитьИЗакрыть.Заголовок = "Завершить историю";  
КонецЕсли;
```

```
// Проверка на каком этапе на данный момент игрок
```

```
//Если этап с выбором истории, то:
```

```
Если Объект.ТочкаМаршрута =  
БизнесПроцессы.НашиИстории.ТочкиМаршрута.ВыборВИстории Тогда  
    //Создадим новую процедуру на сервере, которая будет наследовать заголовки  
    из справочника  
    ПодготовитьКнопкиОтветов();  
    //Выключим видимость кнопки "Дальше"  
    Элементы.ВыполнитьИЗакрыть.Видимость = Ложь;  
Иначе  
    //Выключим видимость группы с ответами  
    Элементы.ГруппаОтветы.Видимость = Ложь;  
КонецЕсли;
```

```
КонецПроцедуры
```

Далее нужно описать вызванную процедуру.

Важно!

Если не указывать директивы &НаСервере или &НаКлиенте, то по умолчанию всегда будет &НаСервере.

Создадим ее чуть ниже, в том же модуле формы:

```
Процедура ПодготовитьКнопкиОтветов()  
Элементы.ОтветПлохой.Заголовок = Объект.БизнесПроцесс.История.ВариантДляПоражения;  
Элементы.ОтветХороший.Заголовок = Объект.БизнесПроцесс.История.ВариантДляПобеды;  
КонецПроцедуры
```

Проверим отображение кнопок в пользовательском режиме (рис. 1.15.5).



Рис. 1.15.5. Кнопки в пользовательском режиме

Если название кнопок наследуется правильно, то можно приступать к обработке события нажатия. Перед этим расположим кнопки в группе по центру с помощью свойства «ГоризонтальноеПоложениеВГруппе» (рис. 1.15.6).

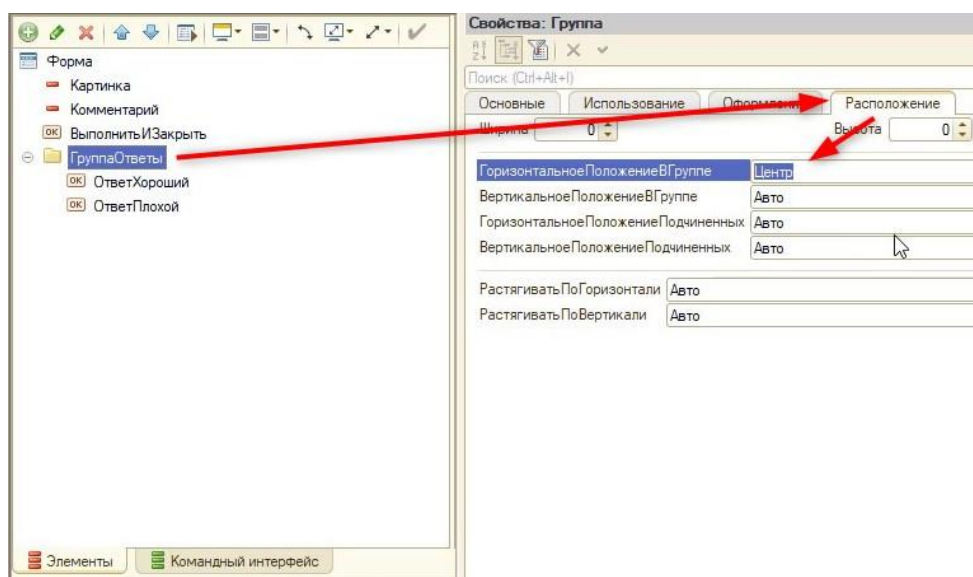


Рис. 1.15.6. Настройка центрального положения

Стандартно в программе 1С есть всплывающие подсказки для кнопок при наведении (рис. 1.15.7).

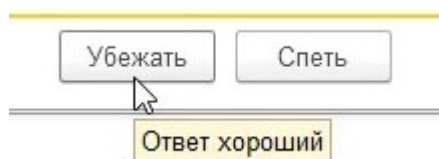


Рис. 1.15.7. Подсказка

Их необходимо отключить для всех кнопок на форме, чтобы не подсказывать игроку, какое действие приведет к хорошему исходу. Отключить их можно с помощью свойства кнопки – «ОтображениеПодсказки» в значении «Нет» (рис. 1.15.8).

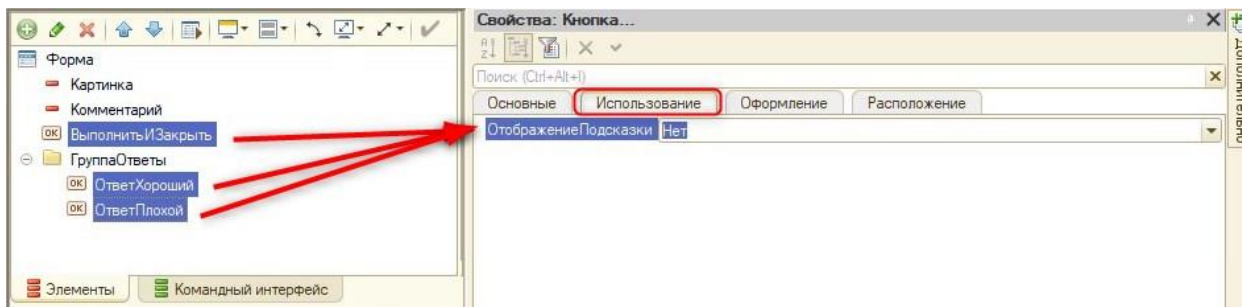


Рис. 1.15.8. Отключение подсказки

Теперь можно приступить к определению действий для кнопок. Код будет универсальным, поэтому сначала через свойства одной из кнопок создадим действие (рис. 1.15.9) на клиенте.

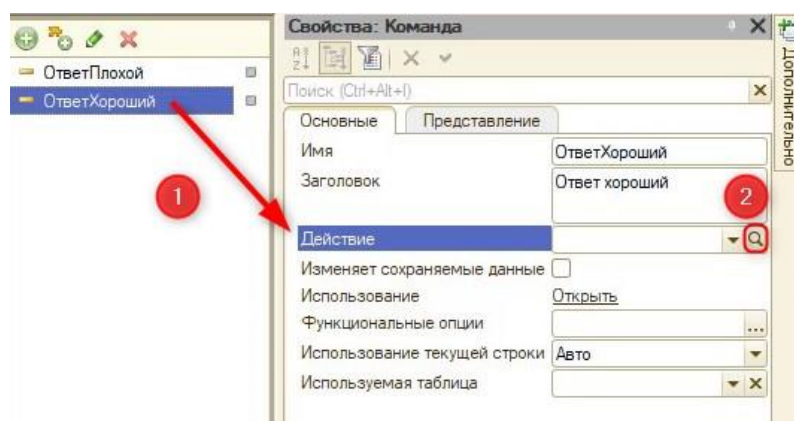


Рис. 1.15.9. Создание действия

После этого поменяем заголовок процедуры на «ОтветНаВыбор» (рис. 1.15.10) и перепривяжем заново для двух кнопок действие (рис. 1.15.11 – 1.15.12).

&НаКлиенте
Процедура ОтветНаВыбор(Команда)
// Вставить содержимое обработчика.
КонецПроцедуры

Рис. 1.15.10. Новое название процедуры

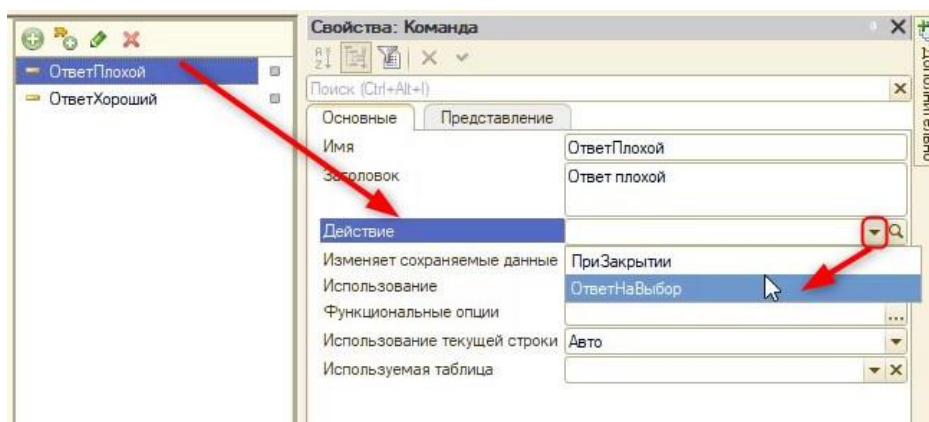


Рис. 1.15.11. Привязка действия для команды «Ответ плохой»

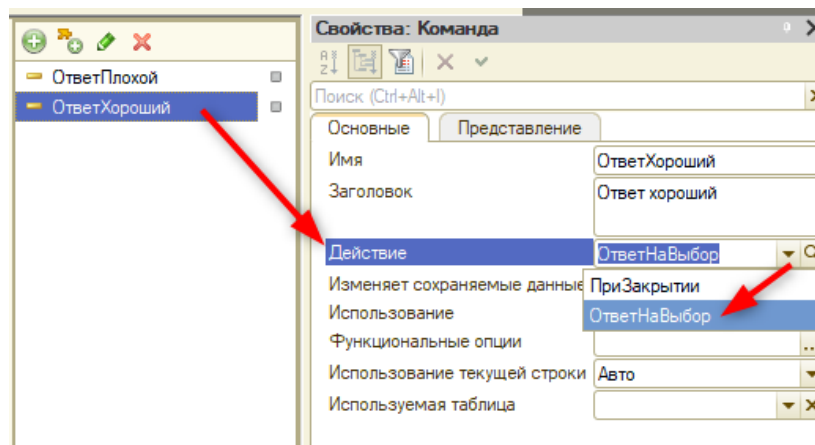


Рис. 1.15.12. Привязка действия для команды «Ответ хороший»

Далее приступим к программированию:

&НаКлиенте

Процедура ОтветНаВыбор(Команда)

//Работать с бизнес процессом можем только на сервере, поэтому создадим новую процедуру и вызовём её тут:

//В параметрах передадим название кнопки, на которую нажали и вызвали данный код.

ЗафиксироватьОтвет(ТекущийЭлемент.Заголовок);

//Закроем форму с выбором:

ЭтотОбъект.Заккрыть();

КонецПроцедуры

Создадим серверную процедуру «ЗафиксироватьОтвет»:

&НаСервере

//Обязательно добавим параметр, или иначе возникнет ошибка, так как мы передаем в процедуру, но параметра для получения нет.

Процедура ЗафиксироватьОтвет(Ответ)

//Получим ссылку на БП, чтобы можно было его редактировать

БП_Объект = Объект.БизнесПроцесс.ПолучитьОбъект();

//В реквизит БП "ОтветНаВопрос" подставим название кнопки, на которую нажали

БП_Объект.ОтветНаВопрос = Ответ;

//Запишем изменения:

БП_Объект.Записать();

//Теперь необходимо выполнить задачу, чтобы открыть доступ к следующей

//Чтобы получить возможность её выполнить нужно из контекста формы перейти в контекст объект:

Задача_Объект = РеквизитФормыВЗначение("Объект");

//Далее выполняем задачу:

Задача_Объект.ВыполнитьЗадачу();

КонецПроцедуры

Перед проверкой кода уберем с формы бизнес-процесса элемент «ОтветНаВопрос» (рис. 1.15.13).

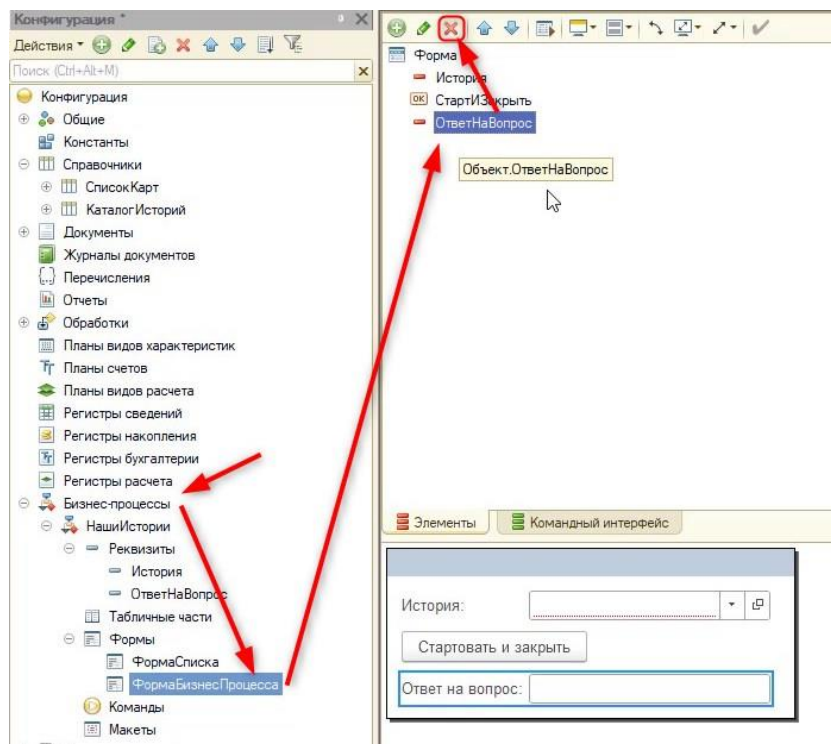


Рис. 1.15.13. Удаление элемента

При проверке в пользовательском режиме можно заметить, что список с задачами между выбором и завершением не обновляется автоматически. Так как в наших кнопках выбора, в отличие от кнопок «Выполнить и закрыть», нет обновления формы. Вызвать его можно, дополнив код процедуры «ОтветНаВыбор»:

&НаКлиенте

Процедура ОтветНаВыбор(Команда)

ЗафиксироватьОтвет(ТекущийЭлемент.Заголовок);

//Оповестим всю программу:

Оповестить("Обновить форму задач");

ЭтотОбъект.Закреть();

КонецПроцедуры

Далее нужно обработать данное оповещение. Откроем форму списка задач, обратимся к событию «ОбработкаОповещения» и создадим для него обработчик на клиенте (рис. 1.15.14).

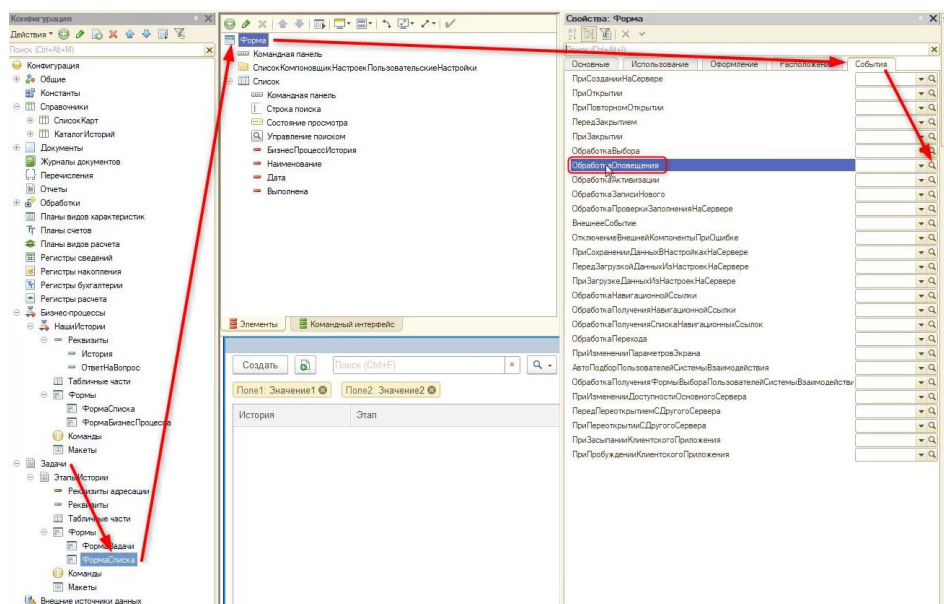


Рис. 1.15.14. Обработка

Код новой процедуры будет выглядеть следующим образом:

&НаКлиенте

Процедура ОбработкаОповещения(ИмяСобытия, Параметр, Источник)

Оповестить()
//Имя события должно совпадать точь в точь с описанным при вызове метода

Если ИмяСобытия = "Обновить форму задач" Тогда

 //Обновляем список элементов:

 Элементы.Список.Обновить();

КонецЕсли;

КонецПроцедуры

На этом создание интерактивных ответов для квеста завершено.

Практикум №2

Поменять заголовок у кнопки «Стартовать и закрыть» на «Начать игру» на форме бизнес-процесса (рис. 1.15.15).

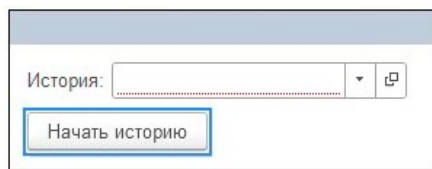


Рис. 1.15.14. Форма бизнес-процесса

Занятие 16 – Создание общего интерфейса

Одна из самых главных вещей в разработке игр или типовых решений – это интерфейс. Задача программиста – настроить интерфейс так, чтобы он был интуитивно понятен пользователю. Если для запуска игры нужна целая инструкция для игрока – это означает, что при составлении интерфейса были допущены ошибки.

Каждый разработчик к своим играм может сделать индивидуальное меню, в занятии рассматриваются механизмы, которые можно использовать для этого.

Для создания меню можно использовать прикладной объект «Общие формы». Общие формы – это формы, которые не принадлежат конкретным объектам (обработкам, справочникам и т. д.)

Создадим общую форму из дерева конфигурации и назовем ее «ФормаДляНачальногоЭкрана» (рис. 1.16.1).

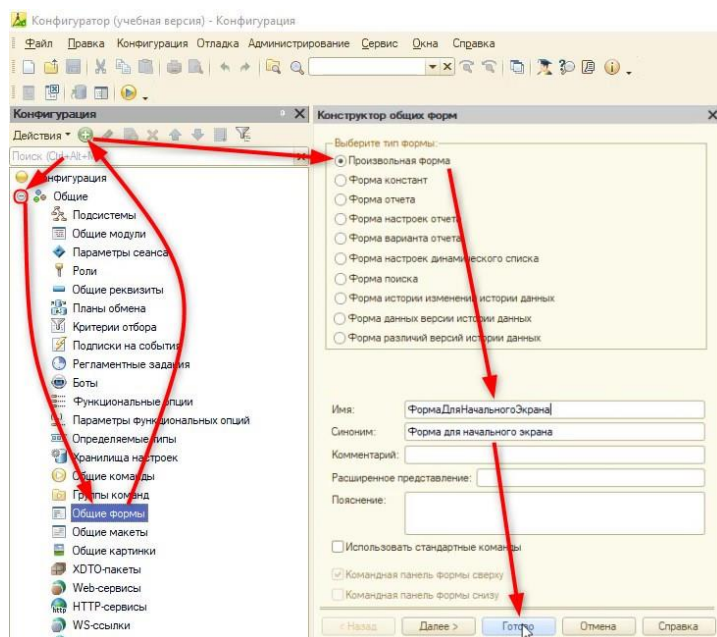


Рис. 1.16.1. Создание обработчика события

У формы не должно быть стандартной командной панели, поэтому выключим ее через свойства формы (рис. 1.16.2).

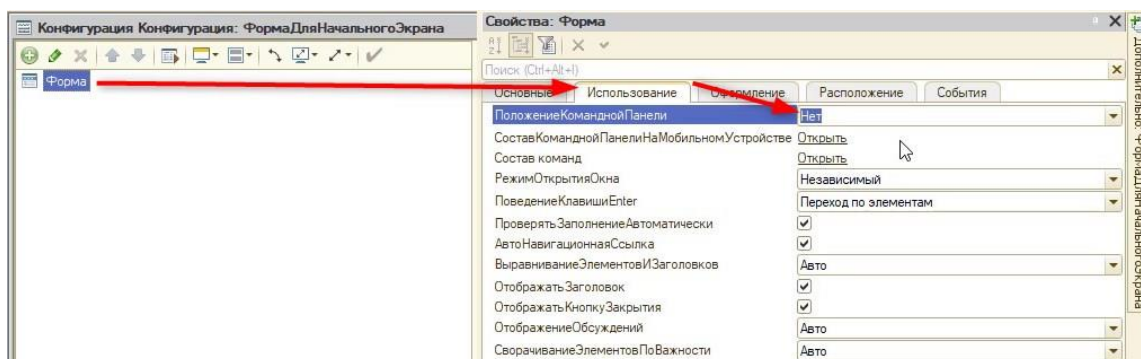


Рис. 1.16.2. Выключение командной панели

Часть кнопок, которые будут на интерфейсе начального экрана, можно получить через глобальные команды. Пример глобальных команд – это создание записи в справочнике, открытие обработок.

Вынесем две команды, которые открывают обработки с играми «Кубики» и «Карты», с помощью зажатия правой кнопки мыши. Эти команды находятся на закладке «Сервис» (рис. 1.16.3).

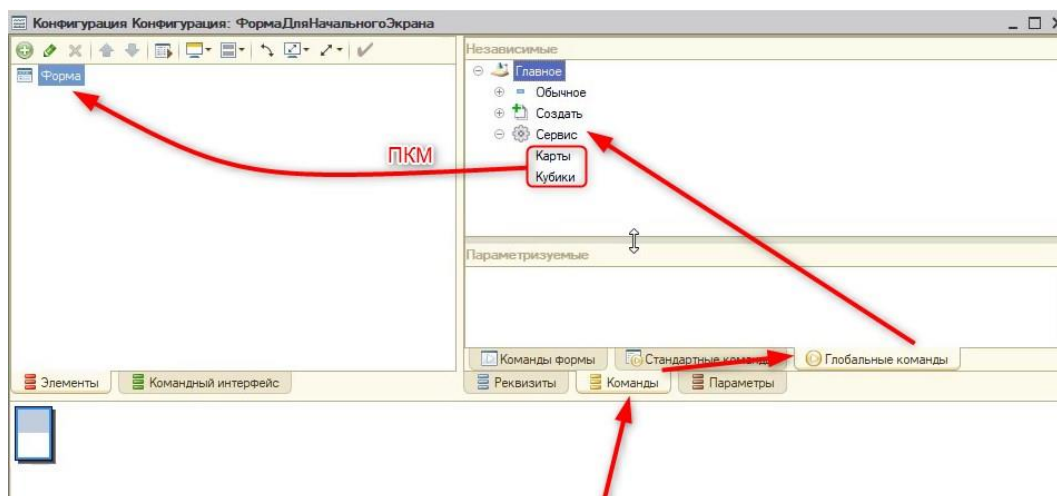


Рис. 1.16.3. Добавление кнопок

Заголовки новых кнопок необходимо поменять на «Играть в кубики» (рис. 1.16.4) и «Играть в карточки», чтобы игроку было понятно, что произойдет при нажатии на кнопку.

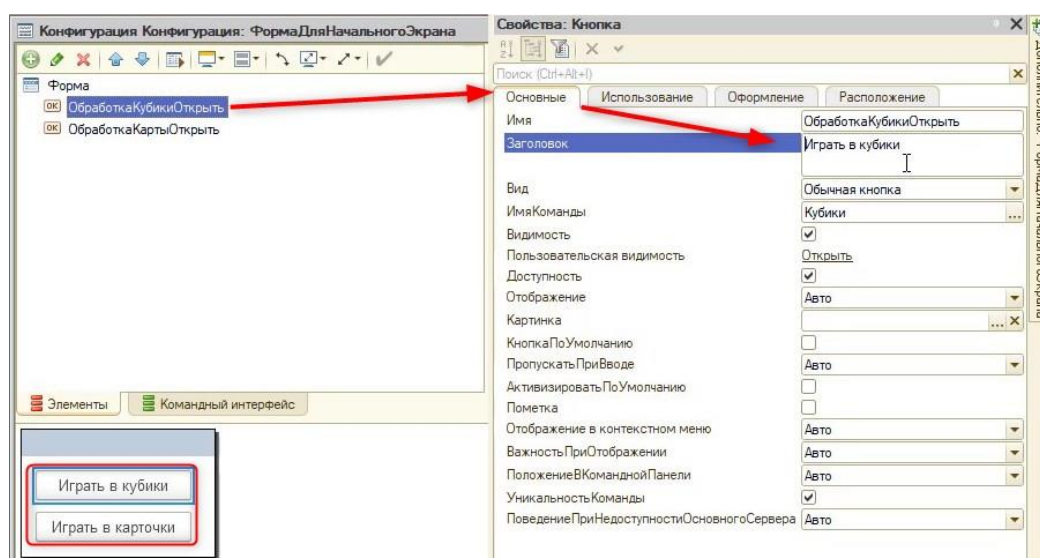


Рис. 1.16.4. Смена заголовков

С квестом немного иначе. Необходимо вынести на форму глобальную команду «Наша история: создать» (на вкладке «Создать») (рис. 1.16.5). Эта команда вызывает форму для создания нового бизнес-процесса (нового прохождения игры). Обязательно после добавления поменять заголовки кнопки.

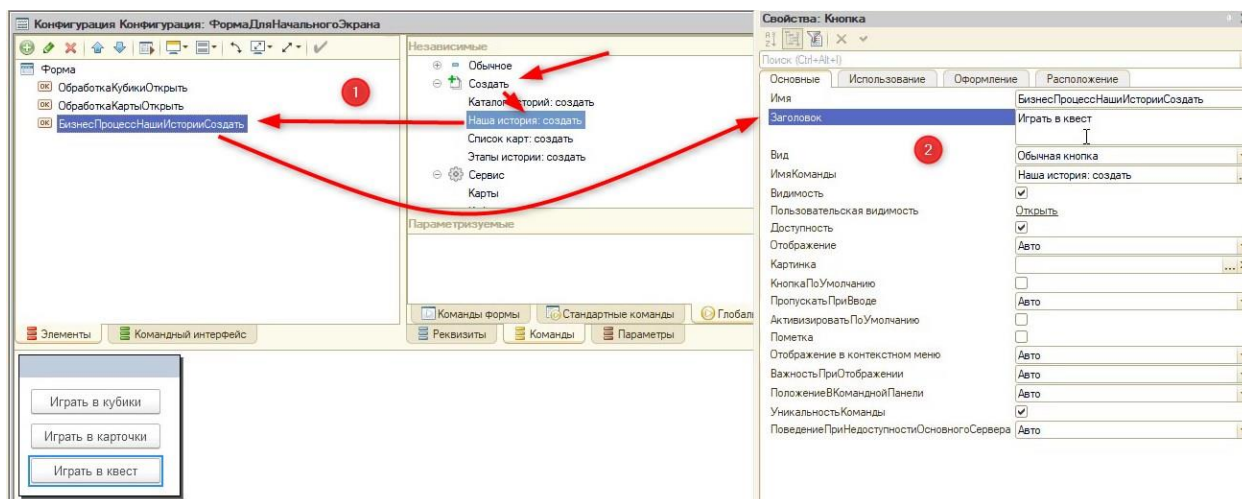


Рис. 1.16.5. Создание кнопки для игры в квест

Далее с помощью клавиши «Ctrl» выделим все три кнопки для настройки их свойств. Для начала уберем у них отображения подсказки (рис. 1.16.6).

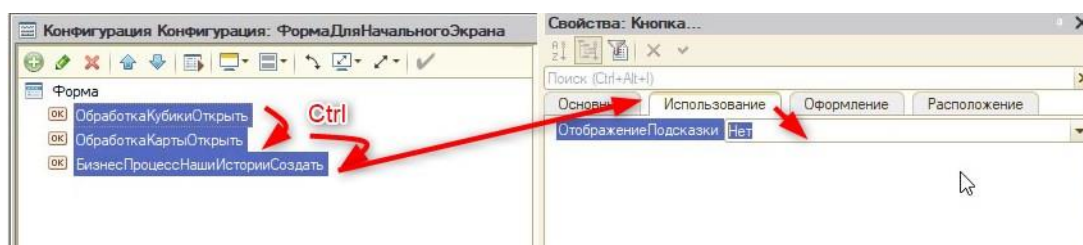


Рис. 1.16.6. Отображение подсказки

Далее на вкладке «Оформление» можно настроить дизайн кнопки (шрифт, фигуру, цвет текста и рамки). К примеру, поменяем шрифт и размер на другие, предварительно сняв галочку «Из стиля» (рис. 1.16.7).

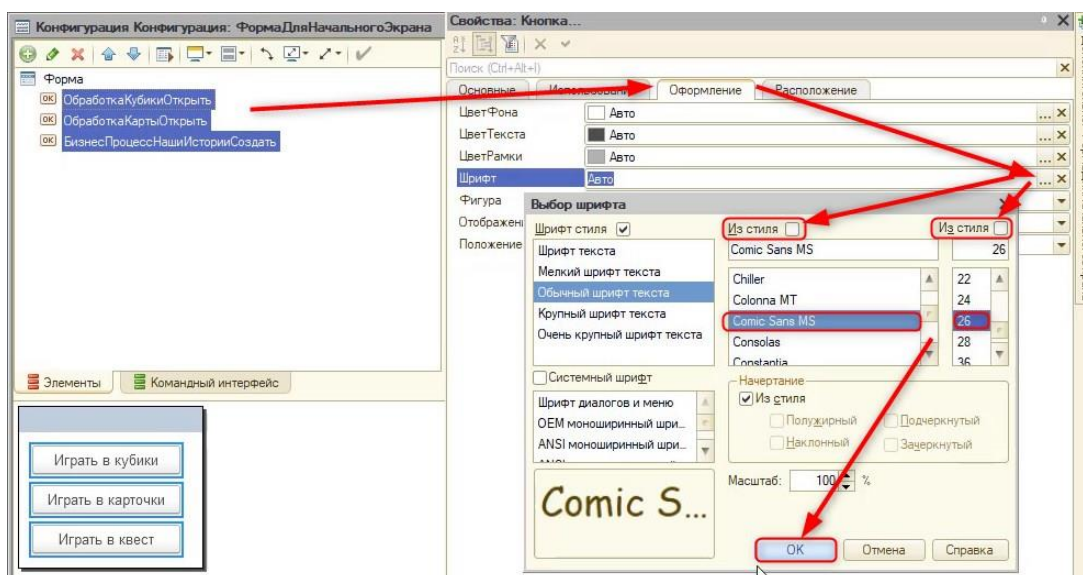


Рис. 1.16.7. Редактирование шрифта кнопки

На этой же вкладке можно настроить форму фигуры, сменив ее на овальную (рис. 1.16.8).

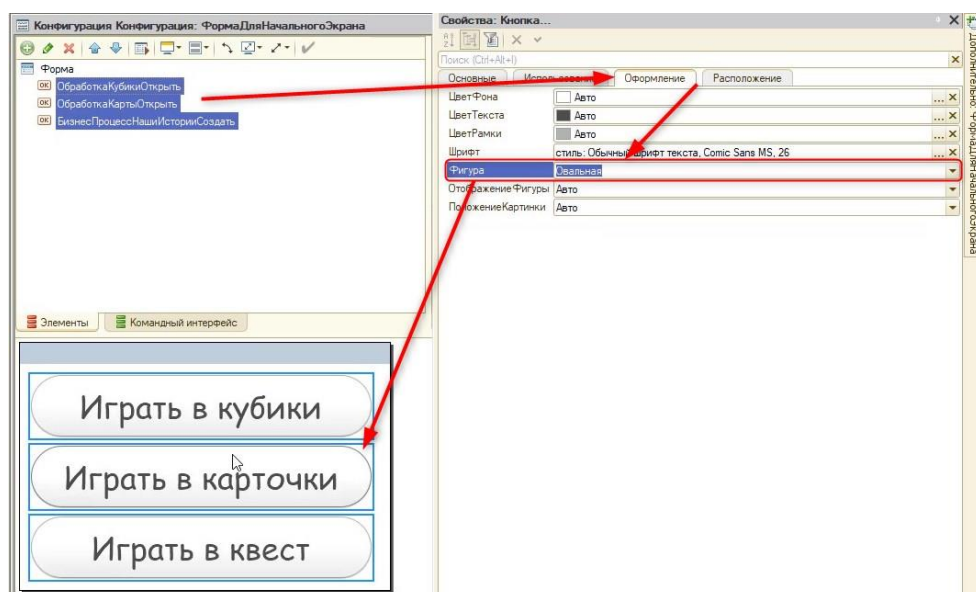


Рис. 1.16.8. Смена фигуры

Поменяем цвет фона кнопок (рис. 1.16.9). К примеру, на желто-оранжевый.

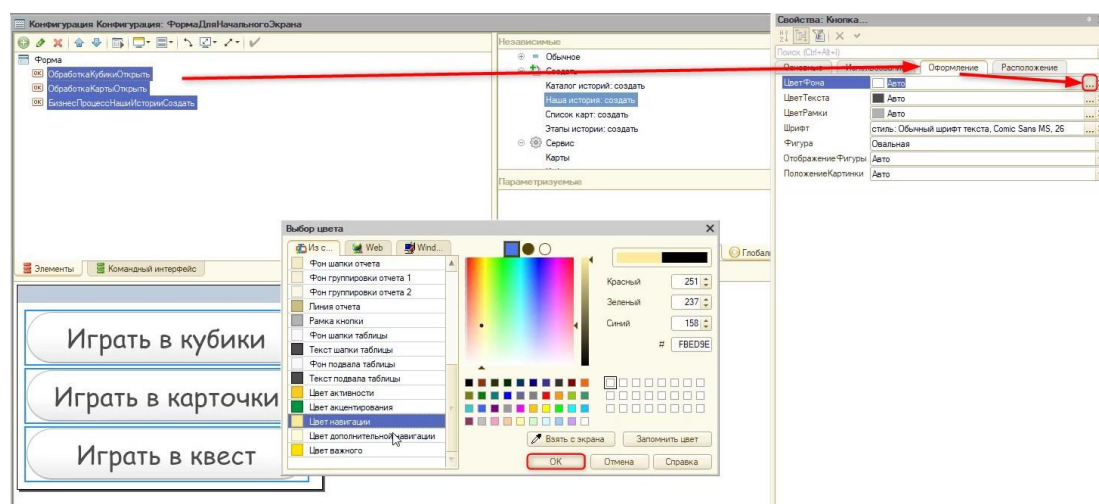


Рис. 1.16.9. Редактирование цвета фона

Помимо стилистики, можно изменить размеры кнопок. Для этого нужно перейти на вкладку свойств «Расположение». Сделать это можно разными способами – задать ручную ширину и высоту, или включить флаг «Растягивать по горизонтали». Можно использовать и первый способ, и второй.

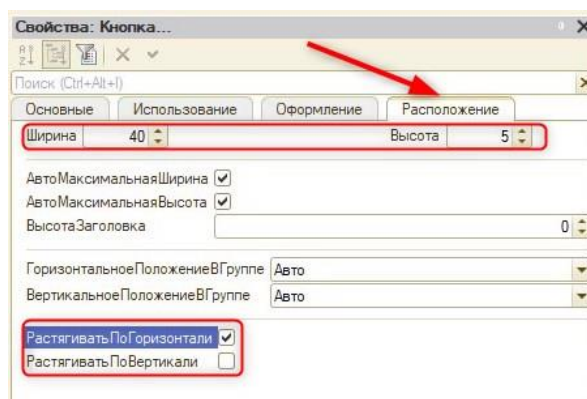


Рис. 1.16.10. Редактирование размера кнопок

Для того чтобы форма отобразилась в интерфейсе, необходимо обратиться к конфигурации в дереве и после нажатия левой кнопки мыши выбрать «Открыть рабочую область начальной страницы» (рис. 1.16.11).

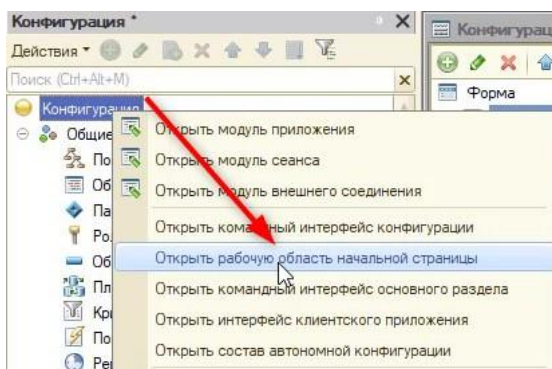


Рис. 1.16.11. Открытие редактора рабочей области начальной страницы

Добавим в левую или в правую колонку с помощью кнопки «+» общую форму для начального экрана (рис. 1.16.12).

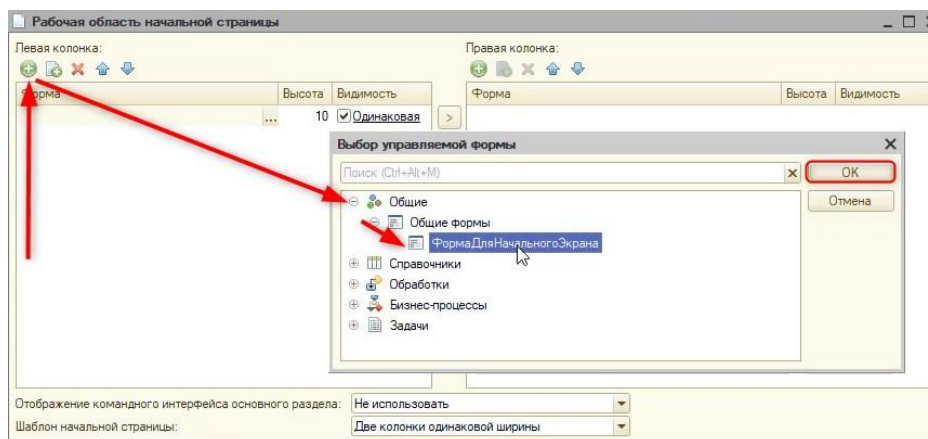


Рис. 1.16.12. Добавление формы на начальный экран

После этого закроем редактор рабочей области.

Далее, чтобы пользователю не отображалась надпись «Форма для начального экрана», выключим флаг «АвтоЗаголовок» в свойствах формы (рис. 1.16.13).

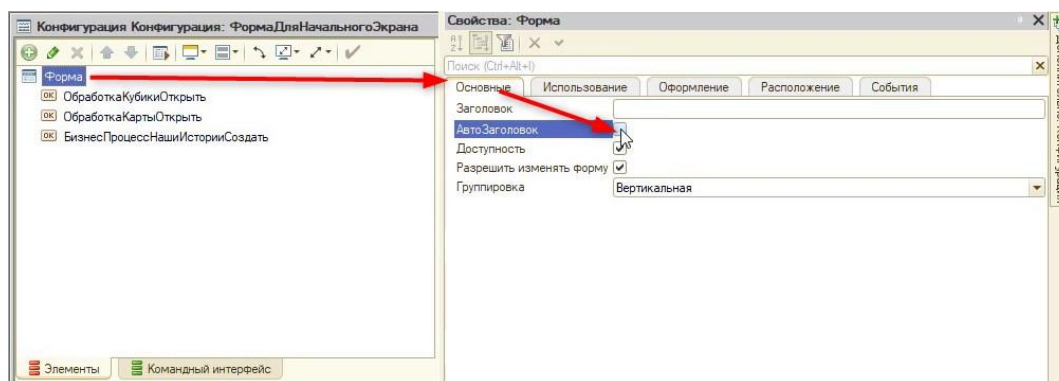


Рис. 1.16.13. Выключение заголовка

Перед тем как двигаться дальше, проверим работоспособность текущего меню (рис. 1.16.14).

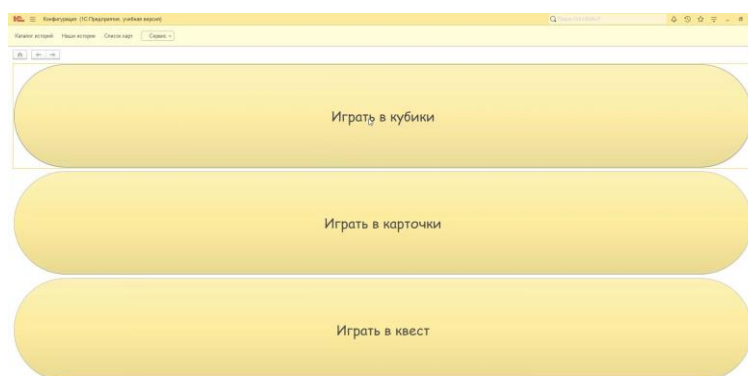


Рис. 1.16.14. Меню начального экрана

Помимо кнопок, можно добавить картинку через добавление нового элемента «Декорация-Картинка» (рис. 1.16.15).

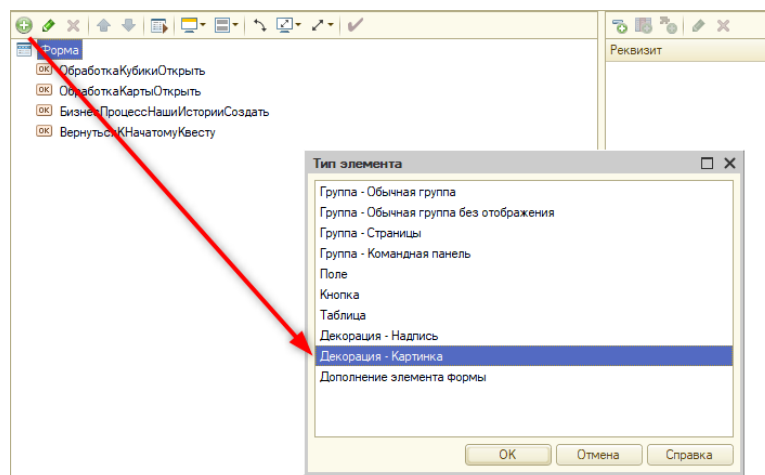


Рис. 1.16.15. Добавление декорации

Само изображение прикрепляется через свойство элемента «Картинка» (рис. 1.16.16).

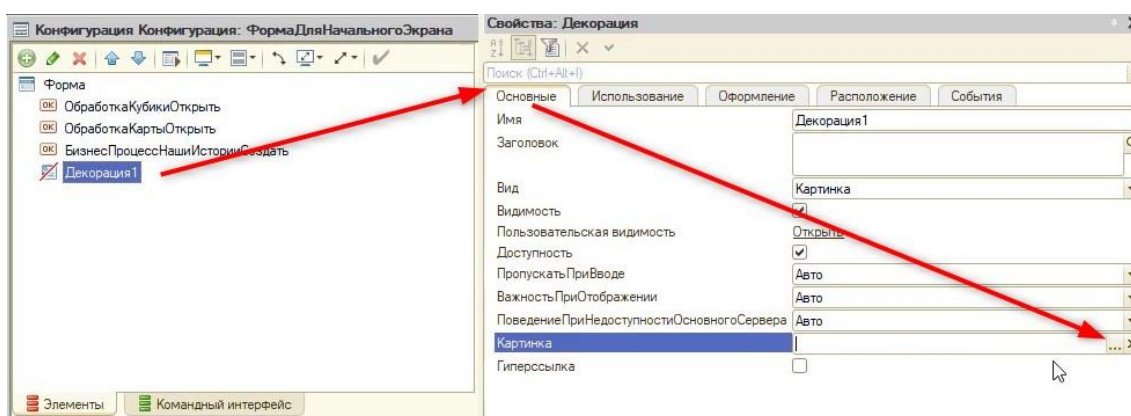


Рис. 1.16.16. Прикрепление картинки

В окне выбора есть два варианта:

- 1) Прикрепить изображение из библиотеки внутри программы (рис. 1.16.17);

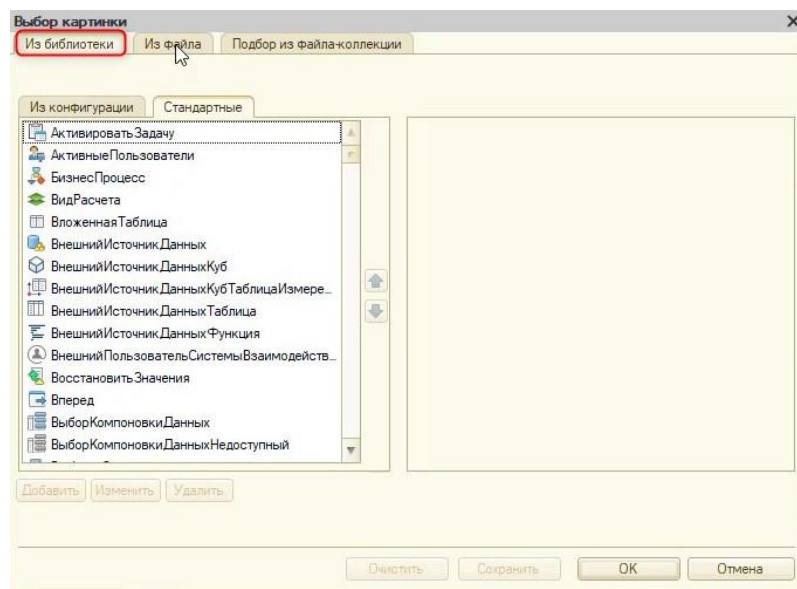


Рис. 1.16.17. Прикрепление картинки из библиотеки

2) Прикрепить собственное изображение с компьютера (рис. 1.16.18).

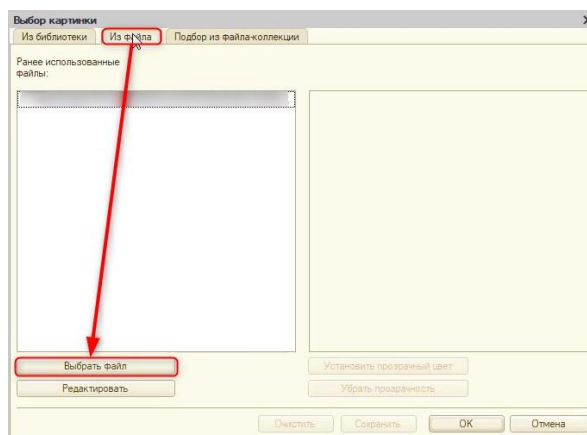


Рис. 1.16.18. Прикрепление картинки из файла

Помимо создания нового прохождения игры «Квест», игрок может захотеть пройти ранее незаконченную игру. В этом случае нужно будет программно получить список незавершенных историй (бизнес-процессов). Для этого создадим отдельную команду «ВернутьсяКНачатомуКвесту» и расположим ее на форме (рис. 1.16.19).

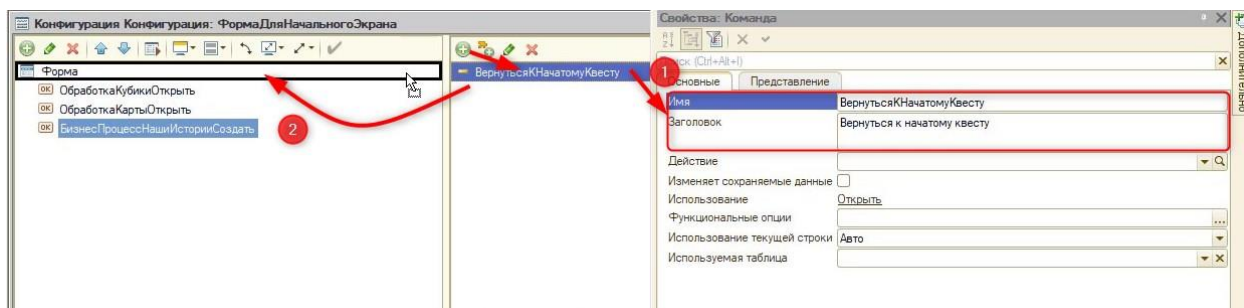


Рис. 1.16.19. Создание новой команды и кнопки

К ней необходимо применить настройки, аналогичные тем, что мы применяли к другим командам в начале занятия. После этого необходимо определить действие для новой команды на клиенте:

&НаКлиенте

Процедура ВернутьсяКНачатомуКвесту(Команда)

```

//Для начала нужно выяснить, есть ли не законченные квесты в программе
//Создадим новую серверную функцию (так как обращаться будем к базе данных)
ПолучитьСписокКвестов()

//И полученный результат запишем в переменную СписокКвестов
СписокКвестов = ПолучитьСписокКвестов();

КонецПроцедуры

```

Далее создадим новую серверную функцию, которую вызывали в процедуре:

```

&НаСервере
Функция ПолучитьСписокКвестов()

//Создадим новый список значений, в который будут записываться не пройденные до конца
квесты.

СписокБП = Новый СписокЗначений;

КонецФункции

```

После этого воспользуемся конструктором запроса с обработкой результата (рис. 1.16.20).

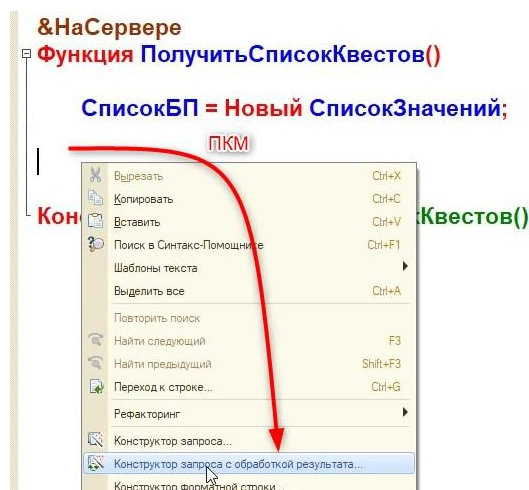


Рис. 1.16.20. Вызов конструктора запросов

Тип обработки – обход результата. На вкладке «Таблицы и поля» укажем, что необходимо получить ссылки из бизнес-процесса (рис. 1.16.21).

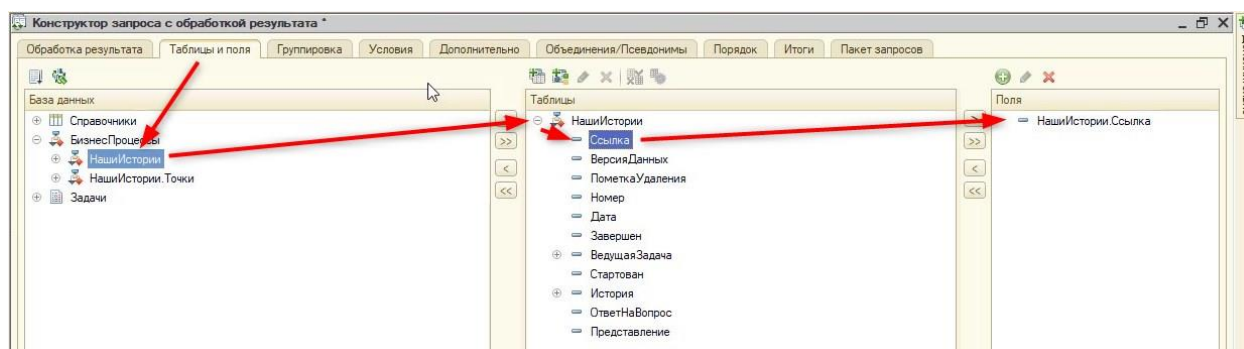


Рис. 1.16.21. Вкладка «Таблицы и поля»

Эти ссылки нужно получать с условием, что значение реквизита «Завершен» должно быть «ЛОЖЬ» (рис. 1.16.22).

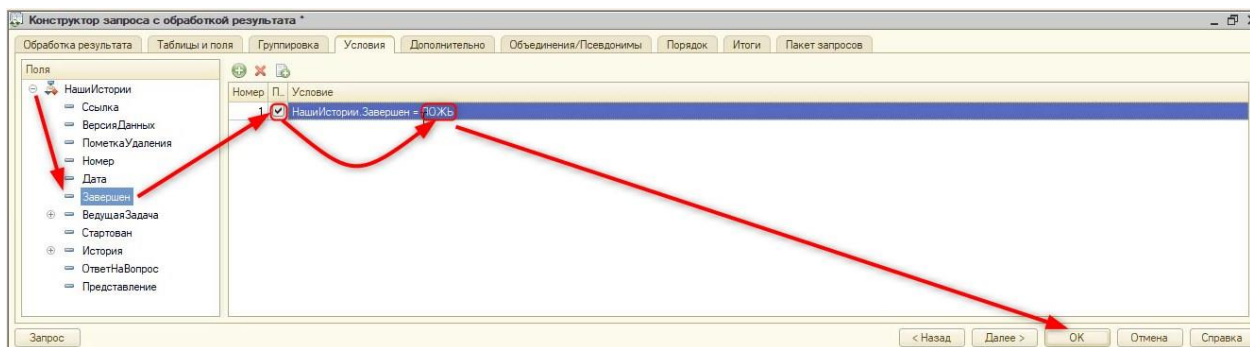


Рис. 1.16.22. Вкладка «Условия»

После этого закроем конструктор и подредактируем код:

&НаСервере

Функция ПолучитьСписокКвестов()

СписокБП = Новый СписокЗначений;

Запрос = Новый Запрос;

Запрос.Текст =

"ВЫБРАТЬ

| НашиИстории.Ссылка КАК Ссылка

|ИЗ

| БизнесПроцесс.НашиИстории КАК НашиИстории

|ГДЕ

| НашиИстории.Завершен = ЛОЖЬ";

РезультатЗапроса = Запрос.Выполнить();

//Проверка на пустой результат:

Если НЕ РезультатЗапроса.Пустой() Тогда

Выборка = РезультатЗапроса.Выбрать();

//Записываем по шаблону номер истории и название в список значений:

Пока Выборка.Следующий() Цикл

Представление = СтрШаблон("История %1. Тема: %2",
Выборка.Ссылка.Номер, Выборка.Ссылка.История);

СписокБП.Добавить(Выборка.Ссылка, Представление);

КонецЦикла;

КонецЕсли;

//Возвращаем список:

Возврат СписокБП;

КонецФункции

Вернемся к действию кнопки:

&НаКлиенте

//Так как в процедуре будем использовать асинх методы, необходимо указать "АСИНХ" в шапке процедуры

АСИНХ Процедура ВернутьсяКНачатомуКвесту(Команда)

```

        СписокКвестов = ПолучитьСписокКвестов();
        //Выведем окно выбора пользователю, в котором будут все полученные запросом не
        выполненные бизнес-процессы
        Выбор = ЖДАТЬ СписокКвестов.ВыбратьЭлементАсинх();
        //Обработаем ситуацию, если пользователь выбрал нужный ему квест
        Если Выбор <> Неопределено Тогда
            //Получаем и открываем форму, как делали ранее
            СтруктураОтбора = Новый Структура("Ключ", Выбор.Значение);
            Форма = ПолучитьФорму("БизнесПроцесс.НашиИстории.ФормаОбъекта",
            СтруктураОтбора);
            ОткрытьФорму(Форма);
        КонецЕсли;
    КонецПроцедуры

```

В пользовательском режиме работа кода выглядит следующим образом (см. рис. 1.16.23).

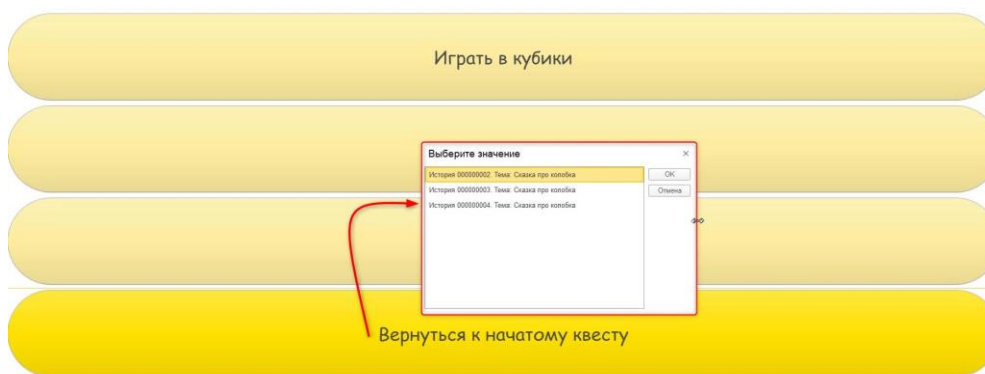


Рис. 1.16.23. Работа новой кнопки

Если в списке будут отсутствовать значения, значит, незаконченных квестов нет. В таком случае было бы лучше, если бы эта кнопка отсутствовала в интерфейсе. Для этого определим событие «ПриСозданииНаСервере» в свойствах формы начального экрана (рис. 1.16.24).

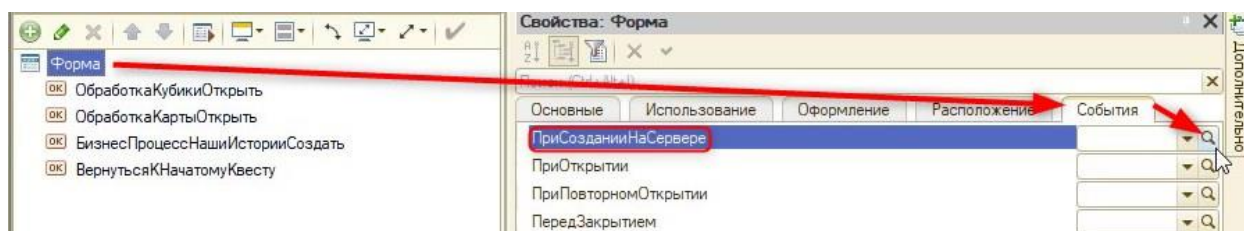


Рис. 1.16.24. Создание нового обработчика

В новой процедуре получим список ссылок невыполненных квестов и обработаем ситуацию, когда список пуст:

&НаСервере

Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

//Получим список ссылок с помощью функции, созданной ранее:

СписокКвестов = ПолучитьСписокКвестов();

//Обработаем ситуацию, когда список пуст

Если СписокКвестов.Количество() = 0 Тогда

//В этом случае скроем видимость кнопки "Вернуться к начатому квесту"

Элементы.ВернутьсяКНачатомуКвесту.Видимость = Ложь;

КонецЕсли;

КонецПроцедуры

Теперь осталось дело за малым – уберем из интерфейса с панели все ненужные кнопки с помощью командного интерфейса основного раздела (рис. 1.16.25 – 1.16.26).

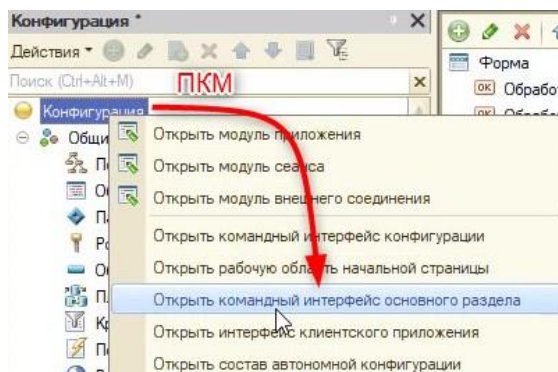


Рис. 1.16.25. Открытие настроек командного интерфейса

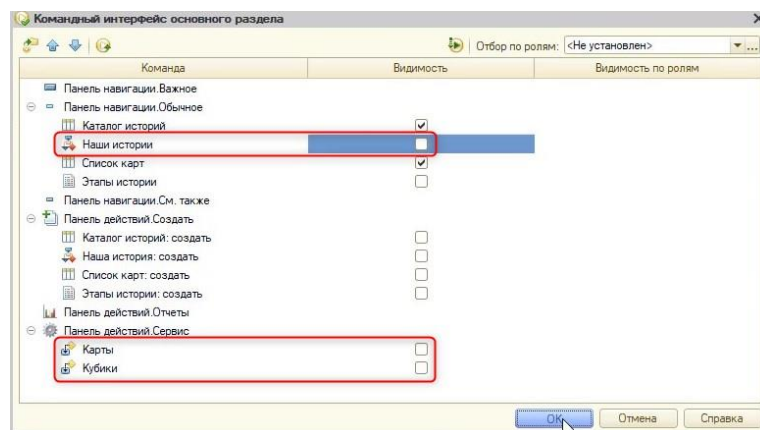


Рис. 1.16.26. Настройка командного интерфейса

Отлично, на этом изучение курса завершено!